



Pontificia Universidad Católica de Valparaíso
Escuela de Informática

Taller Formulación de Proyectos Informáticos

ICI-5444

Antonio Moya Villegas
antonio.moya@pucv.cl

v3.1.0 - 2026

Arquitectura de Software

El recorrido

Vamos a construir un mismo razonamiento en diez pasos. Al final de este recorrido usted debería poder dibujar la arquitectura de su solución y defender por qué es esa y no otra.

1. Fundamentos

Qué es y qué no es arquitectura. Atributos de calidad y vistas.

2. Arquitectura lógica

Monolito, capas, microservicios, eventos y la capa de integración con terceros.

3. Arquitectura física

Modelo OSI, redes, data center, contenedores, datos y almacenamiento.

4. La nube

IaaS/PaaS/SaaS, regiones, serverless, orquestación, on-premise vs nube.

5. Atributos de calidad

Escalabilidad, disponibilidad, rendimiento y el nivel de servicio que se firma.

Arquitectura de Software

Fundamentos

Lógica

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

El recorrido · continuación 1

6. Continuidad

Activo-activo y activo-pasivo, sincronización entre sitios y respaldos.

7. Seguridad

Soluciones expuestas, red interna, home office y portales públicos.

8. Entrega

Ambientes de desarrollo, QA, preproducción y producción; CI/CD.

9. Tendencias

Plataformas, IA, arquitecturas de datos, FinOps, sostenibilidad.

10. Su propuesta

Cómo bajar todo lo anterior a la oferta técnico-económica.

Arquitectura de Software

Por qué esto importa en el curso

En esta asignatura ustedes son una empresa que responde a una licitación. La arquitectura es la bisagra entre lo que prometen y lo que cuesta.

Define el alcance real

Los módulos e integraciones que dibuje son los que después tendrá que construir, probar y cobrar.

Determina el costo

Servidores, licencias, enlaces, respaldos y horas de operación salen del diagrama físico, no de la intuición.

Fija los riesgos

Un punto único de falla o una dependencia externa mal evaluada es un riesgo que el evaluador va a buscar.

Es lo que se compara

Cuando varias ofertas cumplen lo mínimo, gana la que fundamenta mejor sus decisiones técnicas.



Resultado de Aprendizaje 1: construir la arquitectura lógica y física de la solución del caso, con estándares vigentes y las restricciones organizacionales, económicas y ambientales del contexto.

Arquitectura de Software

Lo que se le pedirá en el Informe y Presentación 1

La pauta del curso es explícita sobre lo que debe aparecer. Esta clase entrega los parámetros para los tres ítems del centro.

Ítem del informe	Qué se espera	¿Lo cubre esta clase?
La empresa	Quiénes son, capacidades y experiencia declarada.	No
El problema	Situación actual del mandante y necesidad a resolver.	No
Esquema de solución	Idea general de la solución propuesta.	Parcial
Alcance de la solución	Qué queda dentro y qué queda fuera del contrato.	Parcial
Arquitectura lógica	Módulos, interfaces e integraciones, trazables a las bases técnicas.	Sí
Arquitectura física	Servidores, redes, seguridad, tamaño de BD, uptime y tiempos de respuesta.	Sí
Innovaciones	Elementos diferenciadores y su justificación.	Sí

Tiempo de presentación: 15 minutos + 15 minutos de preguntas y discusión.

01

SECCIÓN 1

Fundamentos

- Qué es —y qué no es— la arquitectura de software
- De qué se hace cargo el arquitecto: módulos, responsabilidades, interacción, ubicación en el hardware
- Atributos de calidad: el modelo ISO/IEC 25010
- Las vistas de una arquitectura y la distinción lógica / física
- Cómo se documenta y se justifica una decisión arquitectónica

Arquitectura de Software

El concepto

El concepto de arquitectura se usa de forma amplia y en campos muy distintos, por lo que su significado es algo... difuso.

En construcción

El arquitecto decide dónde van los muros estructurales. Después se puede cambiar la pintura; mover un muro cuesta.

En hardware

Arquitectura de un procesador: qué componentes hay y cómo se comunican entre sí.

En una organización

Arquitectura empresarial: cómo se relacionan procesos, datos, aplicaciones y tecnología.

En software

Los elementos más importantes del sistema y sus relaciones: una visión global.



En todos los casos aparece lo mismo: decisiones de estructura, tomadas temprano, que condicionan todo lo que viene después y son costosas de revertir.

Arquitectura de Software

Definición

En el campo del software, la arquitectura identifica los elementos más importantes de un sistema, así como sus relaciones. Es decir, nos da una visión global del sistema.

En un sentido amplio, la Arquitectura del Software es el diseño de más alto nivel de la estructura de un sistema, programa o aplicación, y tiene la responsabilidad de:

Módulos

Definir los módulos principales del sistema.

Responsabilidades

Definir de qué se hace cargo cada uno de esos módulos.

Interacción

Definir cómo se relacionan e integran entre sí.

Control y datos

Definir el flujo de control y el flujo de datos.

Protocolos

Definir los protocolos de interacción y comunicación.

Ubicación

Definir dónde se ejecuta cada cosa en el hardware.

Arquitectura de Software

No responde sólo a requisitos estructurales

Las arquitecturas de software no responden únicamente a requisitos estructurales: están relacionadas con aspectos de rendimiento, usabilidad, reutilización, restricciones económicas y tecnológicas, e incluso cuestiones estéticas.

Rendimiento

Tiempos de respuesta y capacidad de proceso.

Usabilidad

Cómo se expone el sistema a quien lo usa.

Reutilización

Qué se compra, qué se reutiliza y qué se construye.

Restricciones económicas

Presupuesto de inversión y de operación anual.

Restricciones tecnológicas

Plataformas, estándares y sistemas heredados del mandante.

Contexto y normativa

Ubicación de los datos, cumplimiento legal, impacto ambiental.

Arquitectura de Software

El objetivo real

El objetivo principal de la Arquitectura del Software es aportar elementos que ayuden a la toma de decisiones y, al mismo tiempo, proporcionar conceptos y un lenguaje común que permitan la comunicación entre los equipos que participen en un proyecto.

Decidir

- ¿Compramos, arrendamos o construimos?
- ¿On-premise, nube o híbrido?
- ¿Un solo sistema o servicios separados?
- ¿Cuánta disponibilidad estamos dispuestos a pagar?
- ¿Qué se integra con los sistemas que ya tiene el mandante?

Comunicar

- Al equipo de desarrollo: qué construir y con qué límites.
- Al área de operaciones: qué van a tener que mantener.
- Al mandante: qué recibe y con qué niveles de servicio.
- Al evaluador de la licitación: por qué esta solución y no otra.
- A quien financia: en qué se gasta la inversión.

Arquitectura de Software

Tres cosas que se confunden

	Arquitectura	Diseño detallado	Infraestructura
Pregunta que responde	¿Cuáles son las partes grandes y cómo se relacionan?	¿Cómo se implementa cada parte por dentro?	¿Sobre qué equipos y redes corre todo esto?
Nivel	Más alto nivel del sistema	Interior de un módulo o componente	Recursos físicos y de red
Artefactos típicos	Diagrama de componentes, de contexto, de despliegue, decisiones registradas	Diagrama de clases, de secuencia, modelo de datos detallado	Topología de red, inventario de servidores, esquema de respaldo
Quién decide	Arquitecto con el negocio	Equipo de desarrollo	Operaciones / infraestructura
Costo de cambiarlo	Alto: afecta a todo el proyecto	Medio: contenido en un módulo	Variable: alto on-premise, bajo en la nube



En su informe, la sección de arquitectura debe contener las tres, claramente separadas. Un diagrama de clases no es una arquitectura lógica y una lista de servidores no es una arquitectura física.

Arquitectura de Software

Las decisiones son el producto

Una arquitectura no es un dibujo: es un conjunto de decisiones con consecuencias. Documentar la decisión vale más que documentar el dibujo.

Registro de decisión arquitectónica (ADR)	Ejemplo
Contexto	El mandante opera en dos regiones y exige que los datos personales permanezcan en Chile.
Decisión	Se despliega en nube pública con región en Chile; el respaldo se replica dentro del mismo país.
Alternativas evaluadas	(a) Data center propio del mandante, (b) nube pública con región extranjera, (c) esquema híbrido.
Fundamento	Menor inversión inicial, elasticidad ante la estacionalidad del negocio y cumplimiento de la normativa de datos personales.
Consecuencias	Costo mensual variable, dependencia del proveedor, necesidad de control de gasto desde el primer mes.
Referencia	Documentación del proveedor y normativa citada en norma APA 7.ª ed.

Tres o cuatro ADR bien escritos sostienen todo el capítulo de arquitectura de la oferta.

Arquitectura de Software

Atributos de calidad · ISO/IEC 25010

Los atributos de calidad son los requisitos que no se ven en un caso de uso, pero que definen la arquitectura. El modelo ISO/IEC 25010:2023 los organiza en nueve características.

Idoneidad funcional

Complejidad, corrección y pertinencia de lo que el sistema hace.

Eficiencia de desempeño

Comportamiento temporal, uso de recursos, capacidad.

Compatibilidad

Coexistencia con otros sistemas e interoperabilidad.

Capacidad de interacción

Aprendizaje, operabilidad, accesibilidad e inclusión (antes: usabilidad).

Fiabilidad

Madurez, disponibilidad, tolerancia a fallos y recuperabilidad.

Seguridad

Confidencialidad, integridad, no repudio, autenticidad y resistencia.

Mantenibilidad

Modularidad, reutilización, analizabilidad, modificabilidad, testeabilidad.

Flexibilidad

Adaptabilidad, instalabilidad, reemplazabilidad y escalabilidad (antes: portabilidad).

Seguridad operacional

Comportamiento seguro ante fallas y limitación del daño (característica nueva).

Arquitectura de Software

Los atributos que se compran y se venden

De las nueve características, hay cinco que casi siempre aparecen en las bases técnicas —y que se pagan.

Atributo	Cómo se mide	Cómo se materializa en la arquitectura	Impacto en el costo
Rendimiento	Tiempo de respuesta (p95), transacciones por segundo	Caché, índices, balanceo, dimensionamiento de CPU/RAM	Medio
Disponibilidad	Uptime comprometido (99,5% / 99,9% / 99,99%)	Redundancia, múltiples zonas, failover automático	Alto
Escalabilidad	Usuarios concurrentes soportados sin degradación	Escalamiento horizontal, autoescalado, servicios sin estado	Medio
Seguridad	Controles implementados, resultado de pruebas	Segmentación de red, WAF, cifrado, gestión de identidades	Alto
Mantenibilidad	Tiempo medio de reparación, esfuerzo de cambio	Modularidad, contratos de interfaz claros, pruebas automatizadas	Se paga después



Regla práctica: cada nueve adicional de disponibilidad multiplica la complejidad y el costo de la arquitectura. No comprometa 99,99% si el caso no lo exige.

Arquitectura de Software

Cómo se escribe un atributo de calidad

Así NO se escribe

- «El sistema será rápido.»
- «El sistema será altamente disponible.»
- «El sistema será seguro.»
- «El sistema soportará muchos usuarios.»
- «La solución será escalable.»

Así SÍ se escribe

- «El 95% de las consultas de saldo responde en menos de 2 segundos con 500 usuarios concurrentes.»
- «El servicio mantiene 99,9% de disponibilidad mensual medido sobre el horario hábil.»
- «Todo dato personal viaja y se almacena cifrado; el acceso queda registrado y es auditable por 12 meses.»
- «Ante un aumento de carga del 100%, el sistema agrega instancias en menos de 5 minutos sin intervención manual.»



Plantilla: ante [estímulo], en [condición de operación], el sistema [respuesta] medida en [métrica y valor]. Si no tiene número, no es un requisito: es una intención.

Arquitectura de Software

Las vistas de una arquitectura

Ninguna vista muestra el sistema completo. Cada una responde una pregunta distinta y se dirige a un público distinto.

Vista lógica

Describe el modelo de objetos y la funcionalidad.

Responde: ¿qué hace el sistema y cómo se divide?

Público: usuario y analista.

Vista de proceso

Muestra la concurrencia y la sincronía entre procesos.

Responde: ¿qué corre en paralelo y cómo se coordina?

Público: integrador.

Vista de desarrollo

Describe la organización del entorno de desarrollo.

Responde: ¿cómo se organiza el código y los equipos?

Público: desarrollo.

Vista física

Muestra la ubicación del software en el hardware.

Responde: ¿dónde se ejecuta cada cosa?

Público: operaciones.

+1 ESCENARIOS · Los casos de uso principales recorren y validan las cuatro vistas: si un escenario crítico no se puede trazar en todas, la arquitectura está incompleta.

Arquitectura de Software

Arquitectura lógica vs. arquitectura física

ARQUITECTURA LÓGICA

- Organiza el software: módulos, componentes, capas, servicios.
- Muestra interfaces, integraciones y flujo de información.
- Es independiente de la marca del servidor o del proveedor de nube.
- Se traza contra los requisitos de las bases técnicas.
- Artefactos: diagrama de componentes, de contexto, de módulos.

ARQUITECTURA FÍSICA

- Organiza la ejecución: nodos, servidores, redes, almacenamiento.
- Muestra dónde se despliega cada componente lógico.
- Depende de tecnología concreta, capacidad y ubicación geográfica.
- Se dimensiona con parámetros verificables (usuarios, volumen, uptime).
- Artefactos: diagrama de despliegue, topología de red, inventario.

El puente entre ambas es una tabla de correspondencia: «el módulo de Facturación (lógico) se despliega en el clúster de aplicaciones, 2 instancias de 4 vCPU y 8 GB, detrás del balanceador (físico)».

Sección 1 · Fundamentos

1

Modelo 4+1 de Kruchten

Lea el artículo original «Architectural Blueprints: The 4+1 View Model» y aplique las cinco vistas a un sistema que ya conozca.

2

ISO/IEC 25010:2023

Revise las nueve características y sus subcaracterísticas en [iso25000.com](https://www.iso25000.com). Identifique las cinco que pesan en su caso.

3

Registros de decisión (ADR)

Busque plantillas de Architecture Decision Record y escriba tres ADR de su propio proyecto: contexto, alternativas, decisión, consecuencias.

4

Modelo C4

Ejemplo en c4model.com para diagramar el nivel 1 (contexto) y el nivel 2 (contenedores) de su solución con una herramienta gratuita.

5

Escenarios de calidad

Investigue el método ATAM y practique la plantilla estímulo – condición – respuesta – métrica con tres requisitos de su caso.

02

SECCIÓN 2

Arquitectura lógica

- Estilos y patrones: monolito, capas, microservicios, eventos
- Cliente-servidor, 3 capas y N capas: la familia clásica
- APIs, REST, API Gateway y los patrones de resiliencia
- Arquitectura basada en eventos y arquitecturas de datos distribuidos
- La capa de integración con servicios externos: estilos, contrato y resiliencia
- Cómo elegir un estilo con criterios explícitos
- Cuatro formas distintas de diagramar la misma arquitectura lógica

Arquitectura de Software

Mapa de estilos arquitectónicos

No son opciones excluyentes ni una escalera de progreso. Son respuestas distintas a problemas distintos.

Monolítico

Todo el código y las funcionalidades acoplados en un solo paquete desplegable.

Cliente-servidor

Un cliente solicita, un servidor responde. La forma más antigua de sistema distribuido.

En capas (3 / N)

Presentación, lógica de negocio y datos, separadas en niveles con responsabilidades claras.

Microservicios

Servicios pequeños, autónomos y desplegables por separado, que se comunican por API.

Basado en eventos

Componentes desacoplados que publican y consumen eventos a través de un intermediario.

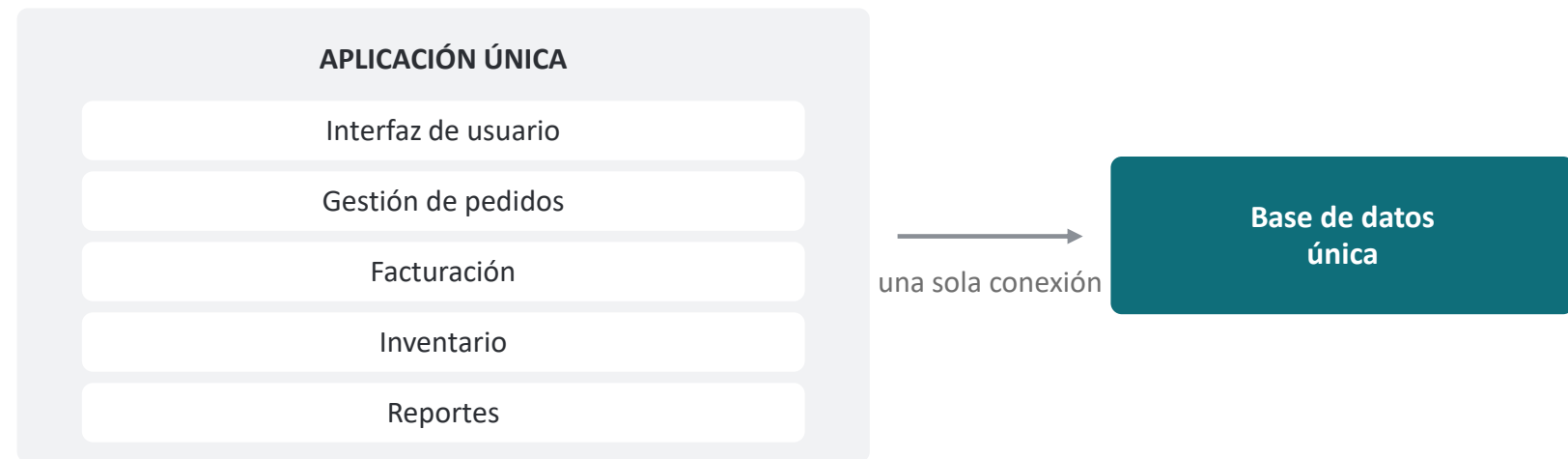
Serverless

Funciones que se ejecutan por evento, sin administrar servidores. Se paga por uso.

Arquitectura de Software

Arquitectura monolítica

El término monolito describe al software en el que el código y las funcionalidades están acoplados en un solo paquete, lo cual funciona muy bien en ciertos escenarios.



Despliegue único

Todo se publica como una sola unidad.

Base de código compartida

Un repositorio, una compilación.

Comunicación en proceso

Llamadas directas entre módulos, sin red de por medio.

Arquitectura de Software

Monolito · ventajas y desventajas

VENTAJAS

- Simplicidad inicial: un proyecto, un despliegue, un entorno.
- Depuración directa: el error se sigue en una sola traza.
- Pruebas de extremo a extremo sencillas.
- Menor latencia: llamadas en memoria, no por red.
- Transacciones simples: una sola base de datos, un solo commit.
- Menor costo de infraestructura y de operación.

DESVENTAJAS

- Escalamiento limitado: se replica todo aunque sólo un módulo esté saturado.
- Acoplamiento fuerte: un cambio pequeño puede propagarse.
- Despliegues riesgosos: cada publicación afecta al sistema completo.
- Tecnología única: todo el sistema queda atado a un lenguaje y versión.
- Conflictos en equipos grandes trabajando sobre el mismo código.
- Al crecer, el costo de cada cambio aumenta.



Para la oferta: si propone un monolito, dígallo con seguridad y justifíquelo por tamaño del equipo, plazo y volumen esperado. Nadie descuenta puntaje por eso; sí lo descuentan por microservicios sin fundamento.

Arquitectura de Software

El punto medio: monolito modular

La industria volvió sobre sus pasos: la recomendación más frecuente hoy es empezar con un monolito bien modularizado y extraer servicios sólo cuando exista una razón concreta.

Un solo despliegue

Se publica como una unidad: baja complejidad operativa, sin orquestadores ni mallas de servicio.

Módulos con fronteras reales

Cada módulo tiene su interfaz pública y su propio esquema de datos; nadie entra por la puerta trasera.

Preparado para dividirse

Cuando un módulo justifique su propio ciclo de vida, se extrae con un costo acotado.

Razones válidas para extraer un servicio: un módulo necesita escalar mucho más que el resto · un equipo distinto necesita liberar a su propio ritmo · el módulo requiere otra tecnología · el módulo tiene requisitos de seguridad o de cumplimiento distintos.

Arquitectura de Software

Monolito · en la vida real

Dónde se usa hoy un monolito, y por qué en esos casos es la decisión correcta.

Un sistema de ventas de una PyME

Un local con tres cajas, un inventario y un módulo de boletas, todo en un servidor.

Por qué encaja: la carga es conocida, el equipo es de dos o tres personas y no hay razón para repartir nada.

La gestión de un municipio pequeño

Permisos, patentes y atención de público en una sola aplicación web.

Por qué encaja: presupuesto acotado, un área de informática de pocas personas y operación en horario hábil.

El primer producto de una startup

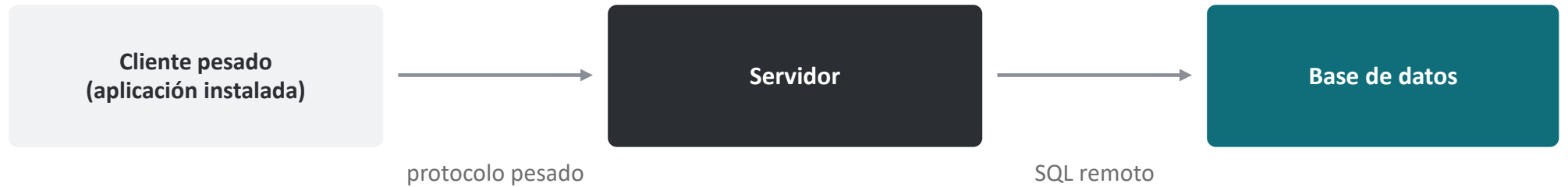
La versión inicial de casi todo producto digital nace como un monolito y se divide sólo cuando el crecimiento lo obliga.

Por qué encaja: hay que llegar rápido al mercado y todavía no se sabe qué partes van a crecer.

Regla que se repite en la industria: casi ningún sistema nace en microservicios. Nacen monolíticos y se dividen cuando el dolor lo justifica.

Arquitectura de Software

Arquitectura cliente – servidor



Se caracteriza por:

- Clientes pesados, no estándar: hay que instalar y actualizar en cada equipo.
- Conexiones dedicadas a la base de datos, una por usuario.
- Protocolos pesados y ejecución remota de sentencias SQL.
- Alta administración: cada cambio implica desplegar en todos los clientes.
- Bajo rendimiento y alto tráfico de red.
- Baja accesibilidad: sólo desde equipos habilitados.

Arquitectura de Software

Cliente – servidor mejorada



Se caracteriza por:

- Lógica de negocio dentro de la base de datos.
- Clientes pesados, no estándar, con conexiones dedicadas.
- Mejora en rendimiento respecto del modelo anterior.
- Alta administración y baja escalabilidad.
- Baja flexibilidad: cambiar la regla implica tocar la base de datos.
- Baja portabilidad: queda amarrada al motor y a su lenguaje propietario.

Sigue existiendo en sistemas heredados. Si el caso del mandante incluye uno, aparecerá como restricción de integración.

Arquitectura de Software

Cliente – servidor · en la vida real

Sigue vivo en sistemas antiguos que los proyectos actuales tienen que integrar o reemplazar.

Sistemas contables de escritorio

El software instalado en cada computador de la oficina, conectado a la base de datos del servidor de la sala.

Dónde se ve: contabilidad, remuneraciones y facturación en empresas medianas.

Punto de venta de farmacia o supermercado

Cada caja es un cliente pesado que conversa con el servidor de la tienda.

Dónde se ve: retail y farmacias. Sigue siendo común porque debe funcionar aunque se caiga Internet.

Sistemas clínicos y de laboratorio

Aplicaciones instaladas que hablan directo con la base de datos del establecimiento.

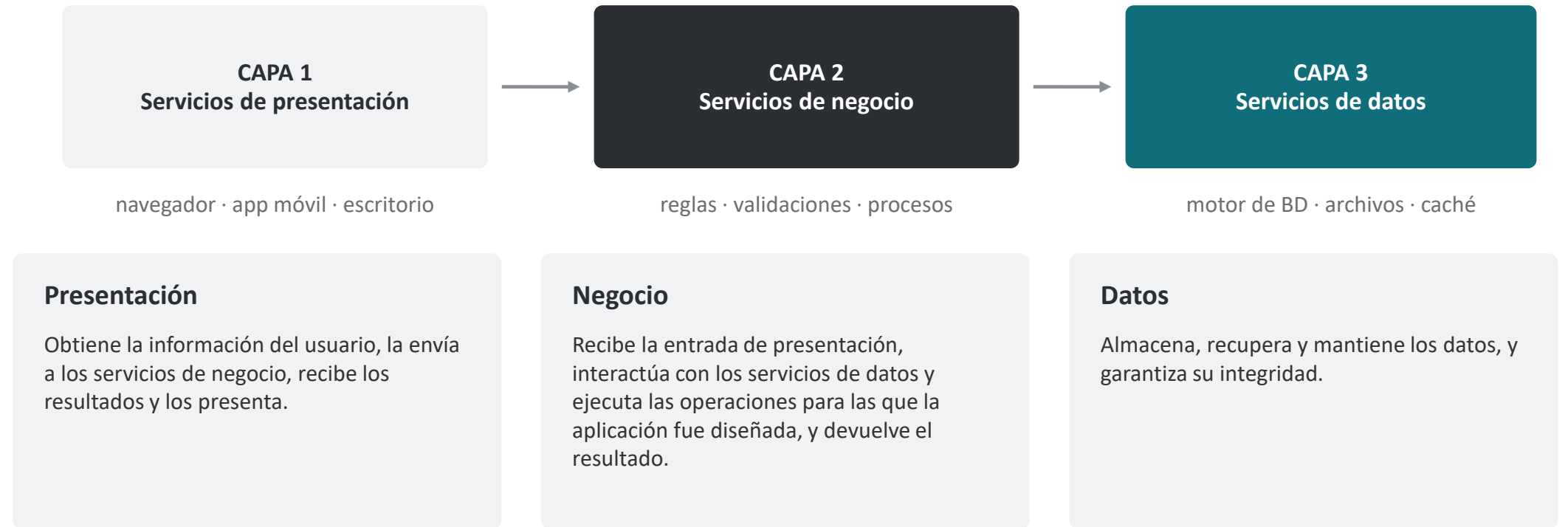
Dónde se ve: fichas clínicas antiguas y equipos de laboratorio con software propietario.

Para su caso: si el mandante tiene un sistema así, aparece como restricción de integración y probablemente como riesgo de migración.

Arquitectura de Software

Arquitectura en 3 capas

Una aplicación de tres capas es aquella cuya funcionalidad puede ser segmentada en tres niveles lógicos.



Arquitectura de Software

Capas (layers) y niveles (tiers)

CAPAS · layers · lógico

- Es una división del software, no del hardware.
- Presentación, negocio y datos pueden convivir en el mismo servidor.
- Se decide en el diseño del código.
- Su objetivo es separar responsabilidades y facilitar el mantenimiento.
- Aparece en la arquitectura LÓGICA.

NIVELES · tiers · físico

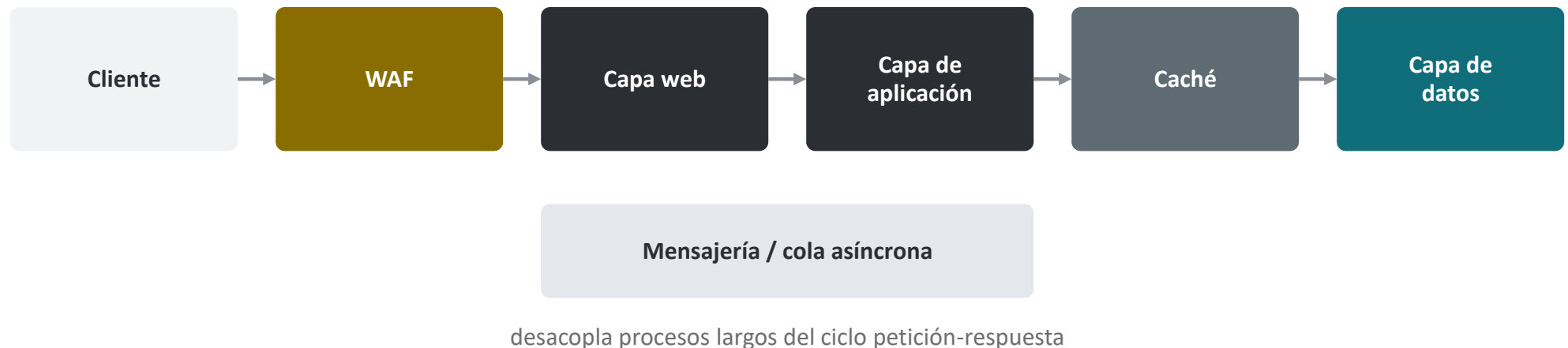
- Es una división de la ejecución en máquinas o procesos distintos.
- Cada nivel puede escalarse, asegurarse y respaldarse por separado.
- Se decide en el despliegue.
- Su objetivo es escalabilidad, aislamiento y seguridad.
- Aparece en la arquitectura FÍSICA.

Una aplicación de 3 capas puede correr en 1 servidor (1 nivel), en 3 servidores (3 niveles) o en 30 contenedores. La cantidad de capas no determina la cantidad de máquinas: eso lo decide el dimensionamiento.

Arquitectura de Software

Arquitectura N capas

La construcción de aplicaciones n-capas distribuidas ha emergido como la arquitectura predominante para aplicaciones multiplataforma en la mayor parte de las empresas.



El modelo presenta algunas ventajas:

- Desarrollos paralelos: cada capa puede avanzar por separado.
- Aplicaciones más robustas gracias al encapsulamiento.
- Mantenimiento y soporte más sencillo: cambiar un componente es más simple que modificar un monolito.
- Mayor flexibilidad: se pueden añadir módulos con nueva funcionalidad.
- Alta escalabilidad: maneja más peticiones con el mismo rendimiento agregando hardware; el crecimiento es casi lineal y no requiere reescribir código.

Arquitectura de Software

Capas y N capas · en la vida real

Es el estilo más frecuente en sistemas empresariales y el que probablemente propongan ustedes.

La banca en línea

Portal web y aplicación móvil, una capa de servicios con las reglas del negocio, y el sistema central con los datos.

Por qué encaja: separa el canal del negocio, y el negocio de los datos que exigen máxima protección.

Un portal de trámites del Estado

Sitio público, capa de negocio con las reglas del trámite, base de datos y una capa de integración con otros servicios.

Por qué encaja: los canales cambian seguido; las reglas y los datos, no.

Un sistema de matrícula universitaria

Portal del estudiante, servicios de matrícula y aranceles, base de datos académica.

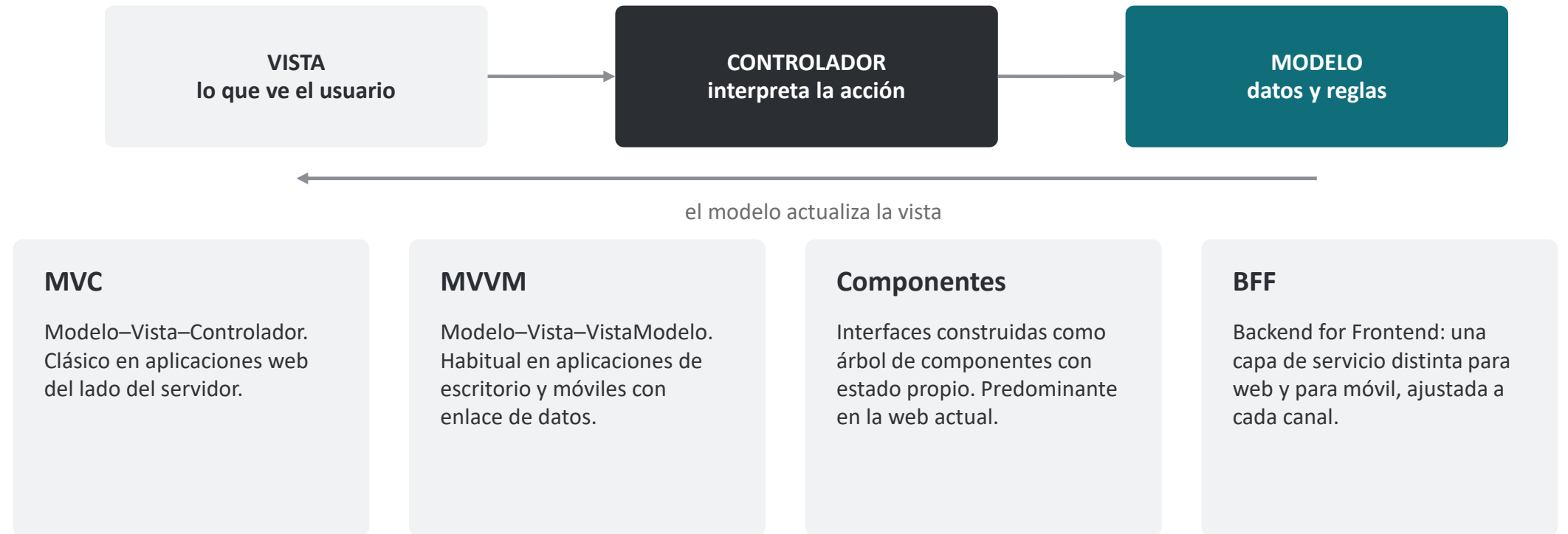
Por qué encaja: hay un peak enorme dos veces al año que se resuelve agregando servidores sólo en la capa web.

Detalle práctico: en los tres casos el peak se absorbe replicando la capa web y la de aplicación, sin tocar la base de datos. Ésa es la ventaja concreta de separar en capas.

Arquitectura de Software

MVC y sus variantes

MVC no es una alternativa a las tres capas: es un patrón que organiza el interior de la capa de presentación.



Arquitectura de Software

Fundamentos

Lógica

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

MVC · en la vida real

MVC no organiza el sistema completo: organiza el interior de la aplicación que el usuario ve.

Aplicaciones web con marcos clásicos

El controlador recibe la petición, pide datos al modelo y elige la vista que se devuelve.

Dónde se ve: la mayoría de los sistemas de gestión construidos en la última década.

Aplicaciones móviles

La variante Modelo–Vista–VistaModelo, con enlace de datos entre la pantalla y el estado.

Dónde se ve: aplicaciones bancarias, de delivery y de transporte.

Interfaces web por componentes

El árbol de componentes con estado propio reemplazó al MVC clásico en el navegador.

Dónde se ve: portales de comercio electrónico y paneles de administración.

Arquitectura de Software

Arquitectura de microservicios

La arquitectura de microservicios es un enfoque para el desarrollo de software que divide una aplicación en pequeños servicios independientes. Cada uno se ejecuta de forma autónoma y se comunica con los demás, por ejemplo a través de APIs.

Una función de negocio

Cada servicio implementa una sola capacidad del negocio: pedidos, pagos, inventario, notificaciones.

Despliegue independiente

Cada servicio se publica a producción sin afectar a los demás.

Tecnología libre

Cada servicio puede escribirse en un lenguaje distinto: se comunican por su API, no por su código.

Acoplamiento flexible

Se relacionan mediante contratos de API estables, no mediante llamadas internas.

Equipos pequeños

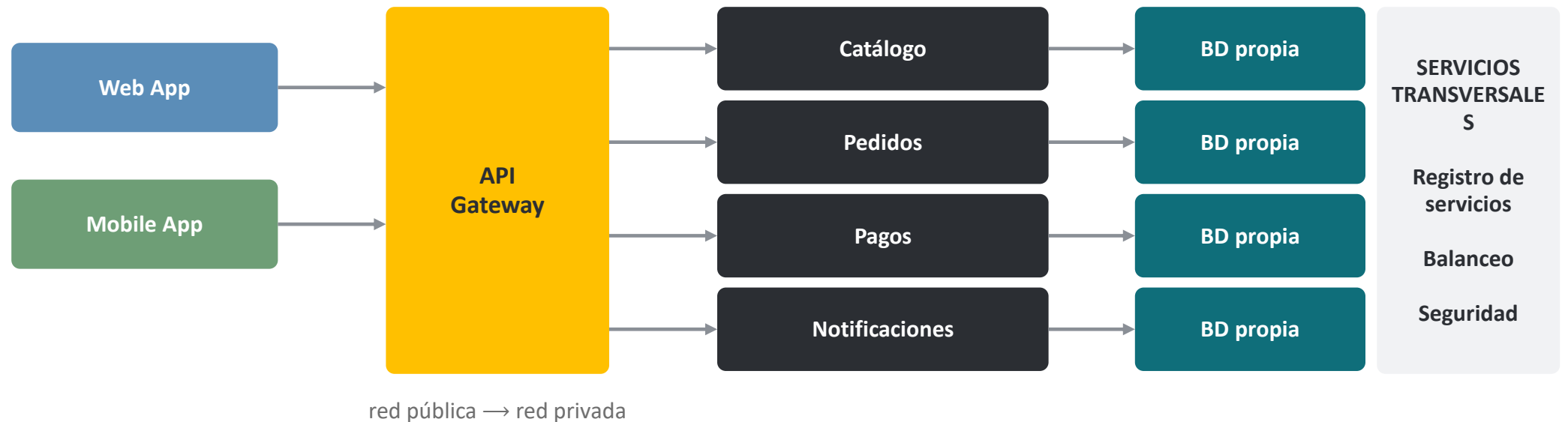
Un equipo dedicado puede construir y operar cada servicio sin mucha coordinación externa.

Requiere DevOps

Es más compleja de compilar y administrar: exige automatización, monitoreo y cultura DevOps.

Arquitectura de Software

Microservicios · vista general



Cada servicio tiene su propio almacén de datos. Nadie consulta la base de datos de otro: si el servicio de Pedidos necesita el precio, se lo pide a Catálogo por su API. Esa regla es la que hace posible el despliegue independiente... y la que introduce la consistencia eventual.

Arquitectura de Software

API · Application Programming Interface

Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos.



Componentes de una API:

Endpoints

Las URL donde la API recibe solicitudes.

Métodos

GET, POST, PUT, DELETE: la acción que se quiere realizar.

Cabeceras

Autenticación, tipo de contenido, versión, trazabilidad.

Cuerpo

Los datos que van y vuelven, normalmente en JSON o XML.

Arquitectura de Software

APIs REST

REST significa transferencia de estado representacional. Define un conjunto de operaciones que los clientes pueden usar para acceder a los datos del servidor. Clientes y servidores intercambian datos mediante HTTP.

Método	Qué hace	Ejemplo
GET	Recupera (o recibe) un recurso solicitado	GET /v1/productos/1024
POST	Crea un recurso	POST /v1/pedidos
PUT	Actualiza un recurso	PUT /v1/productos/1024
DELETE	Elimina un recurso	DELETE /v1/pedidos/88

Anatomía de un endpoint:

https://api.tienda.cl	/v1	/productos	/laptops	?precio_max=1000
Base URL	Versión	Recurso	Sub-recurso	Parámetros

Versionar la API desde el día uno es una decisión arquitectónica: permite evolucionar sin romper a quien ya la consume.

Arquitectura de Software

Otras formas de comunicar servicios

REST / HTTP

Estándar de facto para integraciones externas. Simple, universal, fácil de documentar y de probar.

Use cuando: integra con terceros o con sistemas del mandante.

GraphQL

El cliente declara exactamente qué datos necesita, en una sola consulta.

Use cuando: hay muchos clientes distintos con necesidades de datos muy diferentes.

gRPC

Comunicación binaria de alto rendimiento con contratos fuertemente tipados.

Use cuando: hay mucho tráfico entre servicios internos y la latencia importa.

WebSocket / SSE

Canal permanente para enviar información al cliente sin que la pida.

Use cuando: hay tableros en vivo, chat, seguimiento o notificaciones.

Webhooks

El proveedor llama a una URL suya cuando ocurre algo.

Use cuando: integra con servicios externos de pago, mensajería o logística.

Colas y mensajería

Comunicación asíncrona a través de un intermediario que almacena el mensaje.

Use cuando: el proceso es largo o el destino puede estar caído.

Arquitectura de Software

API Gateway

Un API Gateway actúa como intermediario entre el cliente de API y los servicios de backend, y presenta un único punto de entrada para todas las llamadas.



Recibe todas las llamadas dirigidas a los endpoints de la organización, las autentica, las procesa según las políticas definidas, las enruta al servicio correspondiente y devuelve el resultado agregado al cliente.

Cuidado: el gateway concentra tráfico. Si no es redundante, se transforma en el punto único de falla de todo el sistema.

Arquitectura de Software

Patrones de resiliencia

En un sistema distribuido, la falla parcial es lo normal. Estos patrones existen para que una falla no se propague.

Registro de servicios

Directorio donde cada instancia se anuncia al iniciar. Permite encontrar servicios sin direcciones fijas.

Balanceo de carga

Reparte las peticiones entre las instancias disponibles y saca de rotación a las que no responden.

Cortacircuitos

Si un servicio falla repetidamente, se deja de llamar por un tiempo en vez de acumular esperas.

Reintento con espera

Reintenta la operación con intervalos crecientes, en vez de golpear al servicio caído.

Tiempo límite

Toda llamada remota tiene plazo máximo. Sin timeout, una demora se convierte en caída total.

Compartimentos

Aísla recursos por servicio para que la saturación de uno no consuma los recursos de los demás.

Degradación elegante

Ante la falla de un componente secundario, el sistema sigue operando con funcionalidad reducida.

Idempotencia

Repetir la misma operación no produce efectos duplicados. Imprescindible con reintentos.

Arquitectura de Software

Microservicios · lo que se gana y lo que se paga

LO QUE SE GANA

- Escalar sólo lo que está saturado.
- Publicar a producción con frecuencia y bajo riesgo.
- Equipos autónomos que avanzan en paralelo.
- Tecnología adecuada a cada problema.
- La falla de un servicio no derriba el sistema completo.
- Reemplazar un servicio completo es viable.

LO QUE SE PAGA

- Complejidad operativa: orquestación, despliegue, monitoreo distribuido.
- Latencia y fallas de red donde antes había una llamada en memoria.
- Transacciones distribuidas: se pierde el commit único.
- Depuración difícil: un error cruza varios servicios.
- Más infraestructura y por lo tanto más costo mensual.
- Exige cultura DevOps y automatización desde el día uno.



Criterio honesto para su caso: si su equipo no puede automatizar despliegue, monitoreo y pruebas, los microservicios van a costar más de lo que aportan.

Arquitectura de Software

Microservicios · en la vida real

Los casos donde los microservicios se justificaron tienen algo en común: escalas y frecuencias de cambio muy grandes.

Plataformas de streaming de video

Catálogo, recomendaciones, reproducción, facturación y perfiles, cada uno como servicio independiente.

Por qué se justificó: la reproducción escala miles de veces más que la facturación.

Comercio electrónico grande

Búsqueda, carro, pagos, despacho e inventario, con equipos distintos publicando varias veces al día.

Por qué se justificó: en un evento de descuentos el buscador y el carro se saturan, el resto no.

Aplicaciones de transporte y delivery

Ubicación en tiempo real, asignación de conductor, pagos y calificaciones, separados y con tecnologías distintas.

Por qué se justificó: la ubicación exige tiempo real; la facturación tolera minutos.

Contraste honesto para su caso: si su sistema no tiene una parte que crezca mucho más que las otras, ni varios equipos publicando en paralelo, los microservicios sólo agregan costo.

Arquitectura de Software

Los datos en una arquitectura distribuida

Cuando cada servicio tiene su propia base de datos, se pierde la transacción única. Ese es el verdadero costo de los microservicios.

Base de datos por servicio

Cada servicio es dueño de sus datos y nadie más los toca directamente. Da autonomía, pero obliga a duplicar información.

Consistencia eventual

Los datos quedan consistentes después de un tiempo, no de inmediato. Hay que decidir si el negocio lo tolera.

Saga

Una transacción de negocio se descompone en pasos locales, cada uno con su operación de compensación si algo falla.

CQRS

Separar el modelo de escritura del modelo de lectura, para optimizar cada uno por separado.

Vista materializada

Copia de sólo lectura construida a partir de los datos de varios servicios, para consultas y reportes.

Transacción outbox

Guardar el evento en la misma transacción que el dato, para que no se pierda si el intermediario falla.

Arquitectura de Software

SOA y microservicios

	SOA (Service Oriented Architecture)	Microservicios
Tamaño del servicio	Servicios grandes, alineados a procesos completos	Servicios pequeños, una capacidad de negocio cada uno
Comunicación	Bus de servicios empresarial (ESB) con lógica de enrutamiento y transformación	Comunicación directa por API o mensajería simple
Datos	Frecuentemente comparten base de datos	Cada servicio es dueño de sus datos
Gobierno	Centralizado, con un equipo de integración	Distribuido, cada equipo gobierna su servicio
Despliegue	Coordinado, por ventanas de cambio	Independiente y continuo
Dónde se encuentra	Grandes organizaciones, banca, sector público	Productos digitales, plataformas con alta frecuencia de cambio

Los microservicios son, en el fondo, una forma de SOA con gobierno distribuido y sin bus central.

Arquitectura de Software

Arquitectura basada en eventos (EDA)

La arquitectura basada en eventos se basa en servicios pequeños y desacoplados que interactúan mediante la publicación, el consumo y el enrutamiento de eventos.



El productor no sabe quién lo escucha ni cuántos son. Agregar un nuevo consumidor no obliga a modificar al productor: eso es el desacoplamiento, y es la razón por la que este estilo escala en cantidad de integraciones.

Arquitectura de Software

EDA · beneficios y cuándo usarla

Escalabilidad

Cada consumidor escala según su propia carga, sin arrastrar a los demás.

Flexibilidad

Agregar un consumidor nuevo no obliga a tocar al productor.

Eficiencia

El intermediario actúa como búfer elástico entre servicios de velocidades distintas.

Desacoplamiento

Productor y consumidor no se conocen ni necesitan estar activos al mismo tiempo.

Resiliencia

Si un consumidor falla, el mensaje espera en la cola y se procesa después.

Agilidad

Nuevos procesos de negocio se construyen escuchando eventos que ya existen.

Encaja bien cuando...

- Un mismo hecho de negocio dispara varias acciones distintas.
- Hay procesos largos que no deben bloquear al usuario.
- Se integran muchos sistemas con disponibilidad variable.
- Se necesita histórico de lo que ocurrió, no sólo el estado actual.

Cuesta caro cuando...

- El negocio exige respuesta inmediata y consistente.
- El equipo no tiene experiencia operando colas y reintentos.
- No hay trazabilidad: seguir un flujo asíncrono sin observabilidad es muy difícil.
- Se usa para todo, incluso donde una llamada directa bastaba.

Arquitectura de Software

Eventos · en la vida real

Un hecho de negocio ocurre una vez y desencadena varias acciones que no dependen entre sí.

Una compra en línea

Se emite «pedido pagado» y reaccionan: inventario descuenta stock, bodega prepara el despacho, contabilidad emite la boleta y el cliente recibe un correo.

Ninguno de ellos necesita esperar a los otros.

Transporte público y logística

Cada lectura de posición o de tarjeta es un evento que alimenta el tablero de control, la facturación y el análisis posterior.

El mismo evento sirve a tres áreas distintas.

Detección de fraude bancario

Cada transacción se publica como evento: un servicio la evalúa en tiempo real, otro la registra y un tercero alimenta el reporte regulatorio.

Señal de que su caso pide eventos: cuando en las bases aparece varias veces la frase «al ocurrir X, el sistema debe además...».

Arquitectura de Software

Streaming, CQRS y event sourcing

Event streaming

El evento no se consume y se borra: queda en un registro ordenado y persistente que varios consumidores pueden leer a su ritmo, incluso volver atrás.

Habilita analítica en tiempo real y reprocesos.

CQRS

Separar el camino de escritura del camino de lectura. Cada uno se modela y se escala por separado.

Útil cuando el sistema lee mucho más de lo que escribe, o cuando los reportes ahogan la operación.

Event sourcing

El estado no se guarda: se guarda la secuencia completa de eventos que lo produjeron, y el estado se reconstruye.

Da auditoría perfecta, pero complica las consultas y las correcciones.

Advertencia para la oferta: estos tres patrones resuelven problemas reales, pero también son los que más se proponen sin necesidad. Si los incluye, tiene que poder explicar qué requisito del caso los exige.

Arquitectura de Software

Arquitectura hexagonal y limpia

No es un estilo de despliegue sino de organización interna: mantener las reglas de negocio independientes de la tecnología que las rodea.



Beneficio concreto para su oferta: si el dominio no depende del proveedor, migrar de on-premise a nube —o cambiar de nube— deja de ser una reescritura y pasa a ser un cambio de adaptador.

Arquitectura de Software

La capa de integración

Ninguna solución vive sola. La capa de integración es el conjunto de componentes cuya única responsabilidad es conversar con lo que está fuera de su sistema.



Por qué se separa: el sistema externo cambia de versión, se cae, cambia de formato o cambia de proveedor. Si esa conversación está repartida por todo el código, cada cambio del tercero se convierte en un cambio en su sistema completo.

Arquitectura de Software

Estilos de integración con terceros

API síncrona (REST/SOAP)

Usted llama y espera respuesta.

Use cuando: el usuario necesita el resultado ahora (consultar stock, autorizar un pago).
Riesgo: si el tercero se demora, su sistema se demora.

Webhook / notificación

El tercero le llama a usted cuando ocurre algo.

Use cuando: el resultado llega después (confirmación de pago, estado de un envío).
Riesgo: hay que exponer un endpoint y validar su origen.

Cola / mensajería

Usted deja el mensaje y alguien lo procesa.

Use cuando: el proceso es largo o el destino puede estar caído.
Riesgo: exige manejar orden, duplicados y reintentos.

Archivo por lote

Intercambio de archivos por SFTP en horario fijo.

Use cuando: el mandante tiene sistemas antiguos. Sigue siendo muy común en el sector público.
Riesgo: latencia de horas y errores silenciosos.

Replicación / CDC

Los cambios de una base se propagan a otra.

Use cuando: hay que alimentar reportería o un sistema analítico.
Riesgo: acopla los modelos de datos de ambos sistemas.

Base de datos compartida

Su sistema consulta directamente la base del otro.

Es un antipatrón: cualquier cambio de esquema del tercero rompe su sistema, sin aviso y sin contrato.
Evítelo, y si las bases lo exigen, declárelo como riesgo.

Arquitectura de Software

Los terceros que suelen aparecer

Antes de dibujar, haga la lista. Cada integración tiene esfuerzo de desarrollo, ambiente de pruebas, costo por transacción y un dueño al otro lado del teléfono.

Identidad

Autenticación de usuarios con un proveedor externo o con el directorio del mandante.

Medios de pago

Pasarelas y bancos. Cobran por transacción y exigen certificación.

Documento tributario

Emisión y validación de documentos electrónicos ante la autoridad.

Firma electrónica

Firma avanzada de documentos y su verificación posterior.

Notificaciones

Correo, SMS y mensajería. Cobran por mensaje enviado.

ERP / sistemas del mandante

El sistema que ya existe y con el que hay que convivir. Suele ser el más difícil.

Georreferenciación

Mapas, direcciones y rutas. Cobran por consulta.

Logística y despacho

Estados de envío, etiquetas, seguimiento.

Arquitectura de Software

El contrato de integración

Ocho preguntas que hay que responder por cada tercero antes de estimar el esfuerzo.

Pregunta	Por qué importa	Qué pasa si no lo pregunta
¿Qué protocolo y formato usa?	Define el trabajo de desarrollo y las bibliotecas necesarias.	Descubre a mitad de camino que es SOAP con firma XML.
¿Cómo se autentica?	Certificados, tokens o llaves cambian el diseño y la operación.	Aparece un trámite de certificación de tres semanas.
¿Hay ambiente de pruebas?	Sin sandbox no se puede desarrollar ni probar de verdad.	Se prueba en producción, con datos reales.
¿Cuántas llamadas permite?	Los límites de uso condicionan el diseño y obligan a cachear.	El sistema falla en hora punta por exceso de llamadas.
¿Qué disponibilidad ofrece?	Su SLA no puede ser mejor que el del tercero del que depende.	Compromete 99,9% apoyado en un servicio que ofrece 99%.
¿Tiene costo por transacción?	Costo variable de operación: al flujo de caja.	Aparece con el primer mes de uso real.
¿Cómo versiona su interfaz?	Cuánto aviso hay antes de un cambio que rompe.	Un día la integración deja de operar.
¿Quién responde ante un problema?	Contraparte, canal y tiempos de respuesta.	Nadie responde y usted incumple.

Las respuestas van al informe: son supuestos declarados y, varias de ellas, riesgos con responsable asignado.

Arquitectura de Software

Resiliencia frente al tercero

Regla: su sistema no puede caerse porque se cayó el de otro. Diseñe siempre el comportamiento ante la falla del tercero.

Tiempo límite

Toda llamada externa tiene plazo máximo, siempre. Sin timeout, la demora del tercero se transforma en caída suya.

Reintento con espera

Reintentar con intervalos crecientes y un máximo de intentos. Nunca reintentar en bucle inmediato.

Cortacircuitos

Tras N fallas seguidas se deja de llamar por un tiempo y se responde de inmediato con el modo alternativo.

Idempotencia

Cada operación lleva una clave única para que un reintento no genere un pago o un pedido duplicado.

Cola de reintento

Lo que no se pudo enviar queda en una cola, y lo que falla definitivamente en una cola de mensajes muertos, para revisión humana.

Modo degradado

Qué hace el sistema mientras tanto: aceptar y procesar después, usar el último dato conocido, o avisar con claridad al usuario.

En la defensa le van a preguntar: «¿y si la pasarela de pago está caída?». La respuesta correcta describe el modo degradado, el reintento y el aviso al usuario —no «no debería pasar».

Arquitectura de Software

Seguridad de la integración

Aspecto	Medida mínima esperada	Dónde se declara en la oferta
Canal	Todo el tráfico cifrado con TLS vigente. Sin excepciones, tampoco en la red interna.	Arquitectura física y de seguridad
Autenticación mutua	Certificados de cliente o llaves rotadas periódicamente para servicio a servicio.	Ficha de integración
Autorización	Credenciales con el mínimo privilegio necesario y de uso exclusivo de esa integración.	Modelo de accesos
Integridad	Firma del mensaje cuando el tercero la ofrece, para verificar que no fue alterado.	Ficha de integración
Origen del webhook	Validar firma y lista de direcciones permitidas: un endpoint público es un blanco.	Arquitectura de seguridad
Secretos	Llaves y contraseñas en un almacén de secretos, nunca en el código ni en el repositorio.	Procedimiento de despliegue
Registro	Trazabilidad de cada llamada, sin escribir datos personales ni credenciales en los registros.	Plan de observabilidad
Datos personales	Enviar sólo lo necesario. El tercero que trata datos por usted es un encargado y requiere acuerdo.	Restricciones legales

Arquitectura de Software

Ficha de integración para el informe

Una ficha por cada sistema externo. Es la evidencia de que la integración está pensada, no supuesta.

Campo	Ejemplo
Sistema externo	Pasarela de pago del banco recaudador
Requisito que lo justifica	RQ-08 · Pago en línea de la solicitud
Sentido del flujo	Saliente (autorización) y entrante (confirmación por webhook)
Estilo y protocolo	API REST sobre HTTPS + webhook firmado
Autenticación	Credenciales de cliente OAuth 2.0, rotación semestral
Volumen estimado	1.200 transacciones diarias; peak de 90 por minuto
Disponibilidad del tercero	99,5% mensual según su acuerdo publicado
Comportamiento ante falla	Reintento con espera creciente (3 intentos), cortacircuito a los 5 errores, modo degradado: la solicitud queda pendiente de pago
Ambiente de pruebas	Sandbox del proveedor, credenciales solicitadas en la semana 2
Costo asociado	Comisión por transacción: va al flujo de caja como costo variable
Riesgo declarado	Dependencia de tercero; probabilidad media, impacto alto; mitigación: otro medio de pago

Arquitectura de Software

Hexagonal · en la vida real

Se nota cuando algo del entorno cambia: si el dominio está aislado, cambia sólo el adaptador.

Cambiar de medio de pago

El sistema deja de usar una pasarela y contrata otra.

Con el dominio aislado se escribe un adaptador nuevo y el resto del sistema no se toca.

Migrar de on-premise a la nube

El almacenamiento de archivos pasa de una carpeta compartida a almacenamiento de objetos.

Cambia el adaptador de archivos, no la lógica del negocio.

Reemplazar el motor de base de datos

El mandante decide dejar un motor propietario por uno libre.

Si el acceso a datos está detrás de una interfaz, la migración es acotada.

Argumento para la oferta: «el dominio no depende del proveedor; cambiar de pasarela, de nube o de motor implica sustituir un adaptador, no reescribir el sistema».

Arquitectura de Software

Cómo elegir el estilo

Criterio	Monolito / monolito modular	Microservicios	Basado en eventos
Tamaño del equipo	1 a 10 personas	Varios equipos autónomos	Varios equipos
Plazo del proyecto	Corto: menos de 6 meses	Largo, con evolución continua	Medio a largo
Carga esperada	Predecible y acotada	Muy variable o muy alta	Picos e integraciones múltiples
Necesidad de escalar por partes	Baja	Alta	Alta
Frecuencia de cambio	Baja o media	Alta	Alta
Madurez DevOps requerida	Baja	Alta	Alta
Costo de infraestructura	Bajo	Alto	Medio-alto
Complejidad de operación	Baja	Alta	Alta



En la defensa le van a preguntar «¿por qué esta arquitectura?». La respuesta correcta nunca es «porque es moderna»: es «porque el caso exige X y esta opción lo resuelve con el menor costo total».

Arquitectura de Software

No hay una sola forma de dibujarla

Existen muchas formas de diagramar una arquitectura lógica, y la elección depende del problema y de la solución. Éstos son cuatro estilos que funcionan bien en una oferta técnica.

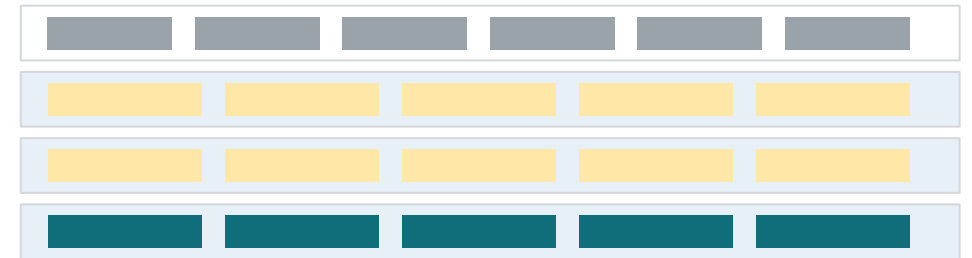
A • Actores en columnas, capas en filas

cada columna es un actor o proceso; cada fila, una capa



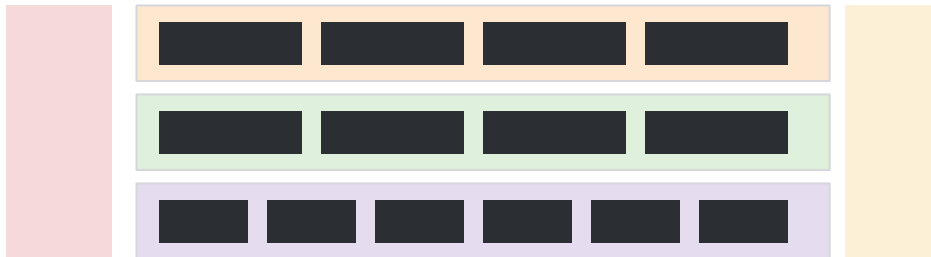
B • Bandas por capa, actores arriba

bandas de color por capa y líneas de flujo entre módulos



C • Capas con microservicios y stack lateral

columna lateral con tecnologías y servicios externos



D • Columnas verticales por capa

cliente · presentación · negocio · servicios · datos



Arquitectura de Software

Cuál estilo conviene

El estilo no es un tema estético: determina qué se ve a primera vista y qué queda escondido.

Estilo	Qué destaca a primera vista	Cuándo conviene	Riesgo
A · Actores en columnas, capas en filas	Quién usa qué, y por qué capa pasa cada actor.	Casos con varios perfiles muy distintos y procesos separados: servicios públicos, utilities, salud.	Con muchos actores la lámina se vuelve ilegible: agrupe por proceso, no por persona.
B · Bandas por capa con actores arriba	El recorrido completo de arriba hacia abajo y las dependencias entre módulos.	Soluciones de tamaño medio con varios departamentos usando el mismo núcleo.	Las líneas de flujo se cruzan; use colores por flujo y una leyenda.
C · Capas con microservicios y stack lateral	La descomposición en servicios y la tecnología de cada capa.	Cuando la propuesta se apoya en microservicios y hay que mostrar el stack elegido.	Mezcla decisiones lógicas con tecnológicas: separe la columna de stack visualmente.
D · Columnas verticales por capa	La separación limpia cliente – presentación – negocio – servicios – datos.	Soluciones con muchas aplicaciones y APIs bien delimitadas; es la más fácil de leer.	Oculto quién usa qué: agregue flechas por actor con color.

Todas son correctas. Elija una, aplíquela de forma consistente y explíquela en 30 segundos al comenzar la presentación.

Arquitectura de Software

Reglas para que el diagrama se entienda

Una dirección de lectura

Izquierda a derecha o arriba abajo, pero una sola. El lector no debería tener que buscar por dónde empezar.

Agrupación explícita

Cajas o bandas que marquen las capas y las zonas. Sin agrupación, veinte rectángulos son ruido.

Leyenda

Qué significa cada color, cada tipo de línea y cada forma. Si usa flechas de colores por flujo, la leyenda es obligatoria.

Nombres de negocio

«Módulo de Facturación», no «MS-FACT-01». El diagrama lógico lo tiene que entender el mandante.

Un nivel por lámina

No mezcle módulos con clases ni con servidores. Si necesita bajar el detalle, haga otra lámina.

Máximo 15 a 20 elementos

Si no cabe, el problema no es la lámina: es que el diagrama está intentando decir dos cosas a la vez.

Sistemas externos distinguibles

Lo que no construye usted va con otro color o fuera del borde del sistema. Marca la frontera del alcance.

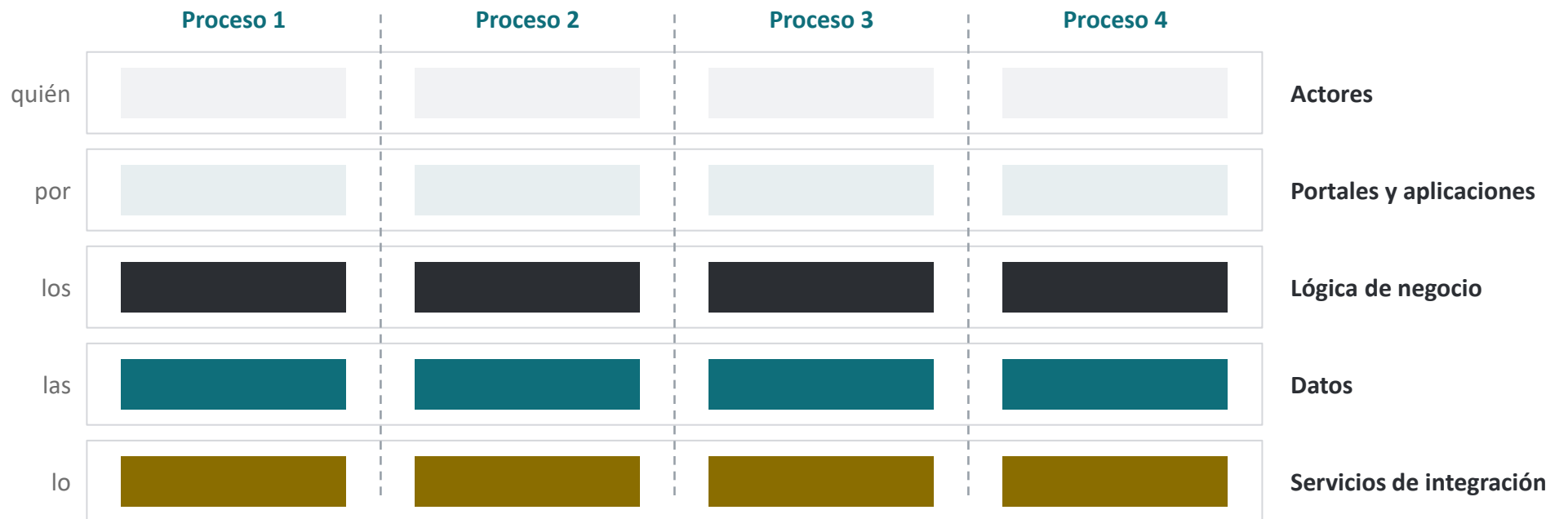
Coherencia con el resto

Los nombres del diagrama deben ser los mismos del texto, de la matriz de trazabilidad y del presupuesto.

Arquitectura de Software

Estilo A · actores en columnas, capas en filas

Columnas verticales por actor o proceso; bandas horizontales por capa técnica.



Se lee así: la columna dice de quién es el proceso; la fila dice en qué capa está el componente. Ejemplo: «Lectura de medidores» tiene su aplicación móvil, su módulo de control, su base de datos y su servicio de integración.

Arquitectura de Software

Estilo A · cuándo usarlo y qué cuidar

Cuándo conviene

- El caso tiene varios perfiles muy distintos que casi no se cruzan: cliente, técnico en terreno, funcionario, call center.
- Las bases técnicas están organizadas por proceso de negocio.
- Hay que mostrar que cada proceso tiene su propia cadena completa, de la interfaz hasta los datos.
- El mandante es una organización con áreas separadas y quiere reconocer la suya en el diagrama.

Qué cuidar

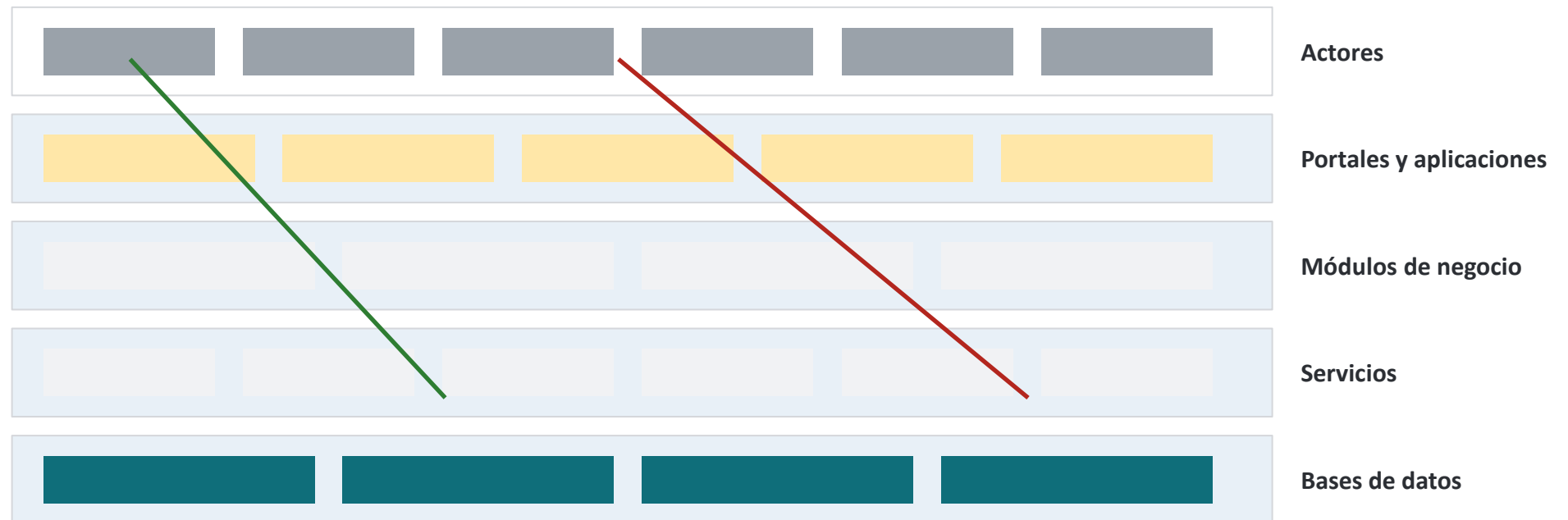
- Con más de cinco o seis columnas la lámina se vuelve ilegible: agrupe por proceso, no por persona.
- Las líneas que cruzan columnas se enredan: use pocas y hágalas explícitas.
- Es fácil terminar dibujando el organigrama en vez del sistema.
- Necesita una banda de título por capa a la izquierda; sin ella, nadie entiende las filas.

Ejemplo real de este estilo: la arquitectura de una empresa sanitaria, con columnas para lectura de medidores, facturación, instalación y mantención, y atención de clientes.

Arquitectura de Software

Estilo B · bandas por capa, actores arriba

Una banda de color por capa, con los actores en la parte superior. Las relaciones se dibujan como líneas de colores que cruzan las bandas, y cada color es un flujo de negocio distinto.



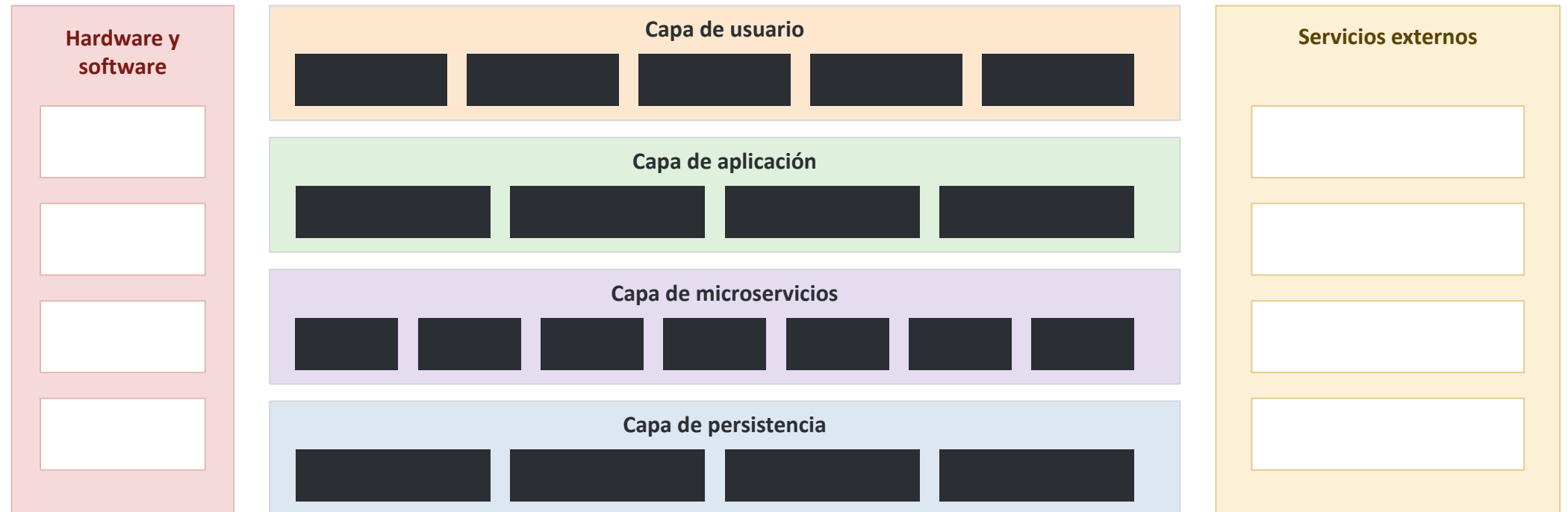
las líneas de color son los flujos de negocio: cada color recorre una funcionalidad completa a través de las capas

Obligatorio en este estilo: una leyenda que diga qué representa cada color de línea.

Arquitectura de Software

Estilo C · capas con microservicios y stack lateral

Al centro, las capas del sistema, con la de microservicios desglosada en servicios y módulos. A los costados, dos columnas: el stack tecnológico y los servicios externos contratados.

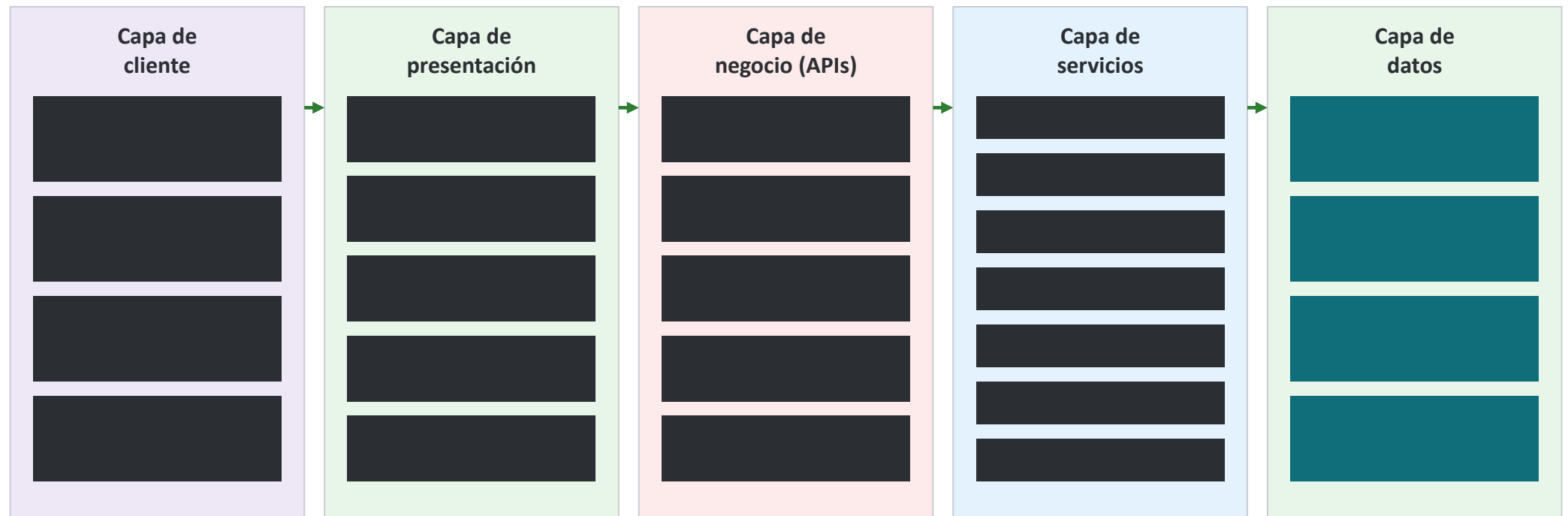


Riesgo de este estilo: mezcla decisiones lógicas con decisiones tecnológicas. Separe visualmente las columnas laterales y diga en la presentación que son anexos, no parte de la arquitectura lógica.

Arquitectura de Software

Estilo D · columnas verticales por capa

Una columna por capa, de izquierda a derecha, en el orden en que viaja una petición: cliente, presentación, negocio, servicios y datos. Las flechas de color indican qué actor recorre qué camino.



Es el estilo más legible y el más fácil de defender. Su límite: no muestra quién usa qué, así que hay que agregar flechas de color por actor y su leyenda.

Arquitectura de Software

Los cuatro estilos, en una frase

A • Actores en columnas

Muestra quién usa qué.

Elíjalo si su caso tiene
perfiles muy distintos y
procesos separados.

B • Bandas por capa

Muestra el recorrido
completo y las
dependencias.

Elíjalo si varios
departamentos usan un
mismo núcleo.

C • Capas con microservicios

Muestra la descomposición
en servicios y la tecnología.

Elíjalo si la propuesta se
apoya en microservicios.

D • Columnas por capa

Muestra la separación limpia
entre capas.

Elíjalo si quiere el diagrama
más legible y fácil de explicar.



No existe el estilo correcto: existe el estilo consistente. Elija uno, aplíquelo en todas las láminas y explíquelo en treinta segundos al abrir la presentación.

Arquitectura de Software

Fundamentos

Ejemplo de Trabajo de años anteriores

Lógica

Física

Nube

Calidad

Continuidad

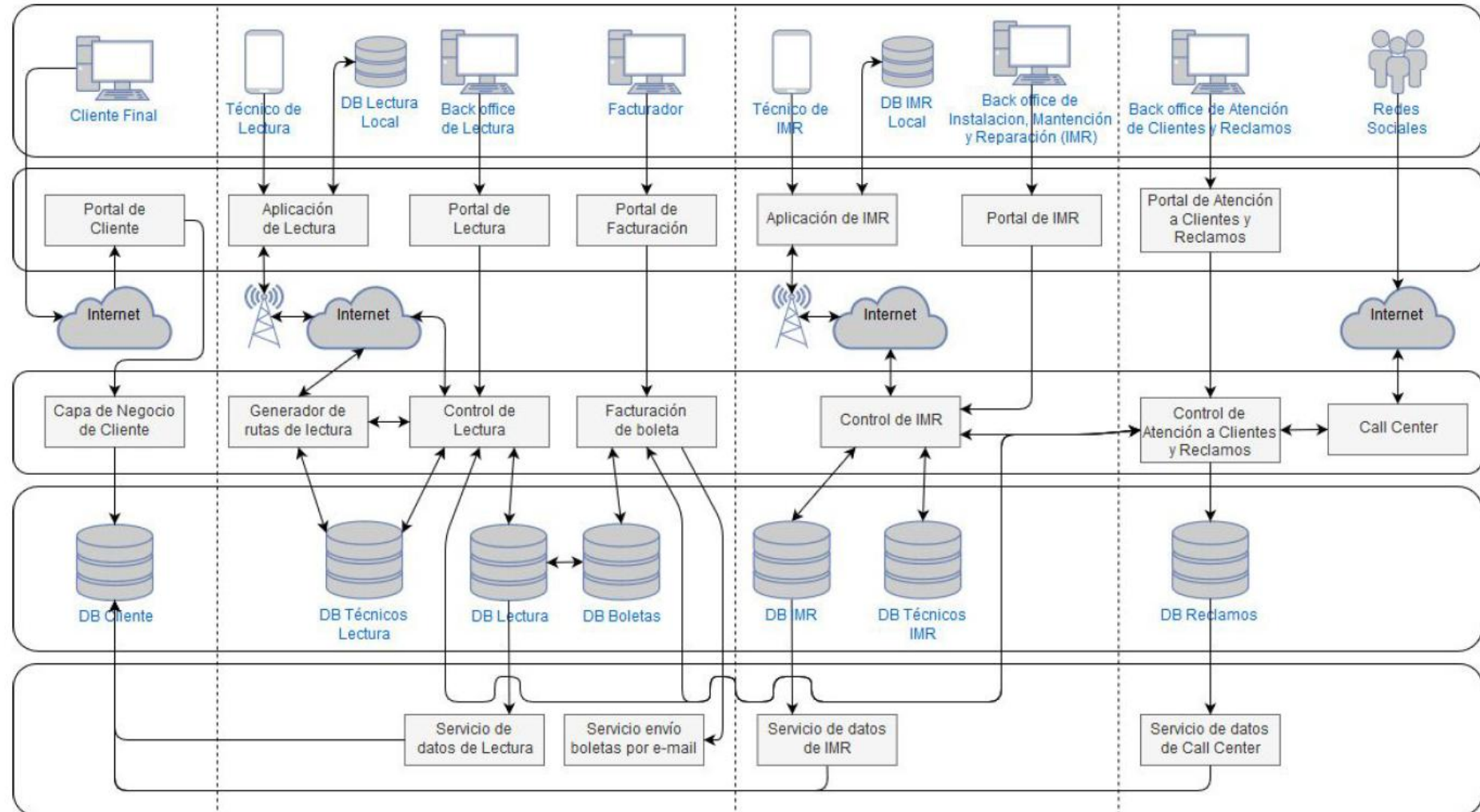
Seguridad

Identidad

Entrega

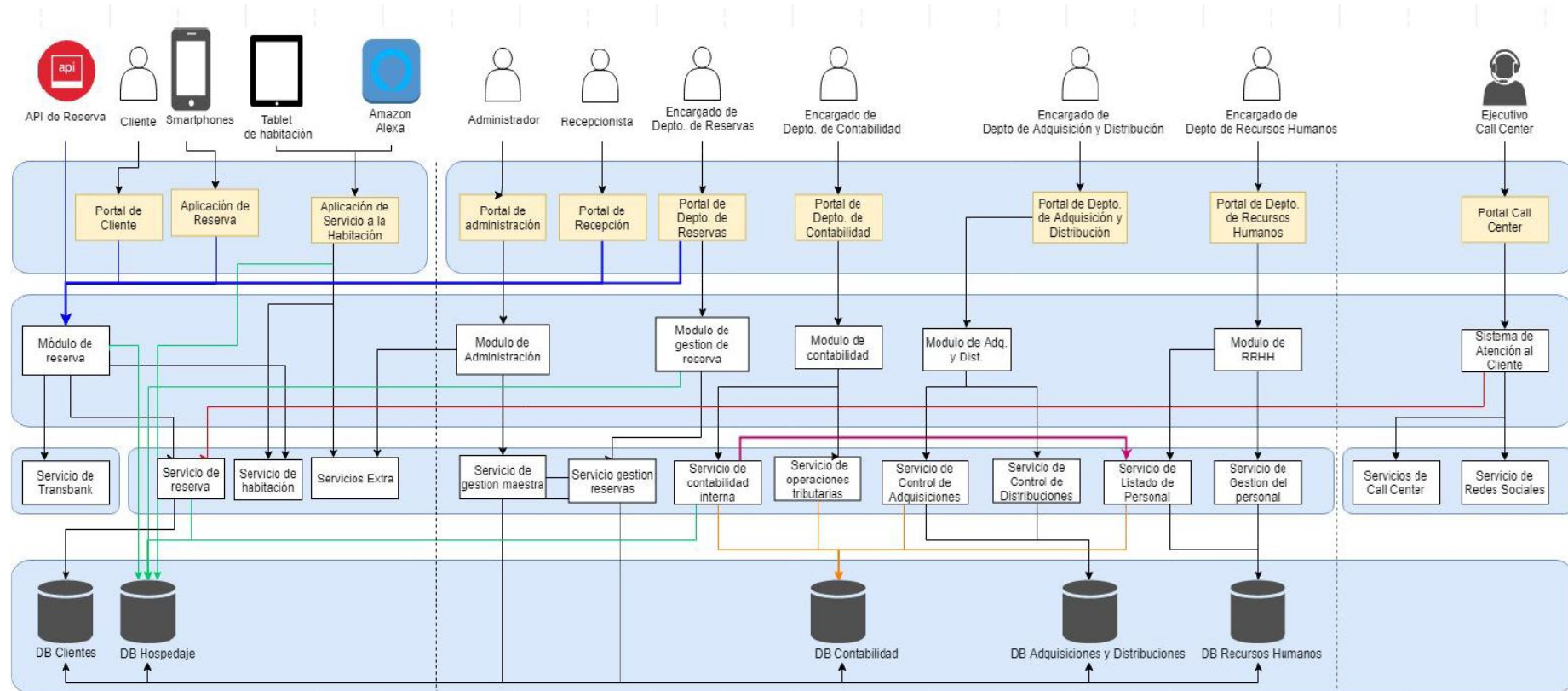
Tendencias

Propuesta



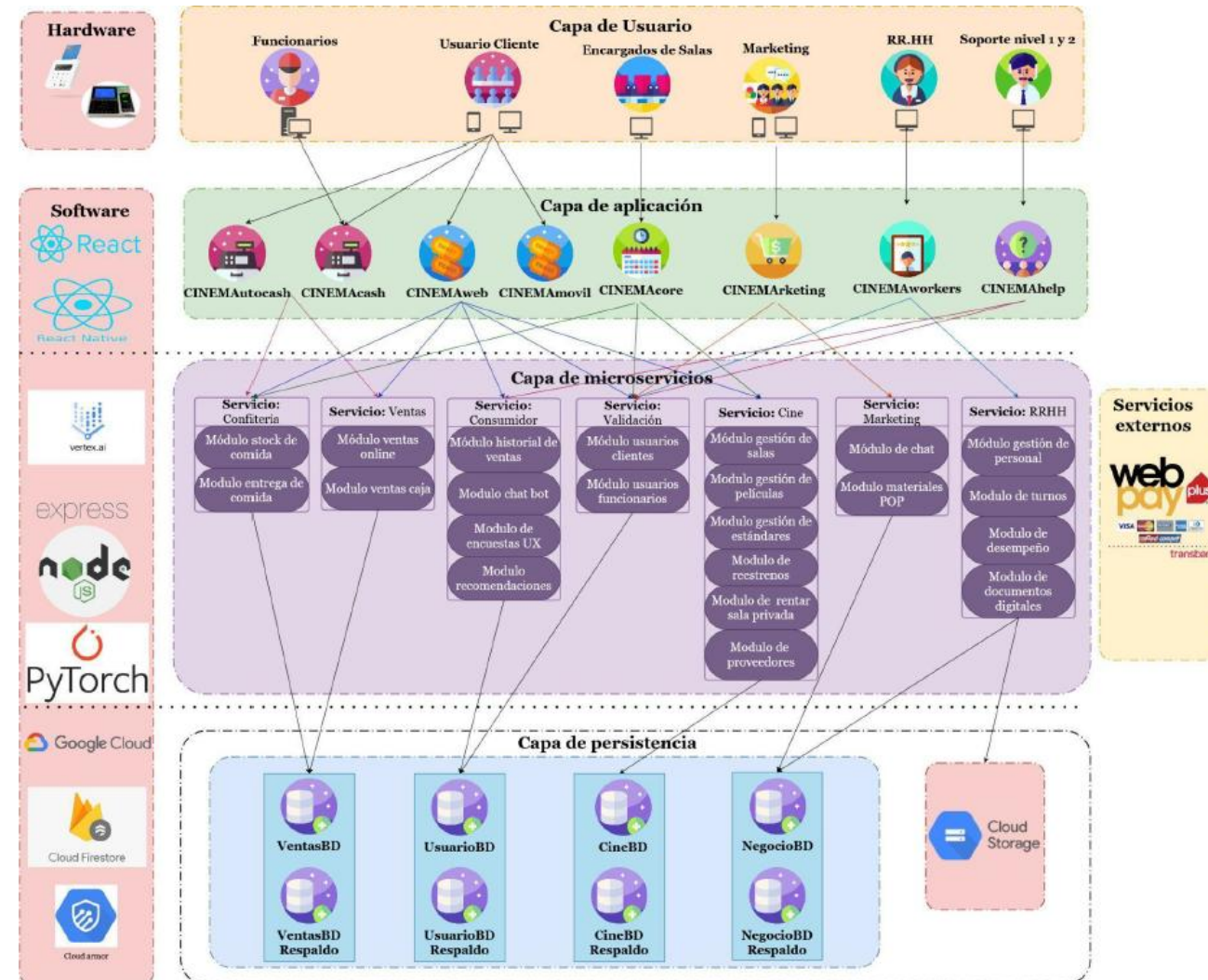
Arquitectura de Software

Ejemplo de Trabajo de años anteriores



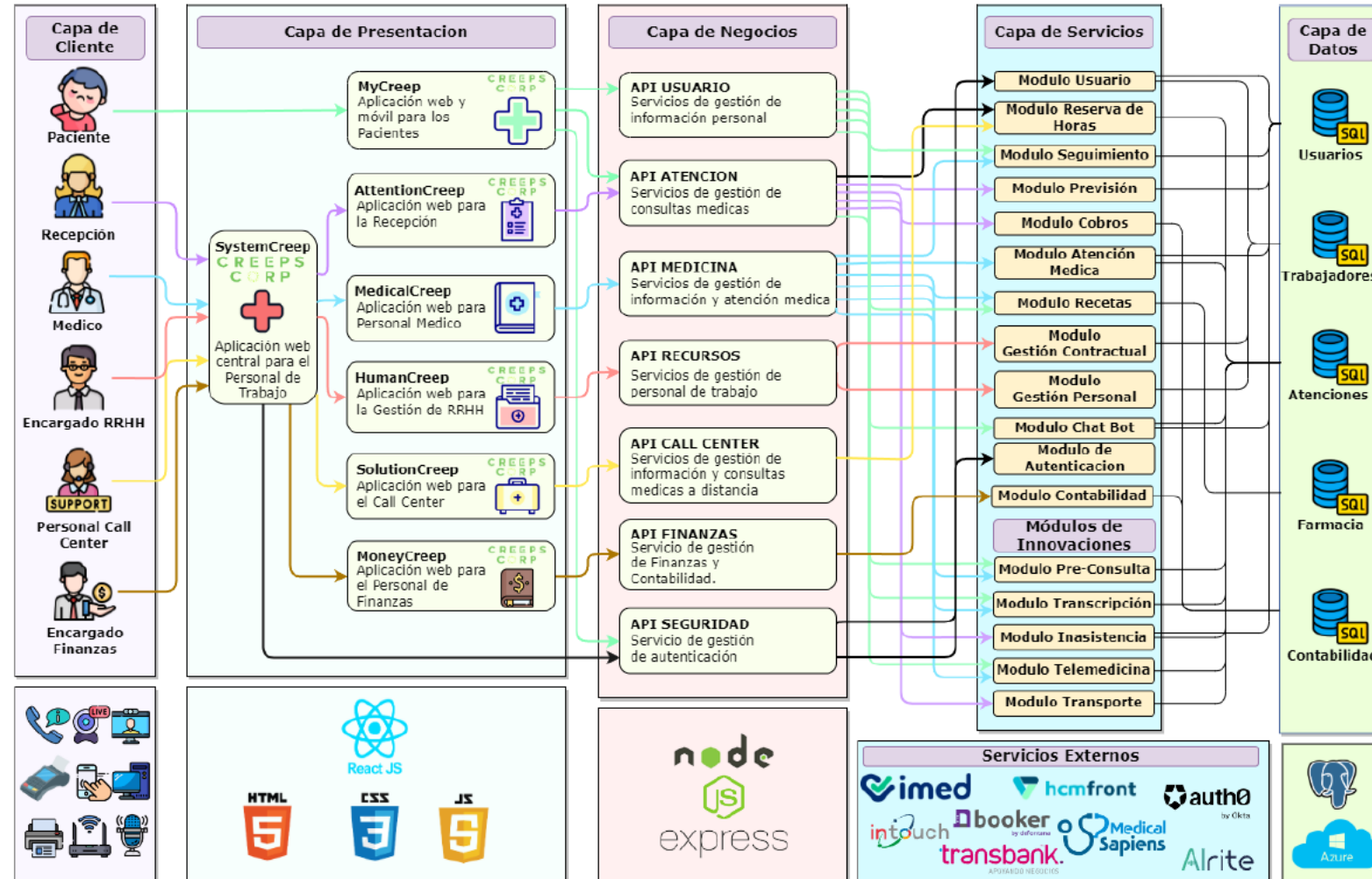
Arquitectura de Software

Ejemplo de Trabajo de años anteriores



Arquitectura de Software

Ejemplo de Trabajo de años anteriores



Sección 2 · Arquitectura lógica

1

Monolito modular

Lea sobre «modular monolith» y sobre el patrón strangler fig. Prepare el argumento de por qué su caso parte modular.

2

Patrones de integración

Revise el catálogo clásico de Enterprise Integration Patterns e identifique cuáles aplican a las integraciones de su caso.

3

Diseño de APIs REST

Estudie una guía de diseño de APIs y documente una de sus interfaces con OpenAPI, incluyendo versión y códigos de error.

4

Saga y consistencia eventual

Busque ejemplos del patrón saga con compensación y decida si su caso tolera consistencia eventual o no.

5

Bosqueje su diagrama

Elija uno de los cuatro estilos, dibuje su arquitectura lógica e intercámbiela con otro grupo: si no la entienden sin explicación, todavía no sirve.

03

SECCIÓN 3

Arquitectura física

- Las 7 capas del modelo OSI y dónde actúa cada componente de su arquitectura
- Topología on-premise: perímetro, DMZ, balanceo, servidores, datos y respaldo
- El data center: energía, climatización, seguridad física y niveles TIER
- Virtualización y contenedores: máquina virtual vs. contenedor
- Motor de base de datos, base de datos y almacenamiento: tres cosas distintas
- Dimensionamiento con parámetros verificables y costos CAPEX
- Master, worker e hipervisor: por qué se necesitan y en qué cantidad
- Los niveles RAID y cuántos discos hay que comprar para la capacidad prometida

Arquitectura de Software

¿Qué es la arquitectura física?

La vista física muestra la ubicación del software en el hardware. Responde una sola pregunta: ¿dónde se ejecuta cada cosa, y sobre qué se ejecuta?

Nodos

Equipos físicos o virtuales donde se ejecuta software: servidores, contenedores, dispositivos.

Artefactos

Lo que se despliega en cada nodo: aplicaciones, servicios, bases de datos, agentes.

Conexiones

Redes, protocolos, puertos y anchos de banda entre nodos.

Ubicación

Dónde están físicamente esos nodos: sala del mandante, data center, región de nube.

Capacidad

Cuánto puede procesar cada nodo: CPU, memoria, disco, entrada/salida.

Redundancia

Qué está duplicado, qué no lo está y qué pasa cuando algo falla.

Arquitectura de Software

Las 7 capas del modelo OSI

El modelo OSI describe la comunicación en red como siete capas, cada una con una responsabilidad acotada. Cada capa usa los servicios de la de abajo y presta servicio a la de arriba.

#	Capa	De qué se hace responsable	Unidad	Ejemplos
7	Aplicación	Expone el servicio al software: define el significado de la conversación.	Datos	HTTP/HTTPS, DNS, SMTP, FTP, gRPC
6	Presentación	Formato, codificación, compresión y cifrado del contenido.	Datos	TLS, JSON/XML, UTF-8, JPEG
5	Sesión	Establece, mantiene y cierra el diálogo entre dos extremos.	Datos	Sesiones TLS, RPC, NetBIOS
4	Transporte	Entrega extremo a extremo: puertos, fiabilidad y control de flujo.	Segmento	TCP, UDP, QUIC, puertos 80/443/1433
3	Red	Direccionamiento lógico y enrutamiento entre redes distintas.	Paquete	IP, ICMP, enrutadores, subredes, NAT
2	Enlace de datos	Entrega dentro del mismo segmento físico y detección de errores.	Trama	Ethernet, MAC, VLAN, switches
1	Física	Transmisión de bits por el medio: señales, conectores, cableado.	Bit	Cobre, fibra, Wi-Fi, patch panel

Regla mnemotécnica de abajo hacia arriba: Física, Enlace, Red, Transporte, Sesión, Presentación, Aplicación.

Arquitectura de Software

Su arquitectura, mapeada sobre OSI

Cada componente de la arquitectura física actúa en una capa determinada. Saber cuál explica qué puede hacer ese componente... y qué no.

Fundamentos			
Lógica			
Física			
Nube			
Calidad			
Continuidad			
Seguridad			
Identidad			
Entrega			
Tendencias			
Propuesta			
	7 · Aplicación	WAF · API Gateway · balanceador de capa 7 · CDN · servidor web	Entiende URLs, cabeceras y contenido. Puede enrutar por ruta, bloquear inyección SQL y limitar por usuario.
	6 · Presentación	Terminación TLS · certificados · compresión	Aquí se cifra y se descifra. Decidir dónde termina el TLS es una decisión de arquitectura y de seguridad.
	5 · Sesión	Sesiones de usuario · sesiones persistentes en el balanceador	Aquí aparece el problema del estado: si la sesión vive en el nodo, no puede escalar horizontalmente.
	4 · Transporte	Balanceador de capa 4 · grupos de seguridad · reglas por puerto	Sólo ve IP y puerto: es más rápido, pero no puede decidir según el contenido de la petición.
	3 · Red	Enrutadores · firewall perimetral · VPN · subredes · NAT	Aquí se hace la segmentación de red: quién puede alcanzar a quién.
	2 · Enlace	Switches · VLAN · agregación de enlaces	Separación lógica dentro del mismo cableado. Relevante on-premise.
	1 · Física	Cableado · fibra · enlaces del proveedor · energía	Redundancia de camino físico: dos proveedores distintos, no dos contratos con el mismo.

Arquitectura de Software

Para qué le sirve OSI en la oferta

Ordenar el diagrama físico

Dibuje de abajo hacia arriba: enlaces, red y segmentación, transporte, y recién arriba el balanceo y la aplicación. El diagrama se vuelve legible y demuestra criterio.

Elegir el balanceo correcto

Capa 4 reparte por IP y puerto: rápido y barato. Capa 7 lee la petición: puede enrutar por ruta, hacer despliegue progresivo y aplicar reglas por usuario. Cuesta más.

Ubicar cada control de seguridad

Firewall en capa 3-4, WAF en capa 7, cifrado en capa 6, segmentación en capa 2-3. Defensa en profundidad significa exactamente esto: un control por capa.

Diagnosticar y dimensionar

Ante un problema, se descarta capa por capa. Y la latencia total es la suma de lo que aporta cada capa: enlace, enrutamiento, TLS y proceso de la aplicación.

Redacción tipo para el informe: «El balanceador opera en capa 7 para permitir enrutamiento por ruta y despliegue progresivo; el filtrado por puerto se resuelve en capa 4 mediante grupos de seguridad, y la segmentación entre subredes en capa 3.»

Arquitectura de Software

Del diagrama lógico al de despliegue

La arquitectura física no se inventa: se deriva. Cada componente lógico tiene que aterrizar en algún nodo, y esa correspondencia se declara.

Componente lógico	Nodo de despliegue	Dimensionamiento	Redundancia
Portal web de clientes	Servidores web (granja)	2 × 2 vCPU / 4 GB	Activo-activo tras balanceador
Módulo de pedidos	Servidores de aplicación	2 × 4 vCPU / 8 GB	Activo-activo
Módulo de facturación	Servidores de aplicación	1 × 4 vCPU / 8 GB	Activo-pasivo
Base de datos transaccional	Clúster de base de datos	2 × 8 vCPU / 32 GB / 500 GB SSD	Réplica sincrónica + failover
Almacén de documentos	Almacenamiento de objetos	2 TB con crecimiento de 40 GB/mes	Replicado
Integración con el ERP del mandante	Servidor de integración	1 × 2 vCPU / 4 GB	Reintento y cola de respaldo

Los valores del ejemplo son ilustrativos: en su informe cada cifra debe derivarse del volumen de operación declarado en las bases.

Arquitectura de Software

Topología clásica on-premise

ZONA
PÚBLICA

Usuarios · Internet

el tráfico llega desde fuera

Firewall perimetral

filtra y publica sólo lo necesario

DMZ

Balanceador de carga / proxy inverso

reparte, corta TLS, saca nodos caídos

Servidores web ×2

capa de presentación

RED
INTERNA

Servidores de aplicación ×2

lógica de negocio

Clúster de base de datos (activo + réplica)

datos transaccionales

ZONA DE
DATOS

Almacenamiento SAN + Respaldo

persistencia y recuperación

Arquitectura de Software

Segmentación de la red

La regla básica: cada zona sólo puede conversar con la siguiente. Nunca se publica una base de datos a Internet.

Zona pública

Sólo se exponen los puertos 80 y 443 hacia Internet. Todo lo demás está cerrado por defecto.

DMZ

Zona intermedia donde viven proxy inverso, balanceador y WAF. No guarda datos sensibles.

Red de aplicación

Subred privada, sin acceso directo desde Internet. Sólo acepta tráfico desde la capa web.

Red de datos

Máximo aislamiento. Sólo acepta conexiones desde la capa de aplicación, en el puerto del motor.

Red de gestión

Segmento separado para administración, monitoreo y respaldo. Acceso restringido y auditado.

Acceso remoto

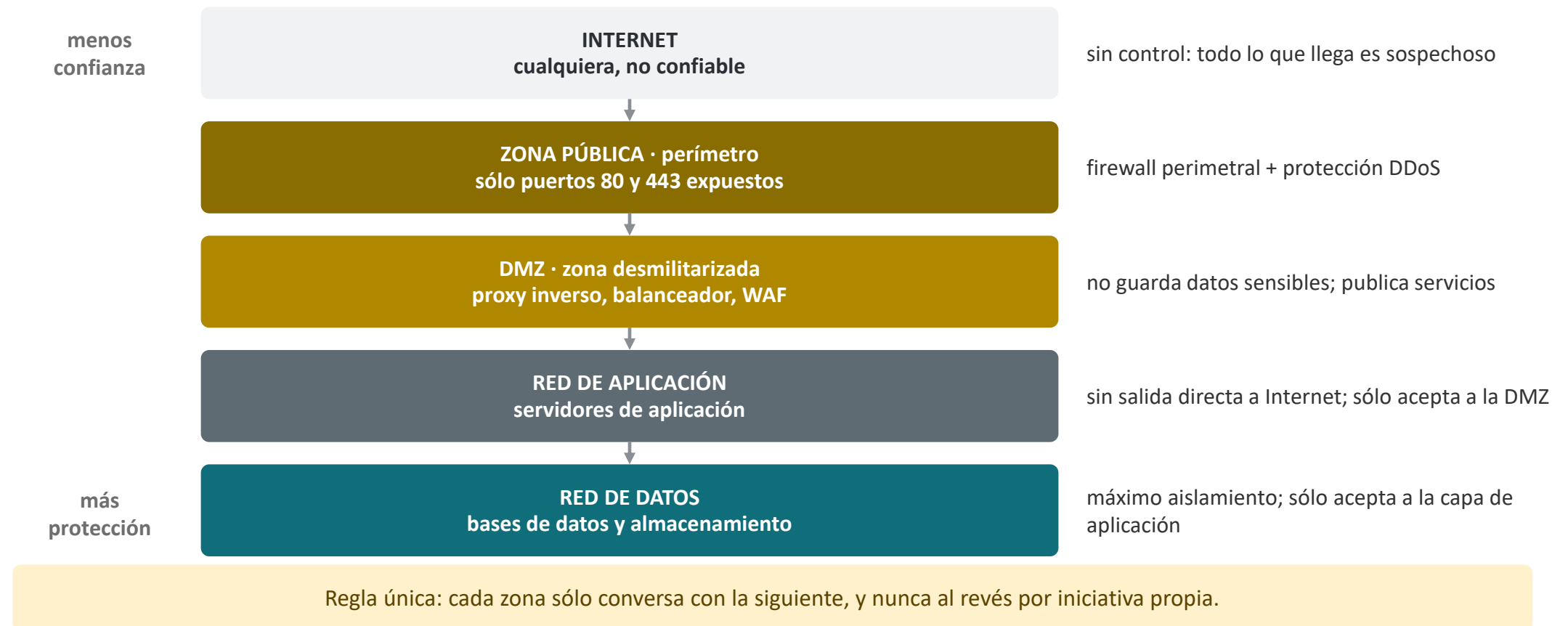
VPN o bastión con doble factor. Nunca administración expuesta directamente a Internet.

Defensa en profundidad: si un atacante supera una capa, todavía tiene otra por delante. En su oferta esto se traduce en equipos concretos (firewall, WAF) y en reglas declaradas, no en la frase «la solución será segura».

Arquitectura de Software

Las zonas de red, en un diagrama

Cada zona es un anillo. El tráfico sólo puede avanzar hacia el anillo siguiente, nunca saltarse uno.



Arquitectura de Software

El data center

Cuando la solución vive en instalaciones del mandante, la infraestructura de soporte también es parte de la arquitectura y del costo.

Energía

Alimentación redundante, UPS y grupo electrógeno. Sin energía no hay disponibilidad.

Climatización

Temperatura y humedad controladas, con redundancia. El calor es la principal causa de falla.

Seguridad física

Control de acceso, cámaras, detección y extinción de incendios, registro de ingreso.

Conectividad

Enlaces redundantes con proveedores distintos y cableado estructurado certificado.

Espacio y racks

Gabinetes, piso técnico, pasillos fríos y calientes, capacidad de crecimiento.

Operación

Personal, turnos, monitoreo, mantenimiento preventivo y contratos de soporte.

Nivel TIER	Redundancia	Disponibilidad aproximada	Inactividad anual estimada
TIER I	Sin redundancia	99,671%	≈ 28,8 horas
TIER II	Componentes redundantes	99,741%	≈ 22,0 horas
TIER III	Mantenimiento sin detener el servicio	99,982%	≈ 1,6 horas
TIER IV	Tolerante a fallas, todo duplicado	99,995%	≈ 0,4 horas

Arquitectura de Software

Del servidor físico a la virtualización

La evolución ha sido pasar de servidores físicos en un data center, a servidores virtuales en ese mismo data center, a servidores virtuales en la nube, a contenedores dentro de servidores virtuales, y ahora a informática sin servidor.

1

Servidor físico

Una aplicación por equipo.
Uso típico del hardware: 10 a 20%. Máximo aislamiento, máximo desperdicio.

2

Virtualización

Un hipervisor divide un equipo físico en varias máquinas virtuales, cada una con su sistema operativo.

3

Contenedores

Varias aplicaciones comparten el mismo sistema operativo, aisladas entre sí. Arrancan en segundos.

4

Serverless

Ni siquiera se administra el servidor: se despliega la función y la plataforma resuelve el resto.

Cada escalón reduce el desperdicio de capacidad y la carga de administración, y aumenta la dependencia de la plataforma que hay debajo.

Arquitectura de Software

¿Qué es un contenedor?

Un paquete de software estándar —el contenedor— agrupa el código de una aplicación con sus bibliotecas y archivos de configuración, junto con las dependencias necesarias para que se ejecute. En esencia, virtualiza el sistema operativo a nivel de aplicación.

El problema que resuelve

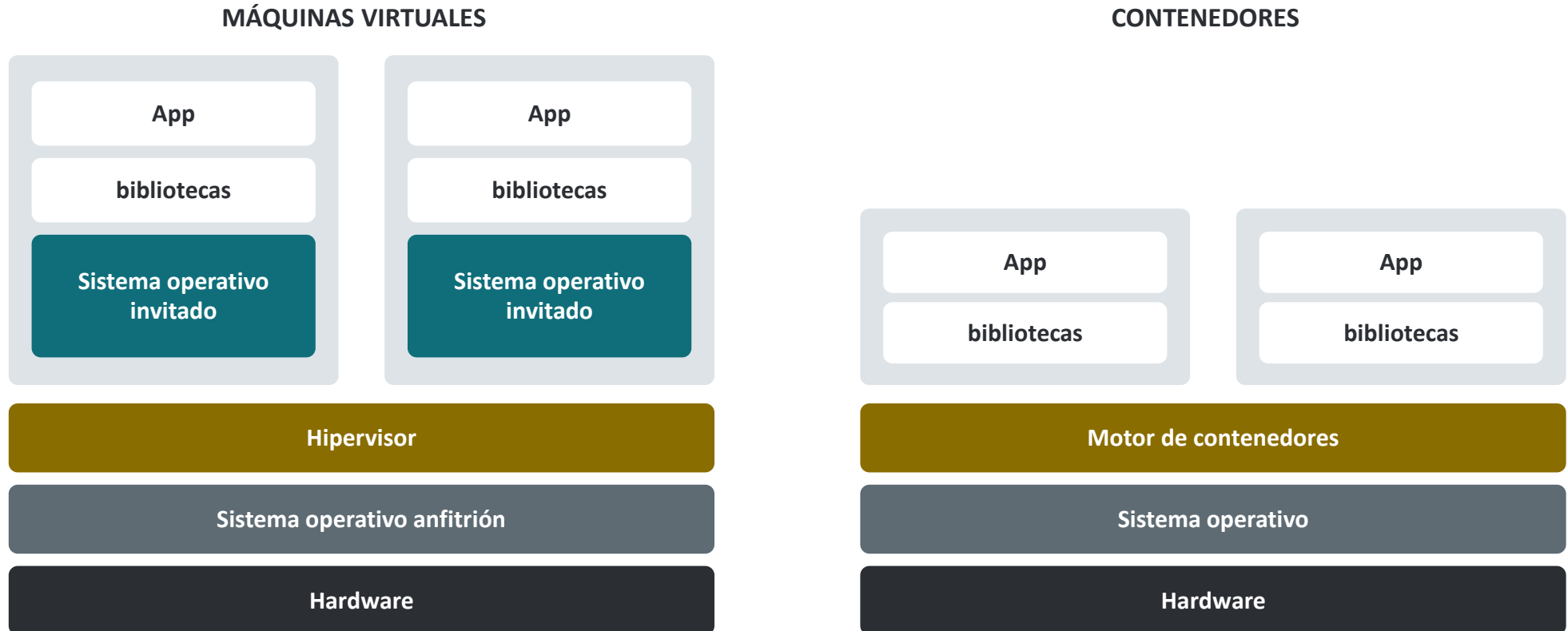
- «En mi máquina funciona»: la aplicación se comporta distinto al cambiar de entorno.
- Las diferencias vienen de versiones de bibliotecas y de configuración del sistema.
- Cada paso —desarrollo, pruebas, producción— exige ajustes manuales.
- El despliegue depende de documentación que siempre está desactualizada.

Lo que aporta

- Infraestructura ligera e inmutable para empaquetar e implementar.
- La misma imagen corre en el portátil del desarrollador y en producción.
- Arranque en segundos y consumo muy inferior al de una máquina virtual.
- Permite mover la aplicación entre entornos sin cambios, o con cambios mínimos.

Arquitectura de Software

Máquina virtual vs. contenedor

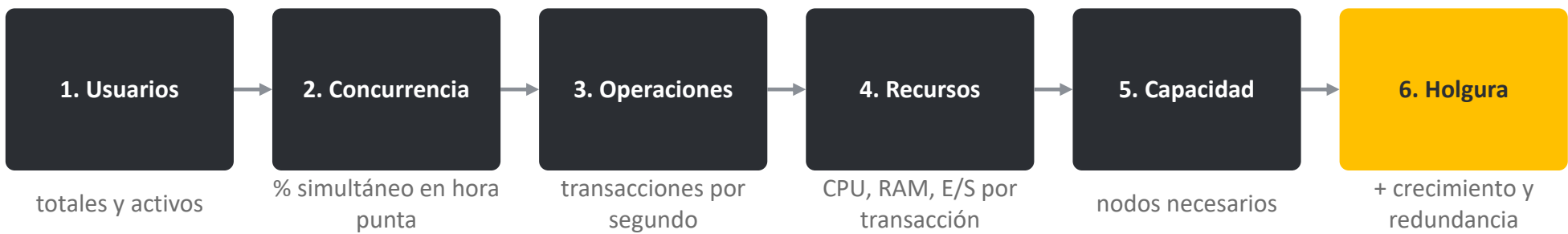


La máquina virtual virtualiza el hardware y carga un sistema operativo completo por instancia. El contenedor virtualiza el sistema operativo: menos peso, arranque más rápido, menor aislamiento.

Arquitectura de Software

Dimensionar: de la demanda al hardware

Dimensionar no es elegir un servidor grande: es una cadena de razonamiento que parte del negocio y termina en una cifra defendible.



Ejemplo trabajado:

Paso	Dato o supuesto	Resultado
Usuarios registrados declarados en las bases	12.000	—
Usuarios activos en hora punta (supuesto: 10%)	1.200 concurrentes	—
Operaciones por usuario por minuto (supuesto: 4)	4.800 op/min	80 operaciones por segundo
Capacidad medida por instancia de aplicación	35 op/s	3 instancias
Holgura de crecimiento (30%) y tolerancia a fallas (N+1)	—	4 instancias de aplicación

Arquitectura de Software

Parámetros de dimensionamiento

Recurso	Qué determina la cifra	Señal de que quedó corto	Cómo se declara en la oferta
CPU (vCPU)	Operaciones por segundo y complejidad del procesamiento	Uso sostenido sobre 70%, tiempos de respuesta que suben	N.º de vCPU por nodo y cantidad de nodos
Memoria (RAM)	Sesiones activas, caché en memoria, tamaño del conjunto de trabajo	Uso de intercambio en disco, reinicios por falta de memoria	GB por nodo
Almacenamiento	Volumen inicial + crecimiento mensual × horizonte + retención	Alertas de espacio, respaldos que no caben	GB o TB, con la tasa de crecimiento
Rendimiento de disco	Operaciones de entrada/salida por segundo del motor de datos	Esperas de disco, consultas lentas sin causa aparente	IOPS y tipo de disco (SSD/NVMe)
Red	Tamaño medio de respuesta × operaciones por segundo	Saturación del enlace en hora punta	Mbps por enlace y redundancia
Licencias	Núcleos, usuarios nominados o instancias, según el fabricante	Costo que aparece después de la adjudicación	Modelo y cantidad, con su costo anual



Todo supuesto se declara. «Se estima un 10% de concurrencia en hora punta, según el patrón de uso descrito en las bases» es defendible; una cifra sin origen no lo es.

Arquitectura de Software

Dimensionamiento de la base de datos

Tecnología

Relacional para transacciones, documental para datos flexibles, clave-valor para caché, columnar para analítica. Justifique la elección con el tipo de dato del caso.

Tamaño

Registros iniciales $\times$ tamaño medio de fila + índices + crecimiento mensual $\times$ horizonte de evaluación + margen. Declare la fórmula, no sólo el número.

Uptime

Disponibilidad comprometida y esquema que la sostiene: réplica sincrónica, failover automático, ventana de mantenimiento acordada.

Tiempos de respuesta

Meta por tipo de consulta, medida en percentil 95. Diferencie consulta transaccional de reporte.

Retención y purga

Cuánto tiempo se conserva cada tipo de dato, qué se archiva y qué se elimina. Impacta directamente en el tamaño.

Respaldo y recuperación

Frecuencia, tipo (completo/incremental), destino, y sobre todo: tiempo que toma restaurar. Un respaldo que nunca se probó no es un respaldo.

Regla 3-2-1 de respaldo: tres copias de los datos, en dos medios distintos, con una fuera del sitio. Es un estándar citable y barato de justificar en la oferta.

Arquitectura de Software

Motor, base de datos y almacenamiento

Tres cosas distintas, tres decisiones distintas y tres líneas distintas en el presupuesto.

Motor de base de datos

El SOFTWARE que administra los datos: recibe consultas, controla concurrencia, garantiza integridad, gestiona respaldos y seguridad.

Es un producto: tiene versión, licencia, requisitos de hardware y ciclo de soporte.

En el presupuesto: licencia y mantención anual, o costo del servicio gestionado.

Base de datos

El CONJUNTO DE DATOS y su estructura: tablas, índices, relaciones, vistas y procedimientos, que el motor administra.

Es lo que usted diseña: modelo, volumen, crecimiento, retención.

En el presupuesto: no cuesta por sí misma, pero determina el tamaño del almacenamiento y la capacidad del motor.

Almacenamiento (storage)

El MEDIO FÍSICO O LÓGICO donde quedan escritos los bytes: discos, cabina, volumen en la nube, almacenamiento de objetos.

Se caracteriza por capacidad, velocidad (IOPS y latencia), durabilidad y disponibilidad.

En el presupuesto: precio por GB al mes o inversión en cabina, más el respaldo.

Analogía: el motor es el bibliotecario, la base de datos es el catálogo y la colección ordenada, y el almacenamiento son las estanterías y el edificio.

Arquitectura de Software

Motores más usados y cuándo elegir cada uno

Motor	Tipo	Licencia	Fortaleza	Cuándo elegirlo
PostgreSQL	Relacional	Libre	Muy completo, extensible, soporta JSON y datos geográficos	Opción por defecto para un sistema transaccional nuevo
MySQL / MariaDB	Relacional	Libre	Simple, muy difundido, enorme comunidad	Aplicaciones web de complejidad moderada
SQL Server	Relacional	Propietaria	Integración con el ecosistema Microsoft y herramientas de BI	El mandante ya opera con tecnología Microsoft
Oracle Database	Relacional	Propietaria	Muy robusto en cargas grandes; fuerte en banca y sector público	Ya existe la plataforma y el personal; la licencia es cara
MongoDB	Documental	Mixta	Esquema flexible, escalado horizontal natural	Datos semiestructurados o de forma variable
Redis	Clave-valor	Mixta	En memoria, latencia de microsegundos	Caché, sesiones y colas, no como almacén principal
Elasticsearch / OpenSearch	Búsqueda	Mixta	Búsqueda de texto y análisis de registros	Buscador del portal y centralización de logs
SQLite	Embebido	Libre	Sin servidor, un solo archivo	Aplicaciones locales o de escritorio; nunca multiusuario

Advertencia de costo: en los motores propietarios la licencia se cuenta por núcleo o por usuario. Duplicar servidores para alta disponibilidad puede duplicar la licencia.

Arquitectura de Software

Tipos de almacenamiento

Tipo	Qué es	Se usa para	Ventaja	Límite
Bloque	Un disco crudo que el sistema operativo formatea y monta (SAN, volumen en la nube).	Bases de datos, sistemas de archivos de los servidores, máquinas virtuales.	Latencia mínima y control total del formato.	Se conecta a un solo servidor a la vez; caro por GB.
Archivo	Un sistema de archivos compartido en red, con carpetas y permisos (NAS, NFS/SMB).	Documentos compartidos entre varios servidores, archivos de aplicación comunes.	Varios servidores lo montan al mismo tiempo.	Menor rendimiento; no escala indefinidamente.
Objeto	Cada archivo es un objeto con metadatos, accesible por API HTTP.	Documentos escaneados, imágenes, respaldos, archivos estáticos del portal.	Capacidad prácticamente ilimitada, muy barato, durabilidad altísima.	No se monta como disco; no sirve para bases de datos.

Regla práctica para su oferta: la base de datos va en almacenamiento de bloque con SSD y una cifra de IOPS declarada. Los documentos y las imágenes van en almacenamiento de objetos, no dentro de la base de datos: guardar archivos en la base multiplica su tamaño, encarece la licencia del motor y hace lentos los respaldos.

Arquitectura de Software

El costo real de una solución on-premise

El servidor es la parte visible. El costo total incluye todo lo que hay que comprar, mantener y renovar durante el horizonte de evaluación.

Categoría	Ítems típicos	Naturaleza
Hardware	Servidores, almacenamiento, switches, firewall, UPS, racks	Inversión (CAPEX)
Software base	Sistemas operativos, virtualización, motor de base de datos, respaldo	Inversión + mantención anual
Instalación	Habilitación de sala, cableado, energía, climatización, puesta en marcha	Inversión (CAPEX)
Operación	Personal de operaciones, turnos, monitoreo, soporte del fabricante	Costo anual (OPEX)
Conectividad	Enlaces, redundancia de proveedor, direcciones IP, certificados	Costo anual (OPEX)
Energía y espacio	Consumo eléctrico, climatización, arriendo del espacio	Costo anual (OPEX)
Renovación	Reemplazo de hardware al final de su vida útil (3 a 5 años)	Reinversión en el flujo

Todos estos ítems se van a repetir en el flujo de caja del proyecto. Levantarlos ahora evita rehacer la evaluación económica después.

Arquitectura de Software

Cómo se dibuja una arquitectura física

Si el diagrama lógico responde «de qué partes se compone», el físico responde «dónde corre y cuánto aguanta». Debe permitir dos lecturas: la de fallas y la de costos.

Zonas de red visibles

Cajas que agrupen Internet, borde, subred pública, subred de aplicación y subred de datos. La segmentación se ve de un vistazo.

Nodos con su capacidad

Cada nodo con su dimensionamiento anotado: «2 × 4 vCPU / 8 GB». Sin cifras es un dibujo, no una arquitectura.

Redundancia explícita

Qué está duplicado y en cuántas zonas. Lo que aparece una sola vez es un punto único de falla.

Protocolos y puertos

En las líneas importantes: HTTPS 443, el puerto del motor de datos, el túnel IPsec.

Nombres de servicio

Si es nube, el nombre del producto: no «balanceador» sino «Application Load Balancer». Es lo que después se cotiza.

Frontera del alcance

Qué provee usted, qué es del mandante y qué es de un tercero. Tres colores bastan, con su leyenda.



Prueba rápida: si al mirar su diagrama físico no puede señalar con el dedo qué componente falla primero ni cuál es el más caro del mes, todavía le falta información.

Arquitectura de Software

Cuándo on-premise sigue siendo la respuesta correcta

Normativa o contrato

El mandante está obligado a mantener los datos en instalaciones propias o dentro de un perímetro determinado.

Latencia física

El sistema controla equipos, planta industrial o procesos que no toleran la latencia de una red externa.

Carga estable y predecible

Sin picos ni estacionalidad, la elasticidad de la nube no aporta y el costo mensual termina siendo mayor.

Inversión ya realizada

El mandante tiene data center vigente, licencias y personal. Migrar sería destruir valor.

Conectividad limitada

La operación está en una zona sin enlace confiable: depender de Internet sería el mayor riesgo del proyecto.

Integración con equipamiento

El sistema debe conversar con hardware específico que sólo está disponible en la red local.



Lo que el evaluador castiga no es elegir on-premise ni elegir nube: es no haber comparado. La decisión debe salir de un cuadro con criterios explícitos.

Arquitectura de Software

Tres piezas que hay que entender antes

En cualquier inventario de infraestructura moderna va a leer «Master», «Worker» e «hipervisor». Son tres cosas de niveles distintos, y confundirlas lleva a dimensionar mal.

Hipervisor

Software que parte un servidor físico en varias máquinas virtuales.

NIVEL: entre el hardware y el sistema operativo.

Ejemplos: VMware ESXi, Proxmox, Hyper-V, KVM.

Nodo Master (manager)

Máquina virtual que decide QUÉ se ejecuta y DÓNDE. Guarda el estado del clúster.

NIVEL: plano de control del orquestador.

Ejemplo: Docker Swarm manager, Kubernetes control plane.

Nodo Worker

Máquina virtual que EJECUTA los contenedores. Aporta la capacidad real de cómputo.

NIVEL: plano de trabajo del orquestador.

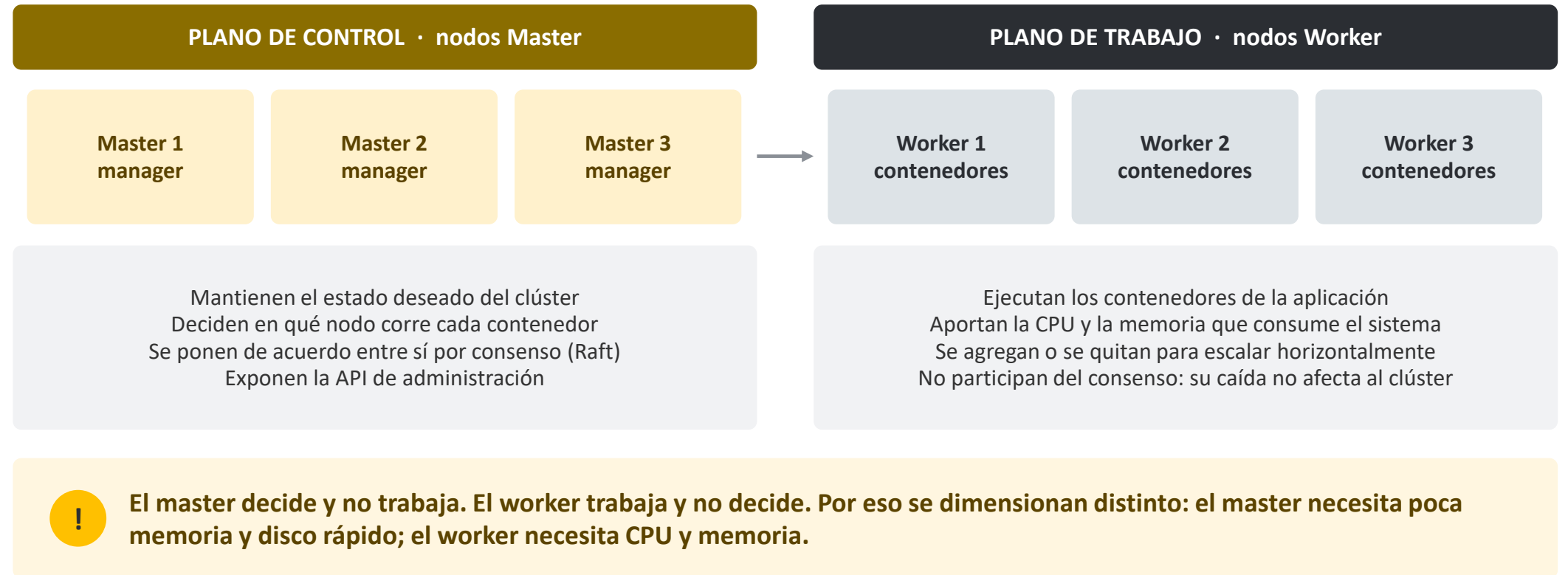
Ejemplo: Docker Swarm worker, Kubernetes node.

El hipervisor crea las máquinas virtuales; el orquestador reparte contenedores entre esas máquinas virtuales. Son dos repartos distintos, uno sobre el otro.

Arquitectura de Software

Plano de control y plano de trabajo

El orquestador separa dos responsabilidades: decidir y ejecutar. Esa separación es la razón de que existan dos tipos de nodo.



Arquitectura de Software

Qué hace exactamente un nodo Master

Guarda el estado deseado

Qué servicios deben existir, con cuántas réplicas, en qué red y con qué configuración. Es la «verdad» del clúster.

Programa los contenedores

Elige en qué worker corre cada réplica, según recursos libres, etiquetas y reglas de reparto.

Reconcilia

Compara continuamente lo que hay con lo que debería haber. Si un contenedor muere, lo vuelve a levantar en otro nodo.

Mantiene el consenso

Los masters replican el estado entre ellos con el algoritmo Raft. Uno es líder; el resto lo sigue y lo reemplaza si cae.

Expone la API y la red

Recibe los comandos de administración y publica la red interna del clúster y el enrutamiento entre servicios.

Gestiona secretos y certificados

Guarda contraseñas y claves cifradas y rota los certificados internos entre nodos.

Consecuencia de dimensionamiento: el master no necesita mucha memoria (16 GB en el caso), pero sí disco rápido y baja latencia de red, porque el consenso escribe en disco en cada cambio. En producción se recomienda no ejecutar carga de la aplicación en los masters.

Arquitectura de Software

Qué hace exactamente un nodo Worker

Ejecuta contenedores

Recibe la orden del master y levanta los contenedores asignados, con sus límites de CPU y memoria.

Reporta su estado

Informa periódicamente cuánto recurso tiene libre y si sus contenedores están sanos.

Aporta la capacidad

La suma de vCPU y RAM de los workers es la capacidad real del sistema. Los masters no cuentan para eso.

Escala horizontalmente

Cuando la carga crece, se agregan workers. No hay que reconfigurar la aplicación.

Es reemplazable

Si un worker cae, el master reprograma sus contenedores en los que quedan. Por eso deben sobrar recursos.

No guarda estado

Todo lo que deba persistir va a la base de datos o al almacenamiento compartido, nunca al disco local del worker.



Regla de capacidad: si tiene 3 workers y quiere sobrevivir a la caída de uno, cada worker no puede pasar del 66% de uso. Ese margen se cotiza, no se improvisa.

Arquitectura de Software

¿Por qué tres masters, y no uno ni dos?

Los masters se ponen de acuerdo por consenso: sólo pueden decidir si hay quórum, es decir, si está disponible más de la mitad de ellos. Quórum = parte entera de $(N \div 2) + 1$.

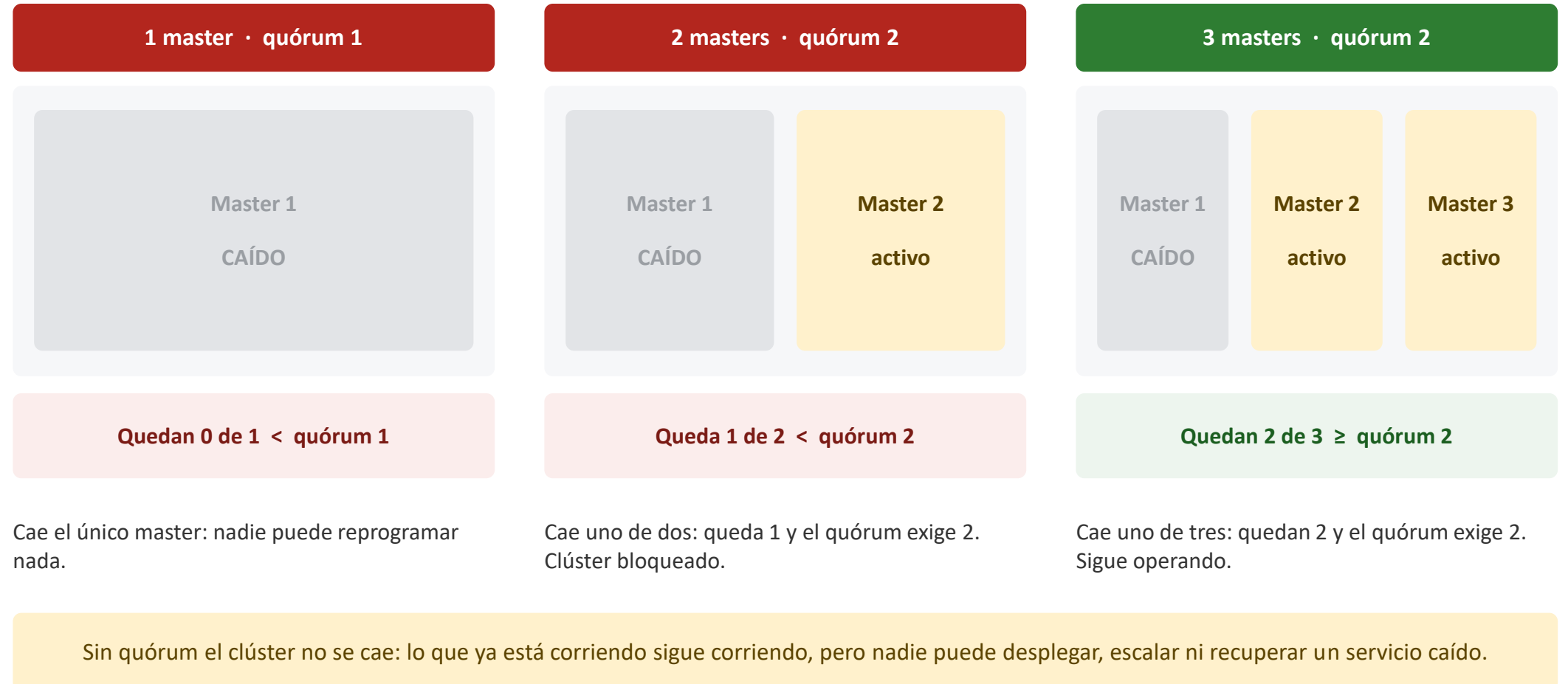
N.º de masters	Quórum necesario	Fallas que tolera	Veredicto
1	1	0	Punto único de falla del plano de control
2	2	0	Peor que uno: hay dos equipos que pueden romperlo
3	2	1	Mínimo razonable en producción
4	3	1	Un equipo más, sin ganar tolerancia: no se justifica
5	3	2	Para clústeres grandes o repartidos en dos salas
7	4	3	Máximo práctico: más masters hacen el consenso más lento



Siempre un número impar, y tres es el primero que tolera una falla. Ése es el «por qué 3» que se lee en todos los documentos de arquitectura: no es superstición, es aritmética del quórum.

Arquitectura de Software

El quórum, en un diagrama



Arquitectura de Software

¿Y cuántos workers?

El número de masters lo fija el quórum. El número de workers lo fija la capacidad más el margen para sobrevivir a la caída de uno.

Paso	Cómo se obtiene	Ejemplo trabajado
1. Demanda de recursos	Suma de CPU y memoria que piden todos los contenedores en hora punta	48 vCPU y 84 GB solicitados
2. Capacidad por worker	Recursos de la máquina virtual menos lo que consume el sistema operativo	8 vCPU y 32 GB, útiles $\approx$ 7 vCPU y 28 GB
3. Workers mínimos	$\text{Demanda} \div \text{capacidad útil}$, redondeado hacia arriba	$48 \div 7 \approx 7$... limitado por memoria: $84 \div 28 = 3$
4. Margen por falla (N+1)	Un worker más, para absorber la caída de cualquiera	$3 + 1 = 4$ nodos si se quiere tolerancia plena
5. Ajuste por presupuesto	Se acepta operar degradado tras una falla y se deja el mínimo	3 workers, con 66% de uso máximo

La configuración habitual es 3 workers, uno por servidor físico. No es casualidad: así la caída de un servidor se lleva un solo worker, y los dos que quedan deben poder absorber su carga.

Arquitectura de Software

¿Por qué se necesita un hipervisor?

Un clúster típico pide entre 6 y 14 nodos. Los servidores físicos que se compran son 1, 2 o 3. El hipervisor es lo que hace posible esa diferencia.

Más nodos que equipos

Un clúster necesita al menos 3 masters. Comprar 3 servidores físicos sólo para el plano de control sería absurdo: se crean como máquinas virtuales.

Sistemas operativos distintos

Es habitual que convivan Oracle Linux, Rocky Linux y Windows Server en la misma solución. No pueden coexistir en un mismo sistema operativo: exigen máquinas separadas.

Aislamiento de fallas

Un núcleo que se cuelga o una actualización que sale mal afectan a una máquina virtual, no a todo el servidor.

Límites de recursos por nodo

Se fija cuánta CPU, memoria y disco puede usar cada nodo. Sin eso, un proceso desbocado se lleva el equipo completo.

Movilidad y respaldo

Una máquina virtual se mueve a otro servidor, se clona para pruebas y se respalda completa. Un servidor físico no.

Aprovechamiento del hardware

Un servidor físico dedicado a una función se usa al 10-20%. Con varias máquinas virtuales encima se llega al 60-70%.

Costo asociado: el hipervisor se licencia (por socket o por núcleo en los productos comerciales) y consume del orden de un 10% de la CPU y la memoria del equipo. Ambas cosas se cotizan.

Arquitectura de Software

Cuándo sí y cuándo no se cotiza un hipervisor

Se cotiza como línea aparte

- Solución on-premise con servidores propios del mandante.
- Producto comercial: VMware vSphere, Microsoft Hyper-V con System Center, Nutanix.
- Se licencia por socket o por núcleo físico, con soporte anual.
- Hay que sumar además la consola de administración y el respaldo de máquinas virtuales.
- Alternativas libres: Proxmox VE, KVM/oVirt, XCP-ng. Sin licencia, con soporte opcional pagado.

No se cotiza como línea aparte

- Nube pública: el hipervisor es del proveedor y está incluido en el precio por hora de la instancia.
- Servicios de contenedores gestionados: ni siquiera se ve la máquina virtual.
- Servidores dedicados en arriendo con hipervisor incluido en el contrato.
- Nodos físicos dedicados (bare metal) donde cada nodo del clúster es un equipo completo: no hay virtualización.
- Contenedores directamente sobre el sistema operativo del servidor, sin capa intermedia.

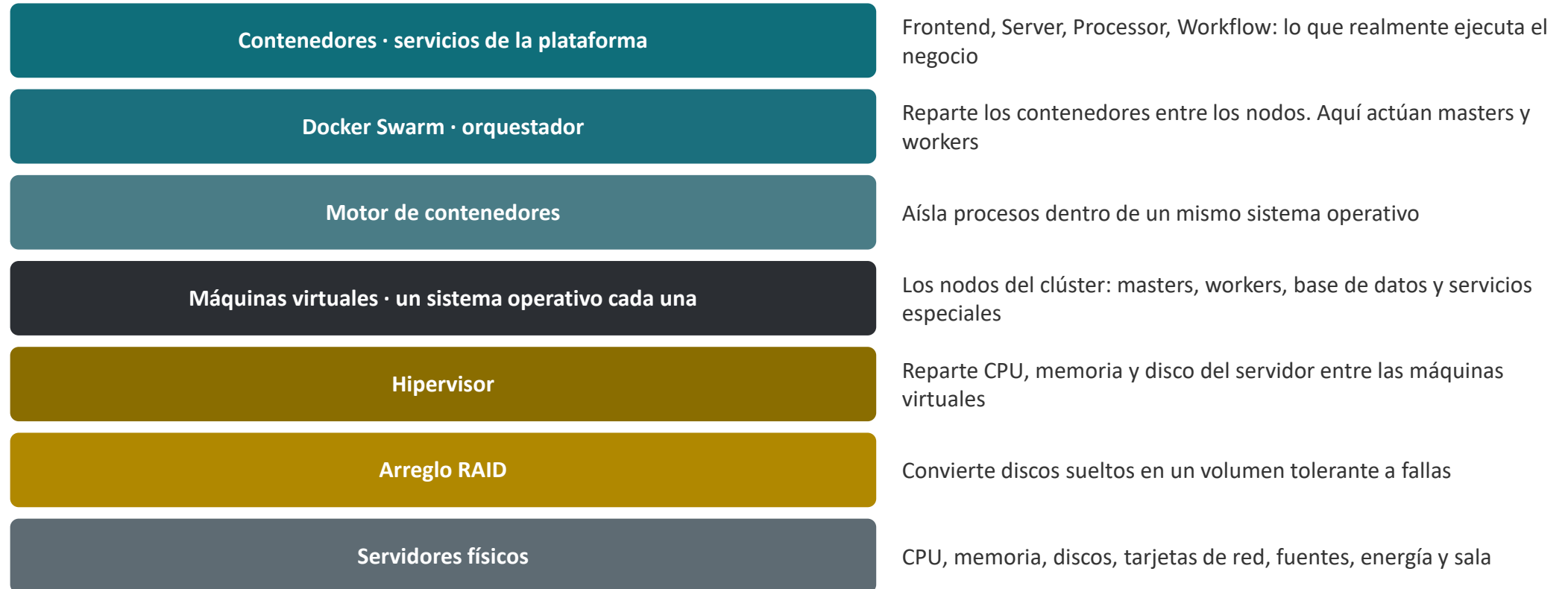


Error frecuente en las ofertas: cotizar instancias de nube y además licencias de virtualización. Se está pagando dos veces lo mismo.

Arquitectura de Software

Las capas, de abajo hacia arriba

Cada capa toma el recurso de la de abajo y lo reparte entre varios consumidores de la de arriba. Hay dos repartos superpuestos: el del hipervisor y el del orquestador.



Cada capa se puede cambiar sin tocar las de arriba: ése es el valor de la separación, y también la razón de que haya tantas piezas que cotizar.

Arquitectura de Software

RAID: qué es y qué no es

RAID —arreglo redundante de discos independientes— combina varios discos físicos para que el sistema operativo vea un solo volumen, más rápido, más grande o más tolerante a fallas.

Qué resuelve

Que la rotura de un disco no detenga el sistema ni destruya los datos, y que varios discos trabajando juntos den más velocidad.

Las tres palancas

División en franjas (striping) para velocidad, espejo (mirroring) para redundancia y paridad para redundancia barata.

Siempre hay un costo

La redundancia se paga en capacidad: parte de los discos comprados no almacena datos útiles. Ese factor entra en la cotización.

Lo que NO resuelve

Borrado accidental, corrupción lógica, cifrado por ransomware, incendio de la sala. Para eso está el respaldo, que es otra cosa.

Frase para el informe: «El arreglo RAID protege la continuidad ante falla de un disco; la protección ante pérdida lógica de datos se resuelve con la política de respaldo descrita en el punto X.»

Arquitectura de Software

Fundamentos

Lógica

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

RAID 0 · división en franjas (striping)

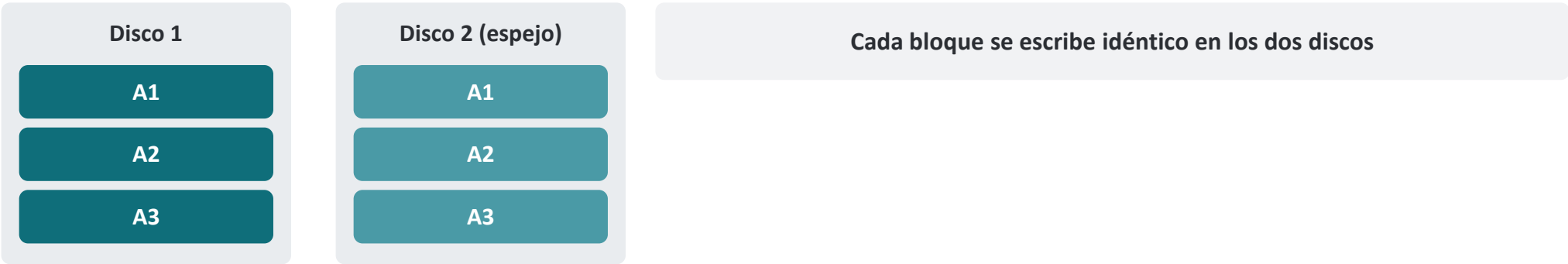


Aspecto	RAID 0
Discos mínimos	2
Capacidad útil	100% de la capacidad comprada
Tolera fallas	Ninguna: si muere un disco, se pierde todo el volumen
Velocidad de lectura	Muy alta: los discos trabajan en paralelo
Velocidad de escritura	Muy alta
Riesgo real	Con 2 discos, la probabilidad de falla del conjunto es el doble que la de uno solo
Cuándo usarlo	Datos temporales, caché, espacio de trabajo que se puede volver a generar

Regla: RAID 0 aumenta el rendimiento y aumenta el riesgo. Nunca para datos que no se puedan volver a producir.

Arquitectura de Software

RAID 1 · espejo (mirroring)

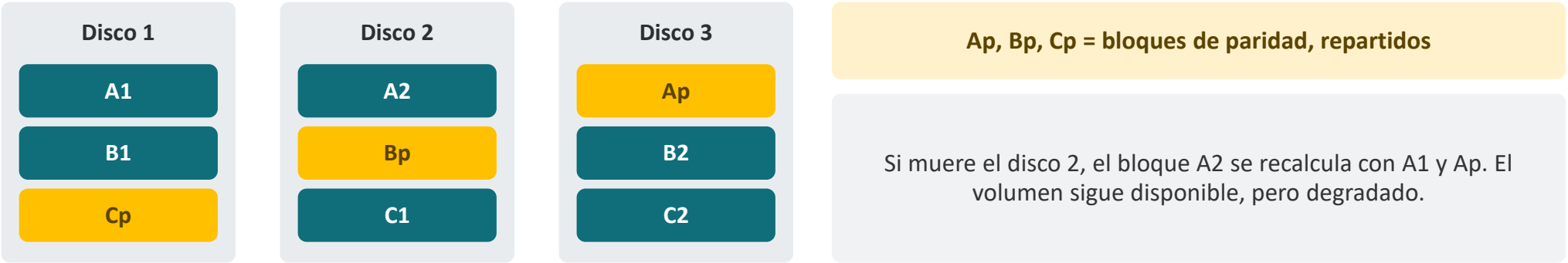


Aspecto	RAID 1
Discos mínimos	2
Capacidad útil	50% de la capacidad comprada: 2 discos de 1 TB dan 1 TB útil
Tolera fallas	1 disco. El sistema sigue funcionando sin interrupción
Velocidad de lectura	Buena: se puede leer de cualquiera de los dos
Velocidad de escritura	Igual a la de un disco solo: hay que escribir dos veces
Reconstrucción	Simple y rápida: se copia el disco sano al de reemplazo
Cuándo usarlo	Disco de arranque del sistema operativo y del hipervisor: es la regla del caso

Por eso el documento dice «la máquina física debe estar en RAID 1»: se refiere a los discos desde donde arranca el servidor, no al almacenamiento de datos.

Arquitectura de Software

RAID 5 · franjas con paridad distribuida



Aspecto	RAID 5
Discos mínimos	3
Capacidad útil	(N – 1) discos: con 5 discos de 1 TB quedan 4 TB útiles
Tolera fallas	1 disco
Penalización de escritura	Cada escritura implica 4 operaciones de disco: leer dato, leer paridad, escribir dato, escribir paridad
Riesgo de reconstrucción	Con discos grandes puede tardar horas o días, con el arreglo degradado y sin margen
Cuándo usarlo	Archivos y datos de lectura predominante, con discos de capacidad moderada

Arquitectura de Software

RAID 6 · doble paridad



Dos bloques de paridad independientes por franja: p y q. Se puede reconstruir aunque falten dos discos.

Aspecto	RAID 6
Discos mínimos	4
Capacidad útil	(N – 2) discos: con 6 discos de 1 TB quedan 4 TB útiles
Tolera fallas	2 discos simultáneos
Penalización de escritura	6 operaciones por escritura: es el más lento para escribir
Ventaja clave	Sobrevive a que falle un segundo disco mientras se reconstruye el primero
Cuándo usarlo	Arreglos grandes de discos de alta capacidad, con datos de lectura predominante

Arquitectura de Software

Fundamentos

RAID 10 · espejo y franjas combinados

Lógica

RAID 1 + 0: primero se forman parejas en espejo, y después los datos se reparten en franjas entre esas parejas.

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta



Aspecto	RAID 10
Discos mínimos	4, y siempre un número par
Capacidad útil	50% de la capacidad comprada
Tolera fallas	Hasta la mitad del arreglo (1 por espejo); 2 del mismo espejo lo destruyen
Velocidad de escritura	La mejor de todos: no hay que calcular paridad
Reconstrucción	Rápida y de bajo riesgo: se copia el disco espejo, sin recalculr el arreglo
Cuándo usarlo	Bases de datos, máquinas virtuales y carga de escritura intensa: la regla del caso

Arquitectura de Software

Los niveles RAID, comparados

Nivel	Discos mín.	Capacidad útil	Tolera	Escritura	Factor de compra	Uso típico
RAID 0	2	100%	0 discos	Muy rápida	×1,0	Datos temporales
RAID 1	2	50%	1 disco	Normal	×2,0	Disco de arranque
RAID 5	3	$(N-1)/N$	1 disco	Lenta (4 E/S)	×1,25 a ×1,5	Archivos, lectura
RAID 6	4	$(N-2)/N$	2 discos	Muy lenta (6 E/S)	×1,3 a ×1,5	Arreglos grandes
RAID 10	4	50%	1 por espejo	La más rápida	×2,0	Bases de datos, VM
RAID 50	6	$(N-g)/N$	1 por grupo	Media	×1,2 a ×1,4	Arreglos grandes mixtos
RAID 60	8	$(N-2g)/N$	2 por grupo	Lenta	×1,3 a ×1,5	Archivo masivo

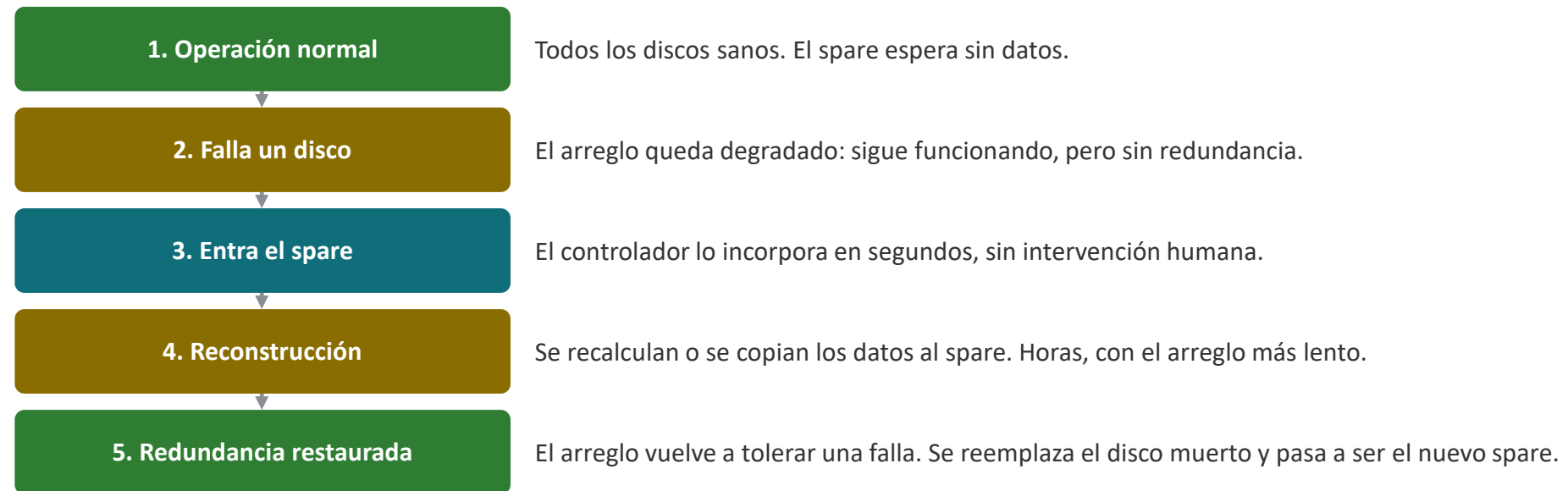


El «factor de compra» es la cifra que se lleva al presupuesto: **capacidad útil requerida × factor = capacidad bruta que hay que comprar. En RAID 10 se compra el doble.**

Arquitectura de Software

El disco de reserva (hot spare) y la reconstrucción

Un disco de reserva es un disco instalado, encendido y sin usar, que el controlador incorpora automáticamente cuando otro falla.



Sin spare, la ventana de riesgo dura desde que falla el disco hasta que alguien va físicamente al data center a cambiarlo: pueden ser días. Con spare, dura minutos. Un disco adicional es barato comparado con perder el arreglo.

Arquitectura de Software

Tres reglas de disco de un documento real, explicadas

Un documento de arquitectura real fija estas tres reglas de disco. Ninguna es arbitraria.

«La máquina física debe estar en RAID 1»

Se refiere a los discos desde donde arranca el servidor: sistema operativo e hipervisor. Con dos discos en espejo, si uno muere el equipo sigue arrancado y encendido. Es poco espacio, así que perder el 50% no importa; y no se justifica calcular paridad para eso.

«El storage debe ser 5 discos SSD iguales, como mínimo, en RAID 10, más spare»

Los datos y las máquinas virtuales escriben mucho: RAID 10 es el único nivel sin penalización de paridad. Iguales, porque en un arreglo todos los discos se usan al tamaño del más pequeño. SSD, por los IOPS que exige la base de datos. Y el spare, para que la reconstrucción empiece sola.

«Cada disco debe tener la mitad de la capacidad total»

Es la consecuencia directa del 50% de RAID 10: cuatro discos de $C/2$ en espejo y franjas dan exactamente C de capacidad útil. El quinto disco, del mismo tamaño, es el de reserva.

Arquitectura de Software

Por qué «la mitad de la capacidad total»

Supongamos que su solución necesita 8 TB de capacidad útil para datos y archivos.

Paso	Valor	Por qué
Capacidad útil requerida	8 TB	Sale del dimensionamiento: volumen inicial + crecimiento × horizonte + retención
Nivel elegido	RAID 10	Escritura intensiva de base de datos y máquinas virtuales
Capacidad útil de RAID 10	50%	La mitad de los discos guarda la copia espejo
Tamaño de cada disco	$8 \text{ TB} \div 2 = 4 \text{ TB}$	«La mitad de la capacidad total»: cada disco es de $C/2$
Discos del arreglo	4 discos de 4 TB	Dos espejos de 4 TB, en franjas: $4 + 4 = 8 \text{ TB}$ útiles
Disco de reserva	+1 disco de 4 TB	El «más spare» que pide la regla
Se compran	5 discos SSD de 4 TB	20 TB brutos para 8 TB útiles: factor de compra ×2,5



Ese factor ×2,5 es lo que hay que llevar al presupuesto. Cotizar 8 TB de discos cuando el diseño exige 20 TB brutos es el error de costeo más caro del dimensionamiento de almacenamiento.

Arquitectura de Software

Cabina compartida o discos en cada servidor

Con un clúster de tres servidores es posible no usar almacenamiento externo, pero entonces cada máquina debe tener la capacidad total: el espacio a comprar se triplica.

Con cabina de almacenamiento externa

- Un solo arreglo compartido por los tres servidores.
- 8 TB útiles → 5 discos de 4 TB → 20 TB brutos, una sola vez.
- Los tres equipos ven el mismo volumen: cualquier máquina virtual arranca en cualquiera.
- Costo adicional: la cabina, sus controladoras redundantes y la red de almacenamiento.
- Riesgo: la cabina pasa a ser el punto único de falla si no es redundante.

Con discos en cada servidor

- Cada equipo lleva su propio arreglo completo.
- 8 TB útiles × 3 equipos → 15 discos de 4 TB → 60 TB brutos.
- No hay cabina que comprar ni red de almacenamiento que instalar.
- La replicación entre servidores la hace el software, no el hardware.
- Factor de compra efectivo: ×7,5 sobre la capacidad útil.



Comparar estas dos alternativas con cifras —y no con adjetivos— es exactamente lo que se espera de la evaluación técnico-económica de su oferta.

Arquitectura de Software

Lista de verificación para su oferta

1

Inventario de nodos

¿Hay una tabla con nombre, función, sistema operativo, vCPU, RAM y disco de cada máquina?

2

Totales sumados

¿Están sumados los vCPU, la memoria y el disco, y esa suma se conecta con el hardware cotizado?

3

Cantidad justificada

¿Se explica por qué tres masters, cuántos workers y por qué esa cantidad?

4

Distribución justificada

¿Se dice qué máquina virtual va en qué servidor físico, y por qué esa repartición?

5

Hipervisor decidido

¿Se declara cuál se usará, con qué licenciamiento, y se reservó su consumo de recursos?

6

Nivel RAID por volumen

¿Se indica RAID 1 para el arranque y el nivel elegido para los datos, con su razón?

Arquitectura de Software

Lista de verificación para su oferta · continuación 1

7

Factor de compra aplicado

¿La capacidad bruta cotizada corresponde a la útil requerida multiplicada por el factor del nivel RAID?

8

Disco de reserva

¿Está considerado el spare y su costo, o se asume que alguien irá a cambiar el disco a tiempo?

9

Prueba de falla

¿Hay una tabla de «qué pasa si cae un servidor», con los puntos únicos de falla declarados?

10

RAID y respaldo separados

¿Queda claro que el RAID no es el respaldo, y hay una política de respaldo aparte?

Sección 3 · Arquitectura física y dimensionamiento

1 Modelo OSI aplicado

Repase las siete capas y ubique en cada una los equipos y servicios que aparecerán en su arquitectura física.

2 Dimensionamiento

Complete la cadena usuarios → concurrencia → transacciones por segundo → nodos para su caso, y súmela en un inventario de máquinas.

3 Quórum y consenso

Lea cómo funciona el algoritmo Raft y verifique por qué el número de nodos de control siempre es impar.

4 Virtualización

Instale Proxmox VE o VirtualBox y cree tres máquinas virtuales. Observe cuánta memoria consume el hipervisor por sí solo.

5 Contenedores y orquestación

Levante un Docker Swarm de tres nodos, apague uno y compruebe qué pasa con el quórum y con los contenedores.

6 RAID y almacenamiento

Calcule, para la capacidad útil de su caso, cuántos discos y de qué tamaño hay que comprar en RAID 1, RAID 5, RAID 6 y RAID 10.

04

SECCIÓN 4

La nube

- Las cinco características esenciales del cloud computing
- IaaS, PaaS y SaaS: quién administra qué, y todos los demás *aaS
- Modelos de despliegue, regiones, zonas de disponibilidad y redes virtuales
- Una aplicación de tres capas desplegada en la nube, componente por componente
- Serverless, orquestación de contenedores y la comparación honesta con on-premise
- Catálogo de servicios AWS y Azure, y arquitecturas físicas de referencia
- Máquina virtual, contenedores, Fargate y funciones: diferencias, costos y cuándo usar cada uno

Arquitectura de Software

Aplicaciones en la nube

El desarrollo de aplicaciones nativas de la nube es un enfoque que permite compilar, ejecutar y mejorar aplicaciones en función de técnicas y tecnologías reconocidas para el Cloud Computing. Estas arquitecturas están diseñadas para ofrecer escalabilidad, resiliencia y agilidad operativa.

Cinco características esenciales de un servicio de cloud computing:

1

Autoservicio bajo demanda

El cliente contrata sólo los servicios que requiere y cuando los necesita, sin mayor interacción con el prestador.

2

Amplio acceso a la red

Los servicios quedan disponibles ampliamente, acorde a las reglas de acceso que se definan.

3

Recursos compartidos

Los recursos del prestador se agrupan para servir a múltiples clientes, asignándose de forma dinámica.

4

Elasticidad

Las capacidades se asignan y se retiran de forma elástica, a menudo automática, respondiendo a la demanda.

5

Servicio medido

Se mide el consumo en un punto apropiado del servicio, permitiendo al cliente consumir sólo lo que necesita.

Arquitectura de Software

IaaS · PaaS · SaaS · ¿quién administra qué?

		On-premise	IaaS	PaaS	SaaS
Aplicaciones		usted	usted	usted	proveedor
Datos		usted	usted	usted	proveedor
Tiempo de ejecución		usted	usted	proveedor	proveedor
Middleware		usted	usted	proveedor	proveedor
Sistema operativo		usted	usted	proveedor	proveedor
Virtualización		usted	proveedor	proveedor	proveedor
Servidores		usted	proveedor	proveedor	proveedor
Almacenamiento		usted	proveedor	proveedor	proveedor
Red		usted	proveedor	proveedor	proveedor

usted

lo administra su equipo

proveedor

lo administra el proveedor de nube

Cada columna hacia la derecha reduce trabajo de operación y aumenta la dependencia del proveedor.

Arquitectura de Software

Las tres familias, en detalle

IaaS · Infraestructura como servicio

Pone a disposición del cliente el uso de la infraestructura informática —capacidad de cómputo, espacio de disco y bases de datos, entre otros— como un servicio.

En vez de adquirir servidores, espacio de data center o equipos de red, el cliente externaliza buscando ahorro en la inversión en sistemas TI. La factura se calcula según los recursos consumidos: pago por uso.

Ejemplo típico: máquinas virtuales, discos, redes virtuales.

PaaS · Plataforma como servicio

Es básicamente un ambiente de desarrollo donde se pueden crear otras aplicaciones que hagan uso de las características del Cloud Computing.

El modelo permite a los usuarios crear aplicaciones de software utilizando herramientas suministradas por el proveedor, sin administrar el sistema operativo ni el servidor de aplicaciones.

Ejemplo típico: plataformas de despliegue de aplicaciones, bases de datos gestionadas.

SaaS · Software como servicio

Consiste en la entrega de aplicaciones como servicio: el proveedor ofrece licencias de su aplicación a los clientes para su uso bajo demanda.

El proveedor puede tener la aplicación instalada en sus propios servidores web, permitiendo el acceso mediante navegador, o entregarla para su instalación, desactivándose al terminar el contrato.

Ejemplo típico: correo, gestión documental, CRM, ERP en línea.

Arquitectura de Software

Otros servicios en la nube

FaaS

Function as a Service: permite ejecutar pequeñas tareas sin necesidad de preocuparse por la infraestructura informática.

DBaaS

Database as a Service: modelo que permite gestionar bases de datos en la nube de forma escalable.

IDaaS

Identity as a Service: soluciones de autenticación y gestión de identidades para aplicaciones en la nube.

SECaaS

Security as a Service: soluciones de seguridad para prevenir amenazas, detectar intrusiones y recuperar sistemas vulnerados.

iPaaS

Integration Platform as a Service: permite integrar aplicaciones y simplificar el flujo de información entre datos de distintas fuentes.

MBaaS

Mobile as a Service: soluciones de infraestructura TI orientadas específicamente a aplicaciones móviles.

CaaS

Container as a Service: ejecución y orquestación de contenedores sin administrar los nodos que los soportan.

DaaS

Desktop as a Service: escritorios virtuales entregados como servicio, útiles cuando el mandante tiene teletrabajo o terceros.

Arquitectura de Software

Cuándo conviene contratar cada servicio

Cada sigla resuelve un problema concreto. La pregunta correcta no es «¿qué es?», sino «¿en qué situación de mi caso me sirve?».

Servicio	Situación concreta en la que conviene contratarlo	Qué se evita construir
IaaS	El mandante exige un software heredado que se instala en el servidor, o usted necesita control total del sistema operativo.	Comprar, instalar y renovar hardware
PaaS	Hay que publicar un portal web y el equipo no tiene a nadie que administre servidores.	Administrar el servidor de aplicaciones
SaaS	El caso pide correo, firma de documentos o gestión documental: nada de eso hay que construirlo.	Desarrollar y operar esa funcionalidad
FaaS	Cada vez que llega un archivo hay que procesarlo. Ocurre pocas veces al día y no justifica un servidor encendido.	Un servidor esperando el archivo
DBaaS	Se necesita una base de datos con respaldo, réplica y parches al día, y no hay un administrador de bases de datos en el equipo.	Instalar y operar el motor
IDaaS	Los usuarios deben entrar con una identidad ya existente, o se exige doble factor sin construirlo.	El módulo de autenticación completo
SECaaS	Se necesita protección contra ataques y monitoreo de seguridad sin montar un equipo dedicado.	Un centro de operaciones de seguridad

Arquitectura de Software

Cuándo conviene contratar cada servicio · continuación 1

Servicio	Situación concreta en la que conviene contratarlo	Qué se evita construir
iPaaS	Hay que integrar cinco sistemas del mandante con formatos distintos y poco tiempo de desarrollo.	Escribir cada integración a mano
CaaS	Se quiere desplegar contenedores sin administrar los servidores que los ejecutan.	Instalar y operar el clúster
DaaS	Personal externo o en teletrabajo debe acceder a aplicaciones internas desde equipos que no controla la organización.	Entregar y administrar equipos

Arquitectura de Software

Cómo se decide entre construir y contratar

Contratar un servicio no siempre es más barato, pero casi siempre es más rápido. La decisión se toma con cuatro criterios.

¿Es parte del valor del negocio?

Si la funcionalidad es lo que distingue su propuesta, constrúyala. Si es un servicio de apoyo —correo, firma, mapas— contrátela.

¿Tiene el equipo la capacidad?

Operar un motor de base de datos, un clúster o un sistema de identidad exige perfiles que quizás no están en el proyecto.

¿Cuánto cuesta a cinco años?

Compare el gasto mensual del servicio contra las horas de construcción, más las de operación durante todo el horizonte.

¿Qué dependencia genera?

Contratar traslada el riesgo operativo, pero crea dependencia de un tercero: hay que declararla y tener plan alternativo.



En la oferta, cada servicio contratado se declara con su función, su costo mensual y su acuerdo de nivel de servicio. Un servicio de tercero sin SLA declarado es un riesgo sin evaluar.

Arquitectura de Software

Modelos de despliegue

Nube pública

Infraestructura de un proveedor, compartida entre muchos clientes.

- + Sin inversión inicial, elasticidad inmediata, catálogo enorme.
- Menor control, costo variable, dependencia del proveedor.

Nube privada

Infraestructura dedicada a una sola organización, propia o alojada.

- + Control total, cumplimiento más simple de acreditar.
- Inversión alta, elasticidad limitada, requiere equipo propio.

Nube híbrida

Combina ambas: lo sensible o estable queda dentro, lo elástico va afuera.

- + Flexibilidad y aprovechamiento de la inversión existente.
- Complejidad de red, identidades y operación duplicada.

Multinube

Más de un proveedor público, por resiliencia o por evitar dependencia.

- + Reduce el riesgo de proveedor único.
- Duplica el aprendizaje, los contratos y las herramientas.



En el informe declare el modelo elegido y las razones. «Nube híbrida: la base de datos con información personal permanece en el data center del mandante; el portal público se despliega en nube pública con autoescalado» es una decisión defendible.

Arquitectura de Software

La geografía de la nube

Región

Ubicación geográfica donde el proveedor agrupa sus centros de datos.

Determina la latencia de acceso y el cumplimiento de las regulaciones de residencia de datos.

Decisión de la oferta: en qué país quedan los datos.

Zona de disponibilidad

Centros de datos físicamente separados dentro de una misma región, con energía y red independientes.

Proporcionan alta disponibilidad y tolerancia a fallos al permitir replicar recursos en ubicaciones aisladas.

Decisión de la oferta: en cuántas zonas se despliega.

Punto de presencia / borde

Nodos distribuidos cerca del usuario final, usados por las redes de distribución de contenido.

Reducen la latencia percibida entregando contenido estático desde el punto más cercano.

Decisión de la oferta: si se usa CDN y para qué contenido.

Una aplicación desplegada en una sola zona de disponibilidad no tiene alta disponibilidad, aunque esté en la nube. La redundancia hay que diseñarla y hay que pagarla: no viene incluida por el hecho de contratar un proveedor.

Arquitectura de Software

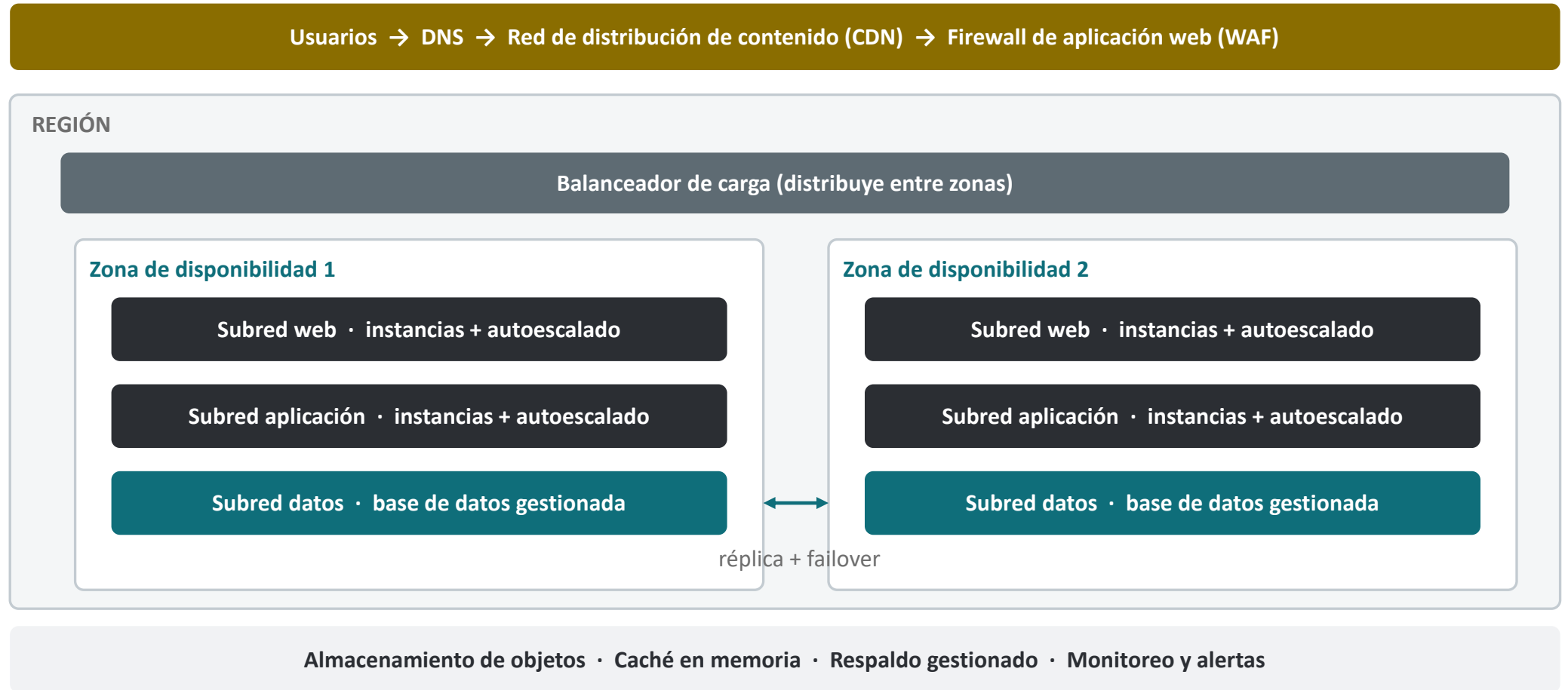
La red virtual

Los mismos principios de segmentación de la arquitectura on-premise se aplican en la nube, con otros nombres.

Elemento en la nube	Función	Equivalente on-premise
Red virtual privada (VPC)	Red virtual aislada donde se despliegan los recursos. Da control sobre direcciones IP, subredes y rutas.	La red interna de la organización
Subred pública	Aloja los recursos con acceso a Internet: pasarelas de salida y balanceadores.	La DMZ
Subred privada	Aloja los servidores de aplicación y las bases de datos. Sin acceso directo desde Internet.	Red interna y zona de datos
Pasarela NAT	Permite que los recursos en subred privada salgan a Internet para actualizarse, sin ser alcanzables desde fuera.	Proxy de salida
Grupo de seguridad	Reglas de tráfico permitido a nivel de instancia: puertos, protocolos y orígenes.	Firewall de host
Firewall de aplicación web	Protege contra vulnerabilidades comunes de la capa de aplicación, como inyección SQL y XSS.	WAF perimetral

Arquitectura de Software

Ejemplo · aplicación de tres capas en la nube



Arquitectura de Software

Los componentes del ejemplo

Componente	Función	Por qué está en la arquitectura
Protección DDoS	Protege de ataques de denegación de servicio.	Primera defensa de la disponibilidad.
DNS gestionado	Dirige el tráfico a distintos destinos según el dominio solicitado.	Resolución de nombres y enrutamiento por dominio.
Firewall de aplicación web	Filtra tráfico malicioso antes de que llegue a los servidores.	Protege contra inyección SQL, XSS y el OWASP Top 10.
Red de distribución de contenido	Caché global de contenido estático cerca del usuario.	Baja latencia y menos carga en los servidores de origen.
Balanceador de carga	Distribuye el tráfico entrante entre múltiples instancias.	Disponibilidad, comprobaciones de estado y terminación TLS.
Grupo de autoescalado	Ajusta la cantidad de instancias según la demanda.	Rendimiento en el máximo y ahorro en las horas de baja demanda.
Base de datos gestionada	Motor relacional administrado, con réplica en otra zona.	Respaldo automático, parches y conmutación ante fallas.
Caché en memoria	Almacena en memoria los datos más consultados.	Baja la carga de la base y la latencia.
Almacenamiento de objetos	Archivos estáticos, documentos y respaldos.	Escala sin límite y bajo costo por GB.
Monitoreo y alertas	Métricas, registros y alarmas de los recursos.	Sin esto no se puede cumplir un SLA.

Arquitectura de Software

Arquitectura sin servidores · serverless

La informática serverless está totalmente gestionada: nunca se reservan explícitamente instancias de servidor. Cada ejecución de una función podría correr en una instancia de cómputo diferente, de forma transparente para el código.

De qué nos podemos olvidar

- Aprovisionar servidores.
- Mantenerlos y gestionarlos.
- Escalar la aplicación ante aumentos de demanda.
- Preocuparnos de la disponibilidad y la tolerancia a fallos.

Cómo funciona

- En lugar de programar una aplicación completa, se escribe una función: código más metadatos (sus disparadores y enlaces con otros sistemas).
- La plataforma programa la ejecución y escala el número de instancias según la tasa de eventos entrantes.
- Encaja muy bien con cargas de trabajo que responden a eventos entrantes.

Modelo de cobro: se paga únicamente cuando se está ejecutando el código. Si no hay ejecuciones activas, no se cobra. Si el código corre una vez al día durante 2 minutos, se factura 1 ejecución y 2 minutos de cómputo.

Arquitectura de Software

Serverless · cuándo conviene y cuándo no

Conviene cuando...

- La carga es intermitente o muy variable: se paga sólo el uso real.
- El proceso es corto y se dispara por un evento (un archivo que llega, un mensaje en cola, una hora del día).
- Se quiere partir sin costo fijo de infraestructura.
- Son tareas de apoyo: notificaciones, transformación de archivos, integraciones puntuales.

No conviene cuando...

- Hay carga alta y constante: a partir de cierto volumen sale más caro que instancias reservadas.
- El proceso es largo: las plataformas imponen un tiempo máximo de ejecución.
- La latencia de la primera invocación importa: el arranque en frío puede agregar cientos de milisegundos.
- Se necesita portabilidad: es el modelo con mayor dependencia del proveedor.

Todos los grandes operadores ofrecen funciones sin servidor. Lo relevante para la oferta no es la marca, sino declarar qué componentes serán serverless y por qué.

Arquitectura de Software

Administrar contenedores · por qué

Un contenedor resuelve el empaquetado de una aplicación. El problema aparece cuando hay muchos, en varios servidores, cambiando todo el tiempo.

Lo que hay que resolver a mano

- ¿En qué servidor levanto cada contenedor y con qué recursos?
- Si el contenedor muere de madrugada, ¿quién lo levanta?
- Si el servidor completo se cae, ¿a dónde se mueve su carga?
- ¿Cómo encuentra un servicio la dirección de otro si cambia en cada arranque?
- ¿Cómo publico una versión nueva sin cortar el servicio?
- ¿Cómo agrego capacidad en el peak y la retiro después?

Lo que resuelve la orquestación

- Programación automática según recursos disponibles.
- Reinicio automático del contenedor y reubicación si cae el nodo.
- Nombres estables de servicio y balanceo interno.
- Despliegue progresivo con reversión automática.
- Escalado por métricas, hacia arriba y hacia abajo.
- Configuración y secretos separados de la imagen.

En una palabra: la orquestación convierte un conjunto de servidores en una sola capacidad de cómputo donde uno declara «quiero tres réplicas de este servicio» y el sistema se encarga del resto.

Arquitectura de Software

Orquestación de contenedores

Un contenedor aislado resuelve el empaquetado. Cuando hay decenas de contenedores en varios servidores, aparece el problema de coordinarlos: eso es la orquestación.

Programación

Decide en qué nodo se ejecuta cada contenedor según los recursos disponibles.

Autorreparación

Si un contenedor muere, lo levanta de nuevo. Si un nodo cae, redistribuye la carga.

Escalado

Aumenta o reduce las réplicas según métricas de uso, de forma automática.

Descubrimiento

Cada servicio tiene un nombre estable; no hay que conocer direcciones IP.

Despliegue progresivo

Publica una versión nueva de forma gradual y revierte si algo falla.

Configuración y secretos

Separa la configuración y las credenciales de la imagen del contenedor.

Para la oferta: proponer orquestación exige declarar quién opera el clúster. Un servicio gestionado por el proveedor traslada esa carga; un clúster propio agrega perfiles y horas al presupuesto.

Arquitectura de Software

Ciclo de vida y seguridad de las imágenes

Administrar contenedores no es sólo ejecutarlos: es gobernar el ciclo de vida de las imágenes que se ejecutan.

Registro de imágenes

Repositorio privado donde se publican las imágenes construidas. Es el equivalente al almacén de artefactos: sin él no hay trazabilidad de qué está corriendo.

Versionado

Cada imagen se etiqueta con una versión inmutable. Nunca desplegar la etiqueta «latest» en producción: no se sabe qué se está ejecutando.

Imagen base mínima

Mientras menos trae la imagen, menos vulnerabilidades tiene y más rápido arranca. Nada de herramientas de depuración en producción.

Análisis de vulnerabilidades

Escaneo automático de la imagen en cada construcción y de forma periódica: una imagen segura hoy no lo es en tres meses.

Sin secretos adentro

Contraseñas y llaves se inyectan en tiempo de ejecución desde el almacén de secretos, nunca se hornean en la imagen.

Límites de recursos

Cada contenedor declara CPU y memoria máxima. Sin límites, un contenedor con fuga de memoria arrastra a todo el nodo.

Para la oferta: comprometer «imágenes versionadas, escaneadas en cada construcción y con límites de recursos declarados» es concreto, verificable y cuesta muy poco ofrecerlo.

Arquitectura de Software

¿Necesita usted un orquestador?

Hay una escalera de opciones. Suba sólo hasta donde el caso lo justifique: cada peldaño agrega capacidad y agrega costo de operación.

		QUÉ ES	CUÁNDO SE JUSTIFICA
1	Sin contenedores	Aplicación desplegada directamente en el servidor o servicio gestionado.	1-2 componentes, despliegues poco frecuentes, equipo pequeño.
2	Contenedores gestionados	El proveedor ejecuta el contenedor: usted entrega la imagen y declara réplicas.	Hasta ~10 servicios. Cubre la mayoría de los proyectos de este curso.
3	Orquestador gestionado	Clúster administrado por el proveedor; usted opera sólo las cargas.	Muchos servicios, despliegues frecuentes, necesidad de control fino.
4	Orquestador propio	Clúster instalado y operado por su equipo, on-premise o en máquinas virtuales.	Exigencia de datos en sitio o requisitos que ningún servicio gestionado cubre.

Proponer el peldaño 4 en un proyecto de cuatro meses con cinco personas es, casi siempre, un riesgo mal evaluado.

Arquitectura de Software

On-premise vs. nube · comparación honesta

Criterio	On-premise	Nube pública
Inversión inicial	Alta: hardware, licencias, habilitación	Baja o nula
Costo mensual	Predecible y relativamente fijo	Variable: depende del consumo
Tiempo de puesta en marcha	Semanas o meses (compra, envío, instalación)	Minutos u horas
Elasticidad ante picos	Se dimensiona para el máximo y se desperdicia el resto del año	Se ajusta automáticamente
Control y personalización	Total	Limitado a lo que ofrece el proveedor
Residencia de los datos	Donde el mandante decida	Donde existan regiones disponibles
Equipo requerido	Operaciones, redes, respaldo, seguridad	Menos operación, más arquitectura y control de gasto
Dependencia del proveedor	Baja	Alta si se usan servicios propietarios
Renovación tecnológica	Reinversión cada 3 a 5 años	Incluida en el servicio
Continuidad ante desastre	Exige un segundo sitio, con su costo	Replicación entre zonas y regiones

Arquitectura de Software

CAPEX, OPEX y el costo total

La decisión no se juega en el precio de una máquina: se juega en la forma del flujo de caja a lo largo del horizonte de evaluación.

On-premise · perfil CAPEX

- Desembolso grande en el año 0 y reinversión al final de la vida útil.
- Se deprecia: genera escudo tributario, hay que reflejarlo en el flujo.
- Costos anuales relativamente estables y predecibles.
- Capacidad ociosa comprada por adelantado: se paga el máximo todo el año.
- Riesgo: quedar corto obliga a una compra no presupuestada.

Nube · perfil OPEX

- Sin desembolso inicial relevante: la inversión se traslada a gasto mensual.
- No se deprecia: es gasto del ejercicio, con otro efecto tributario.
- Costo variable con el uso: puede crecer más rápido que los ingresos.
- Se paga sólo la capacidad utilizada, si se configura bien el autoescalado.
- Riesgo: gasto que se dispara sin control y sin nadie que lo mire.



Compare siempre el costo total sobre el mismo horizonte que usará en la evaluación económica —típicamente 5 años— e incluya reinversión, licencias, personal y crecimiento de la demanda en ambos escenarios.

Arquitectura de Software

Los costos de la nube que se olvidan

Salida de datos

Subir datos suele ser gratis; sacarlos, no. En soluciones con mucho contenido o con respaldo hacia afuera, este ítem sorprende.

Ambientes no productivos

Desarrollo, pruebas y capacitación también consumen. Suelen sumar entre 30% y 50% del ambiente productivo si no se apagan.

Licencias sobre la nube

El motor de base de datos o el sistema operativo pueden facturarse aparte del cómputo.

Soporte del proveedor

El plan de soporte con tiempos de respuesta comprometidos es un porcentaje del gasto mensual.

Sobredimensionamiento

Instancias grandes «por si acaso» y recursos que quedaron encendidos sin uso. El desperdicio típico es alto.

Respaldo y retención

Guardar copias por años tiene costo acumulativo: el almacenamiento crece todos los meses y nunca baja solo.

Observabilidad

Registros y métricas se cobran por volumen ingerido y por tiempo de retención. Se dispara con facilidad.

Transferencia entre zonas

El tráfico entre zonas de disponibilidad también se factura: la alta disponibilidad tiene costo de red.

Arquitectura de Software

Del concepto al nombre del servicio

En el diagrama lógico se nombra la función. En el diagrama físico y en la cotización aparece el producto. Estas cuatro láminas son el diccionario entre ambos.

Primero la función

«Necesito distribuir el tráfico entre varias instancias y sacar de rotación las que no responden.» Eso es lo que va en la arquitectura lógica.

Después el producto

«Application Load Balancer» en AWS, «Application Gateway» en Azure. Eso es lo que va en el diagrama físico y en la cotización.

Y siempre el porqué

«Se elige capa 7 porque el enrutamiento por ruta y el despliegue progresivo son requisitos del caso.» Eso es lo que da puntaje.

Advertencia importante: los nombres comerciales cambian, se renombran y se discontinúan. En el informe, escriba siempre la función primero y el producto como ejemplo —«balanceador de carga de capa 7 (por ejemplo, AWS Application Load Balancer)»—. Así la propuesta sigue siendo válida aunque el proveedor cambie el nombre, y no queda amarrada a un solo fabricante.

Arquitectura de Software

Catálogo 1 · red, entrega y perímetro

Función	AWS	Azure	Equivalente on-premise
DNS y enrutamiento por dominio	Route 53	Azure DNS + Traffic Manager	Servidor DNS propio (BIND, Windows DNS)
Red virtual aislada	VPC (Virtual Private Cloud)	Virtual Network (VNet)	La red interna y sus VLAN
Subred pública	Public subnet + Internet Gateway	Subnet con IP pública	La DMZ
Subred privada	Private subnet	Subnet privada	La red interna de aplicaciones y datos
Salida a Internet desde red privada	NAT Gateway	Azure NAT Gateway	Proxy de salida
Red de distribución de contenido	CloudFront	Azure Front Door	Caché inverso propio (Varnish, Nginx)
Balanceador de capa 7 (HTTP)	Application Load Balancer (ALB)	Application Gateway	Proxy inverso / balanceador de aplicación
Balanceador de capa 4 (TCP/UDP)	Network Load Balancer (NLB)	Azure Load Balancer	Balanceador de red (F5, HAProxy)
Firewall de aplicación web	AWS WAF	Azure Web Application Firewall	WAF perimetral en appliance
Protección contra denegación de servicio	AWS Shield	Azure DDoS Protection	Servicio de mitigación del proveedor de enlace
Reglas de tráfico por instancia	Security Groups	Network Security Groups (NSG)	Reglas de firewall de host
Firewall de red gestionado	AWS Network Firewall	Azure Firewall	Firewall perimetral (appliance)

Arquitectura de Software

Catálogo 2 · cómputo y contenedores

Función	AWS	Azure	Comentario para la oferta
Máquina virtual	EC2	Azure Virtual Machines	Control total; usted administra el sistema operativo y su ciclo de parches.
Grupo de escalado automático	Auto Scaling Group	Virtual Machine Scale Sets	Es lo que convierte un servidor en una capa elástica.
Orquestador de contenedores	EKS (Elastic Kubernetes Service)	AKS (Azure Kubernetes Service)	Estándar de facto; el plano de control lo opera el proveedor.
Orquestador propio del proveedor	ECS (Elastic Container Service)	—	Más simple que Kubernetes y suficiente para muchas soluciones.
Contenedores sin administrar servidores	AWS Fargate	Azure Container Apps	Se despliega el contenedor y no existe un nodo que mantener.
Contenedor suelto por tarea	ECS Task / Fargate task	Azure Container Instances (ACI)	Ideal para procesos puntuales o tareas programadas.
Aplicación web gestionada (PaaS)	AWS App Runner / Elastic Beanstalk	Azure App Service	El camino más corto para publicar una aplicación web.
Funciones sin servidor	AWS Lambda	Azure Functions	Se ejecuta por evento; se paga por invocación y por tiempo.
Registro de imágenes	ECR (Elastic Container Registry)	Azure Container Registry	Imprescindible si usa contenedores: es donde vive el artefacto.
Procesamiento por lotes	AWS Batch	Azure Batch	Para cargas masivas programadas, no para atención en línea.

Arquitectura de Software

Catálogo 3 · datos, almacenamiento y observabilidad

Función	AWS	Azure	Equivalente on-premise
Base de datos relacional gestionada	RDS (PostgreSQL, MySQL, SQL Server)	Azure Database for PostgreSQL / SQL Database	Motor instalado en un servidor propio
Base de datos documental / NoSQL	DynamoDB / DocumentDB	Cosmos DB	MongoDB autogestionado
Caché en memoria	ElastiCache (Redis)	Azure Cache for Redis	Redis o Memcached instalado
Almacenamiento de objetos	S3 (bucket)	Azure Blob Storage (contenedor)	Servidor de archivos o almacenamiento de objetos propio
Almacenamiento de bloque	EBS (Elastic Block Store)	Azure Managed Disks	Discos de la SAN
Sistema de archivos compartido	EFS (Elastic File System)	Azure Files	NAS con NFS o SMB
Respaldo gestionado	AWS Backup	Azure Backup	Software de respaldo y librería de cintas
Métricas, registros y alarmas	CloudWatch	Azure Monitor + Log Analytics	Zabbix, Nagios, ELK autogestionado
Trazas distribuidas	AWS X-Ray	Application Insights	Instrumentación propia
Mensajería / colas	SQS y SNS	Azure Service Bus / Event Grid	RabbitMQ o ActiveMQ instalado
Flujo de eventos	Kinesis / MSK	Azure Event Hubs	Kafka autogestionado

Arquitectura de Software

Catálogo 4 · conectividad híbrida, identidad y secretos

Función	AWS	Azure	Alternativa abierta o propia
Túnel permanente entre sitios	Site-to-Site VPN	VPN Gateway (site-to-site)	IPsec en appliance propio, o WireGuard entre extremos
Enlace dedicado con el data center	Direct Connect	ExpressRoute	Enlace punto a punto contratado al proveedor de red
Acceso remoto de personas	AWS Client VPN	VPN Gateway (point-to-site)	WireGuard, OpenVPN, o acceso Zero Trust de un tercero
Acceso administrativo sin exponer puertos	Systems Manager Session Manager	Azure Bastion	Servidor bastión propio con doble factor
Conexión privada a servicios gestionados	PrivateLink / VPC Endpoints	Azure Private Link	No aplica: es propio de la nube
Interconexión de redes virtuales	VPC Peering / Transit Gateway	VNet Peering / Virtual WAN	Enrutamiento entre VLAN
Identidad y permisos de la plataforma	IAM	Microsoft Entra ID + RBAC	Directorio corporativo (LDAP, Active Directory)
Identidad de los usuarios de la aplicación	Cognito	Microsoft Entra External ID	Proveedor de identidad propio o de un tercero
Almacén de secretos	Secrets Manager / Parameter Store	Azure Key Vault	HashiCorp Vault autogestionado
Certificados TLS	AWS Certificate Manager	Azure Key Vault (certificados)	Let's Encrypt con renovación automatizada

WireGuard es un protocolo de VPN de código abierto, muy liviano y rápido, integrado al núcleo de Linux. Es una opción legítima y de bajo costo para unir el data center con la nube cuando no se justifica un enlace dedicado.

Arquitectura de Software

Antes de comparar · las cinco palabras

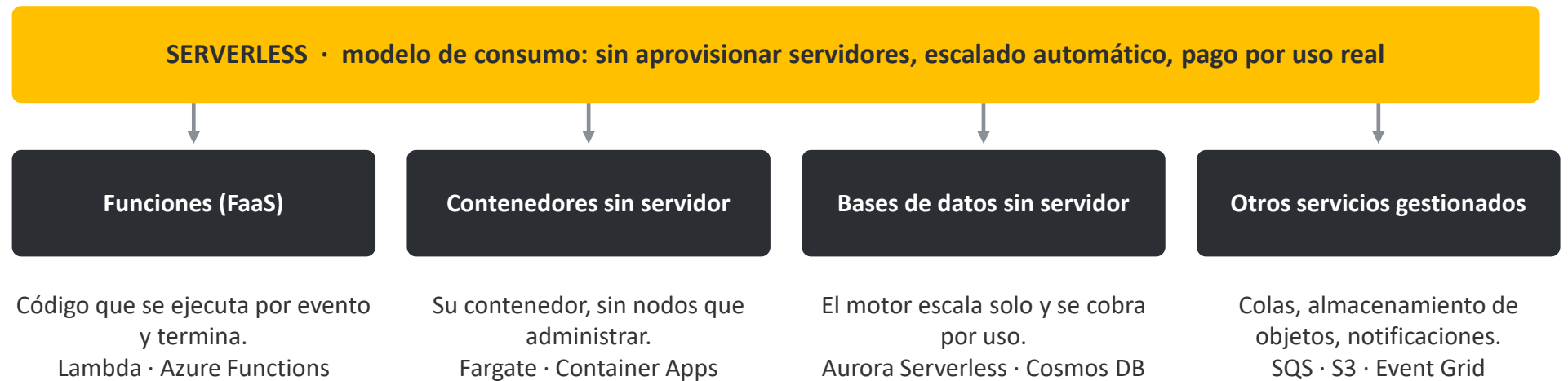
Cinco términos que se usan como sinónimos y no lo son. Fijarlos ahora evita la confusión en todo lo que sigue.

Término	Qué es exactamente	Ejemplo
Máquina virtual	Un computador simulado por software, con su propio sistema operativo completo. Usted lo enciende, lo configura, lo actualiza y lo apaga. Es un servidor, sólo que no es de metal.	EC2 · Azure Virtual Machines
Contenedor	Un paquete con la aplicación y sus dependencias, que se ejecuta aislado sobre un sistema operativo compartido. No trae sistema operativo propio: por eso pesa e inicia mucho menos que una máquina virtual.	Una imagen de contenedor
Contenedores gestionados	Un orquestador operado por el proveedor que coordina muchos contenedores. Ojo: los servidores donde corren esos contenedores —los nodos— siguen siendo suyos y hay que dimensionarlos y parcharlos.	EKS · AKS
Contenedores sin servidor	El mismo contenedor, pero sin nodos: usted declara cuánta CPU y memoria necesita cada tarea y el proveedor se encarga de dónde se ejecuta. No hay servidor que administrar.	AWS Fargate · Azure Container Apps
Serverless	No es un producto: es un modelo de consumo. Significa que no se aprovisiona ni se administra ningún servidor, que la plataforma escala sola y que se paga sólo por lo que se usa.	Un modelo, no un servicio

Arquitectura de Software

¿«Funciones» es lo mismo que «serverless»?

No. Serverless es el paraguas —un modelo de consumo—; las funciones son sólo una de las formas que toma ese modelo, y no la única.



Entonces: toda función es serverless, pero no todo lo serverless son funciones. Fargate también es serverless —no hay servidor que administrar— y sin embargo ejecuta contenedores, no funciones.

Y la contraparte: los contenedores gestionados (EKS, AKS) NO son serverless, porque los nodos siguen siendo responsabilidad suya, aunque el plano de control lo opere el proveedor.

Arquitectura de Software

Cuatro modelos de cómputo · quién administra qué

		Máquina virtual (EC2 · Azure VM)	Contenedores gestionados (EKS · AKS)	Contenedores sin servidor (Fargate · Container Apps)	Funciones (Lambda · Azure Functions)
Nube	Código de la aplicación	usted	usted	usted	usted
Calidad	Imagen o paquete de despliegue	usted	usted	usted	compartido
Continuidad	Escalado	usted	compartido	proveedor	proveedor
Seguridad	Runtime y servidor de aplicación	usted	usted	usted	proveedor
Identidad	Sistema operativo y parches	usted	usted	proveedor	proveedor
	Nodos y clúster	usted	compartido	proveedor	proveedor
	Hardware, red y virtualización	proveedor	proveedor	proveedor	proveedor
Entrega		usted	compartido	proveedor	

Hacia la derecha desaparece trabajo de operación —y con él, horas del presupuesto—; a cambio aumenta la dependencia de la plataforma y bajan las opciones de configuración fina.

Arquitectura de Software

Los cuatro modelos, comparados

	Máquina virtual	Contenedores gestionados	Contenedores sin servidor	Funciones serverless
Qué entrega usted	Un servidor que administra	Imágenes + un clúster con nodos	Sólo la imagen y su tamaño	Sólo el código de la función
Quién parchea el SO	Usted	Usted (los nodos)	El proveedor	El proveedor
Tiempo de arranque	Minutos	Segundos con nodo libre	Decenas de segundos	Milisegundos
Granularidad de escalado	Por instancia completa	Por réplica	Por tarea	Por invocación
Límite de duración	Ninguno	Ninguno	Ninguno	15 min en Lambda
Procesos de larga duración	Sí	Sí	Sí	No
Costo con carga constante	El más bajo (con reserva)	Bajo si el clúster está bien lleno	Medio-alto	Alto a partir de cierto volumen
Costo con carga intermitente	Alto: se paga apagado o encendido	Alto: el nodo sigue encendido	Bajo	El más bajo; cero si no hay uso
Complejidad de operación	Alta	La más alta	Media	Baja
Dependencia del proveedor	Baja	Baja: Kubernetes es portable	Media	Alta

Arquitectura de Software

Máquina virtual vs. hosting de contenedores

MÁQUINA VIRTUAL · EC2 / Azure VM

- + Control total: sistema operativo, versiones, agentes, configuración de red.
- + Es lo único viable para software heredado o que exige instalación en el servidor.
- + Con instancias reservadas es el costo por hora más bajo del mercado.
- + Modelo mental conocido: se parece a un servidor físico.
- – Usted parchea, actualiza y endurece el sistema operativo: son horas de operación todos los meses.
- – Escala por instancia completa: se paga capacidad ociosa.
- – Arranque en minutos: no responde bien a picos súbitos.
- – «Servidores mascota»: cada uno termina con su propia configuración irreplicable.

CONTENEDORES GESTIONADOS · EKS / AKS

- + Portabilidad real: la misma imagen corre en cualquier nube y on-premise.
- + Densidad: varios servicios comparten un nodo, se aprovecha mejor el hardware.
- + Autorreparación, despliegue progresivo y escalado por réplica, incorporados.
- + El plano de control lo opera el proveedor.
- – Los nodos siguen siendo suyos: hay que dimensionarlos, parcharlos y actualizarlos.
- – Curva de aprendizaje alta; exige un equipo que sepa operarlo.
- – Costo fijo del clúster aunque esté poco usado.
- – Es la opción con mayor complejidad operativa de las cuatro.

Regla: el orquestador se justifica por la cantidad de servicios y la frecuencia de despliegue, no por la cantidad de usuarios.

Arquitectura de Software

Fargate vs. funciones serverless

CONTENEDORES SIN SERVIDOR · Fargate / Container Apps

- + Desaparecen los nodos: no hay servidor que dimensionar, parchear ni escalar.
- + Se sigue desplegando una imagen de contenedor: no hay que reescribir la aplicación.
- + Sin límite de tiempo de ejecución: sirve para procesos largos y para servicios permanentes.
- + Aislamiento por tarea: cada tarea tiene su propio entorno.
- – Costo por vCPU y por GB de memoria mientras la tarea está viva: con carga constante sale más caro que una instancia reservada.
- – El arranque de una tarea toma decenas de segundos: no absorbe un pico instantáneo.
- – Menos control fino sobre el nodo (versión del núcleo, agentes, discos locales).

FUNCIONES · Lambda / Azure Functions

- + Se paga sólo por invocación y por tiempo de ejecución: si no hay uso, no hay costo.
- + Escala de cero a miles de ejecuciones sin configurar nada.
- + Cero operación: no hay servidor, ni nodo, ni contenedor que mantener.
- + Ideal para tareas por evento: un archivo que llega, un mensaje en cola, una hora programada.
- – Límite de duración por invocación (15 minutos en Lambda) y de memoria.
- – Arranque en frío: la primera invocación agrega latencia.
- – Obliga a diseñar por eventos y sin estado: no todo software se puede llevar ahí.
- – Es el modelo con mayor dependencia del proveedor.

Fargate es el punto medio: elimina la administración de servidores sin obligar a rediseñar la aplicación por eventos.

Arquitectura de Software

Cómo se cobra cada modelo

La unidad de cobro es lo que determina si el costo de su solución es fijo, semifijo o proporcional al uso.

Modelo	Unidad de cobro	Mínimo facturable	Se paga cuando está ocioso	Forma del costo
Máquina virtual	Por hora de instancia encendida (o descuento por reserva de 1 a 3 años)	Por segundo, con mínimo de 1 minuto	Sí, mientras esté encendida	Prácticamente fijo
Contenedores gestionados	Por hora de los nodos + tarifa fija del plano de control del clúster	La del nodo subyacente	Sí: los nodos siguen encendidos	Fijo por nodo
Contenedores sin servidor	Por vCPU-hora y por GB-hora mientras la tarea está en ejecución	Por segundo, con mínimo de 1 minuto por tarea	No, si la tarea se detiene	Según tiempo activo
Funciones	Por invocación + por GB-segundo de ejecución	Del orden del milisegundo	No: sin uso, costo cero	Según el uso real

Consecuencia para la evaluación económica: con carga estable, la máquina virtual reservada suele ganar; con carga intermitente o estacional, las funciones ganan por lejos; y en el medio, los contenedores sin servidor. Modele los tres escenarios antes de decidir.

Arquitectura de Software

Cómo elegir el modelo de cómputo

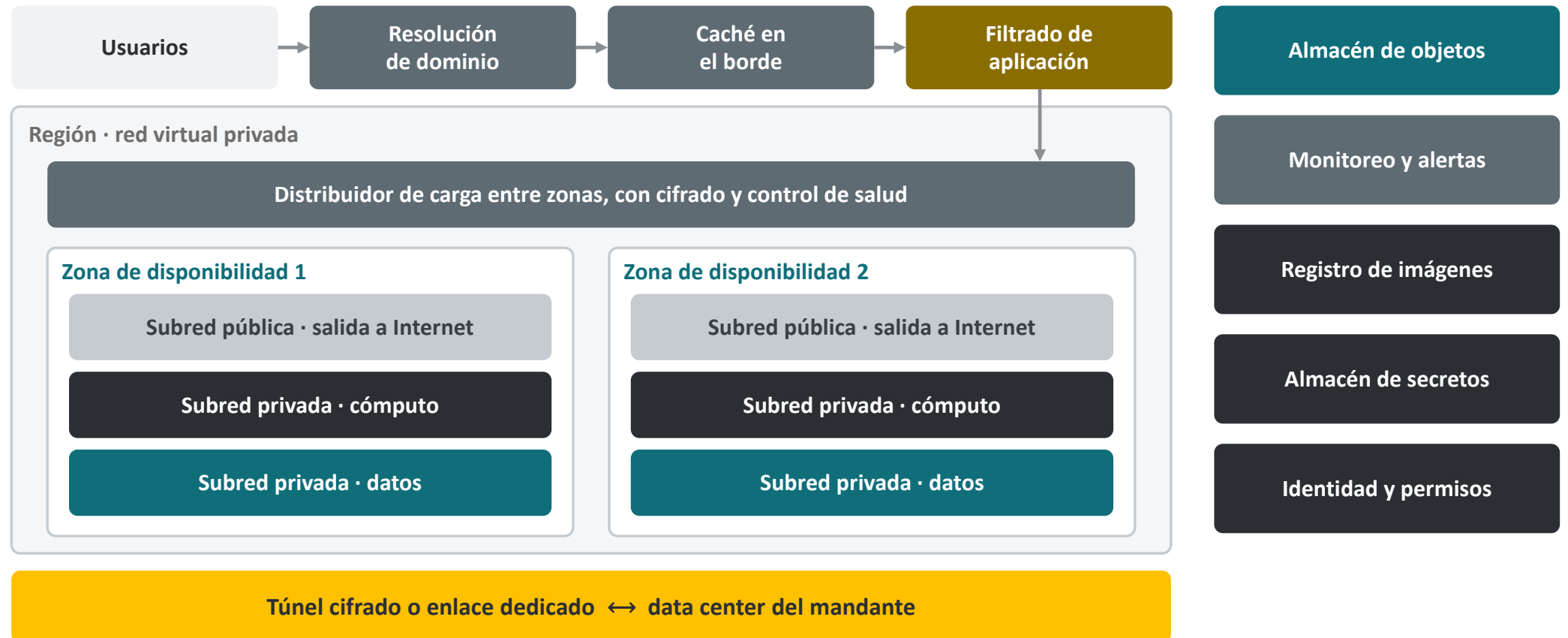
Cinco preguntas, en este orden. La primera que responda «sí» determina el modelo.



Arquitectura de Software

Arquitectura de referencia · primero los conceptos

Antes de poner nombres de productos, hay que saber qué elementos se necesitan y para qué.



Arquitectura de Software

Qué hace cada elemento y por qué está

Elemento conceptual	Para qué está	Qué pasa si no está
Resolución de dominio	Traduce el nombre del sitio a una dirección y puede desviar el tráfico a otro sitio si el principal falla.	No hay forma de conmutar rápido ante una caída
Caché en el borde	Entrega el contenido estático desde el punto más cercano al usuario.	Toda la carga llega al origen y la latencia sube
Filtrado de aplicación	Bloquea ataques conocidos antes de que lleguen al sistema.	El portal queda expuesto a ataques automatizados
Red virtual privada	Aísla los recursos y permite separarlos en subredes con reglas distintas.	Todo queda en la misma red: no hay segmentación
Subred pública	Aloja lo que necesita hablar con Internet: salida a la red y punto de entrada.	Los servidores quedarían expuestos directamente
Subred privada de cómputo	Aloja la aplicación, sin dirección pública.	La aplicación sería alcanzable desde Internet
Subred privada de datos	Aloja la base de datos, que sólo acepta a la capa de aplicación.	La base de datos quedaría expuesta
Distribuidor de carga	Reparte el tráfico entre zonas y saca de rotación lo que no responde.	Una instancia caída seguiría recibiendo usuarios

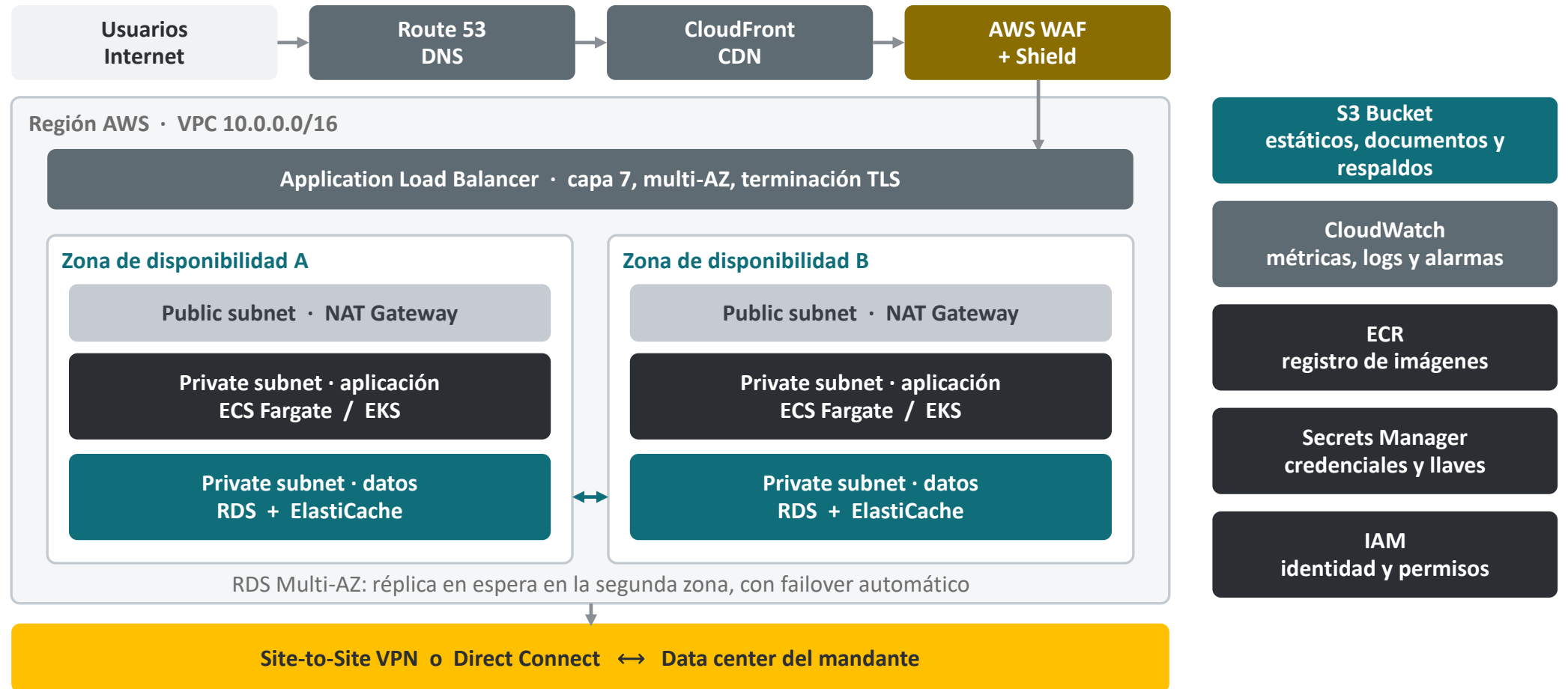
Arquitectura de Software

Qué hace cada elemento y por qué está · continuación 1

Elemento conceptual	Para qué está	Qué pasa si no está
Dos zonas de disponibilidad	Permite que la caída de un centro de datos no detenga el servicio.	El compromiso de disponibilidad no se sostiene
Almacén de objetos	Guarda documentos, imágenes y respaldos a bajo costo.	Todo terminaría dentro de la base de datos
Monitoreo y alertas	Avisa cuando algo se degrada, antes de que reclame el usuario.	No se puede cumplir ningún acuerdo de servicio
Almacén de secretos	Guarda credenciales y llaves fuera del código.	Las contraseñas terminan en el repositorio

Arquitectura de Software

Arquitectura física de referencia · AWS



Arquitectura de Software

La misma arquitectura, traducida a Azure

Cuando se comparan proveedores, la traducción prueba que la comparación es equivalente.

Componente de la arquitectura	AWS	Azure
Resolución de dominio y enrutamiento	Route 53	Azure DNS + Traffic Manager
Entrega de contenido en el borde	CloudFront	Azure Front Door
Protección de aplicación y DDoS	AWS WAF + AWS Shield	Azure WAF + Azure DDoS Protection
Red virtual privada	VPC + public / private subnets	Virtual Network + subnets
Salida a Internet desde subred privada	NAT Gateway	Azure NAT Gateway
Balanceo de capa 7	Application Load Balancer	Application Gateway
Cómputo de la aplicación	ECS Fargate o EKS	Azure Container Apps o AKS
Base de datos relacional con réplica	RDS Multi-AZ	Azure Database, zona redundante
Caché en memoria	ElastiCache for Redis	Azure Cache for Redis
Almacenamiento de objetos	S3 Bucket	Azure Blob Storage
Observabilidad	CloudWatch + X-Ray	Azure Monitor + App Insights
Secretos y certificados	Secrets Manager + ACM	Azure Key Vault
Enlace con el data center	Site-to-Site VPN / Direct Connect	VPN Gateway / ExpressRoute

Arquitectura de Software

Arquitectura híbrida · unir la nube con el data center



Opción	Cómo funciona	Banda y latencia	Costo	Cuándo elegirla
Site-to-Site VPN	Túnel IPsec cifrado sobre Internet, entre el firewall del mandante y la pasarela de la nube.	Depende del enlace a Internet; latencia variable.	Bajo	La opción por defecto: rápida de habilitar y suficiente para la mayoría.
Enlace dedicado	Circuito privado contratado que no pasa por Internet.	Garantizado y con latencia estable.	Alto y con plazo de instalación	Replicación continua de datos, volúmenes grandes o exigencia contractual.
WireGuard	Túnel de código abierto, muy liviano, montado sobre servidores propios en ambos extremos.	Buen rendimiento; depende del enlace y del equipo.	Muy bajo	Presupuesto acotado, o acceso remoto de personas al ambiente de la nube.

Arquitectura de Software

Cómo declarar todo esto en la oferta

No basta con listar servicios. Cada línea debe decir qué función cumple, qué producto la implementa y por qué se eligió.

Función en la arquitectura	Servicio propuesto	Justificación	Costo estimado
Entrega de contenido estático y caché en el borde	CloudFront (AWS)	El portal atiende a todo el país; reduce latencia y descarga los servidores.	Por GB transferido y por solicitud
Protección de la aplicación web	AWS WAF con reglas OWASP	Requisito RQ-14 de seguridad; el portal está expuesto a Internet.	Fija + por millón de solicitudes
Cómputo de la aplicación	ECS Fargate, 2 a 6 tareas	Sin administración de servidores; la carga variable no justifica un clúster.	Por vCPU-hora y GB-hora
Base de datos transaccional	RDS PostgreSQL Multi-AZ	Motor libre, réplica en segunda zona para cumplir el 99,9% comprometido.	Por hora de instancia + disco
Documentos y respaldos	S3 con política de retención	Muchos documentos escaneados; el GB cuesta mucho menos que en la base.	Por GB almacenado y por solicitud
Observabilidad	CloudWatch con alarmas	Sin detección temprana no es posible cumplir el SLA ofrecido.	Por métrica, por GB y por alarma
Enlace con el data center	Site-to-Site VPN	El ERP sigue en el data center; el volumen no justifica enlace dedicado.	Por hora de conexión + tráfico

Sección 4 · La nube

1

Calculadora de costos

Use la calculadora pública del proveedor y arme la estimación mensual de su solución. Guarde el enlace: es evidencia para el informe.

2

Marco de buena arquitectura

Lea el Well-Architected Framework del proveedor que elija y revise su diseño contra sus pilares.

3

Arquitecturas de referencia

Descargue dos diagramas de referencia oficiales de una aplicación web de tres capas y compárelos con el suyo.

4

Modelos de cómputo

Profundice en Fargate, Azure Container Apps y las funciones sin servidor: límites, arranque en frío y modelo de cobro exacto.

5

Nivel gratuito

Cree una cuenta con capa gratuita y despliegue una aplicación mínima. Es la única forma de entender de verdad la operación.

05

SECCIÓN 5

Atributos de calidad

- Escalabilidad: vertical, horizontal y elasticidad
- Alta disponibilidad, balanceo de carga y los «nueves» del uptime
- Rendimiento, observabilidad y detección temprana de errores
- El compromiso de nivel de servicio: SLA, SLO y SLI

Arquitectura de Software

Escalabilidad

Este concepto se refiere a la capacidad y configuración que tiene la arquitectura de un sistema de darle servicio a un número creciente de usuarios sin perder sus características de performance originales.

Idealmente, una arquitectura tendría que poder darle servicio a un número creciente de usuarios simplemente agregando más equipos o servidores de aplicación a la solución, con el mínimo impacto posible en cuanto a las funcionalidades desarrolladas.

Escalamiento vertical

Agregar más recursos al mismo equipo: más procesadores, más memoria.

Escalamiento horizontal

Agregar más nodos a la solución, repartiendo la carga entre ellos.

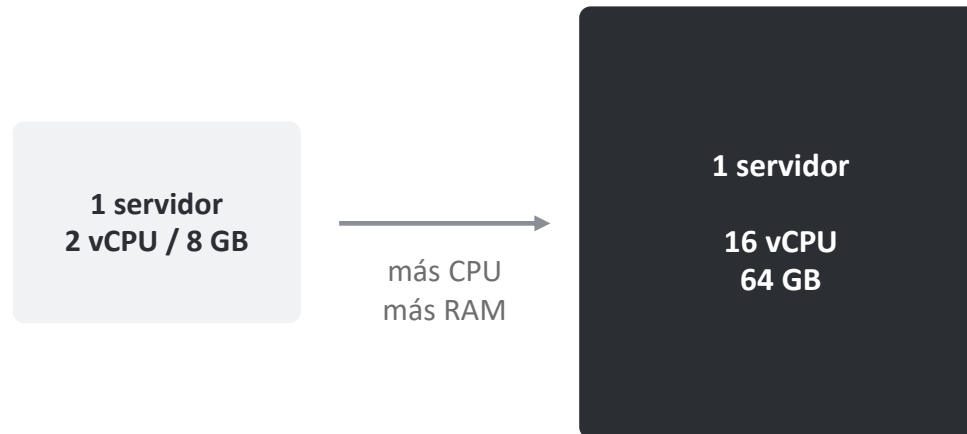
Elasticidad

Que ese ajuste ocurra solo, hacia arriba y hacia abajo, según la demanda real.

Arquitectura de Software

Escalamiento vertical · scaling up

Consiste en añadir más recursos de hardware en el mismo equipo: en general, adicionando más procesadores y memoria.



- Es fácil de aplicar: no exige cambios en la aplicación.
- Puede llegar a ser un método costoso: el precio no crece de forma lineal con la capacidad.
- No evita el problema de tener un único punto de falla: si el servidor queda fuera de servicio, no importa cuántos recursos tenga, dejará de proveer servicio.
- Tiene un techo físico: llega un momento en que no existe un equipo más grande.

Sirve como primera respuesta y para bases de datos que no se reparten con facilidad. No sirve como estrategia de disponibilidad.

Arquitectura de Software

Fundamentos

Escalamiento horizontal · scaling out

Lógica

Significa agregar más nodos a la solución. Esto se puede lograr utilizando equipos de bajo costo.

Física

Nube

Calidad

Continuidad

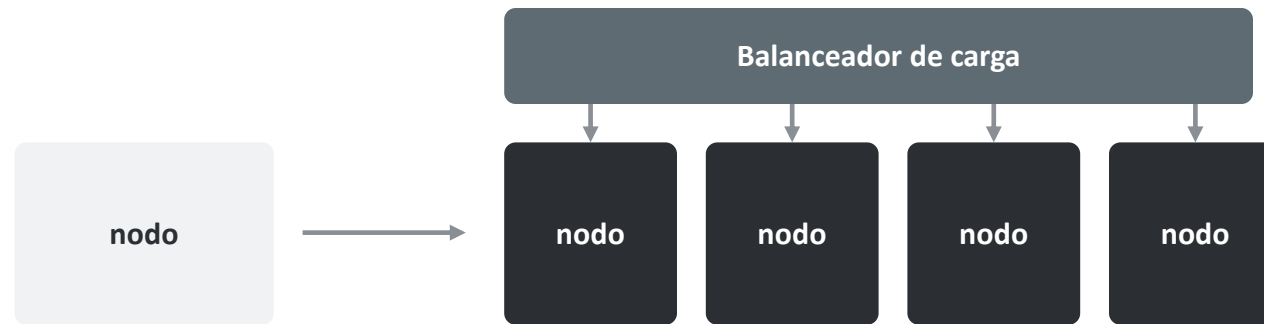
Seguridad

Identidad

Entrega

Tendencias

Propuesta



- Por lo general es más económico: varios equipos modestos en vez de uno muy grande.
- Le permite a la solución tener capacidad de alta disponibilidad: si uno de los nodos se cae, el resto de la arquitectura se adapta para repartir la carga en los N-1 servidores restantes.
- Exige que la aplicación no guarde estado en el nodo: la sesión debe vivir en un almacén compartido.
- La base de datos no siempre acompaña: repartirla es un problema aparte.

Consecuencia para la arquitectura lógica: si quiere escalar horizontalmente, la aplicación tiene que ser «sin estado». Esa decisión se toma al diseñar el software, no al comprar el servidor.

Arquitectura de Software

Elasticidad y autoescalado

La elasticidad es escalabilidad automática y en ambos sentidos: crecer cuando hay demanda y —sobre todo— decrecer cuando no la hay.

Disparadores

Métricas que gatillan el ajuste: uso de CPU, cantidad de peticiones en cola, latencia observada, o una programación horaria conocida.

Límites

Mínimo y máximo de instancias. El mínimo protege la disponibilidad; el máximo protege el presupuesto.

Tiempo de reacción

Desde que sube la carga hasta que la instancia nueva atiende tráfico pasan minutos. Si el pico es más rápido, hay que anticiparlo.

Efecto económico

Se dimensiona para la carga habitual y no para el peak anual. Ese ahorro es el argumento central a favor de la nube.

Ejemplo para su oferta: «El sistema opera con 2 instancias en régimen normal y escala hasta 8 cuando el uso de CPU supera el 65% durante 3 minutos. El costo se estima sobre un promedio de 3 instancias, con el peak de matrícula modelado por separado.»

Arquitectura de Software

Balanceo de carga de trabajo

Esta característica se basa en que la configuración de la arquitectura asegure que cada equipo o servidor de aplicaciones tenga una carga de trabajo justa: que compartan todo el trabajo de forma equitativa y que no haya un único servidor con carga muy alta mientras el resto se encuentra ocioso.

Reparto por turnos

Cada petición va al siguiente nodo de la lista. Simple y suficiente cuando los nodos son iguales.

Menor cantidad de conexiones

Envía la petición al nodo que tiene menos conexiones activas. Mejor cuando las peticiones duran distinto.

Comprobación de estado

El balanceador consulta periódicamente a cada nodo y saca de rotación al que no responde. Esta es la parte que da disponibilidad.

Sesiones pegajosas

Mantener a un usuario en el mismo nodo. Resuelve el estado, pero rompe el reparto equitativo: mejor sacar la sesión del nodo.

Capa 4 o capa 7: un balanceador de capa 4 reparte por IP y puerto —rápido y barato—; uno de capa 7 lee la petición y puede enrutar por ruta o por cabecera. Y él mismo puede fallar: en una arquitectura seria va redundado o es un servicio gestionado con disponibilidad comprometida.

Arquitectura de Software

Alta disponibilidad

El propósito principal de tener múltiples servidores en una arquitectura lleva al concepto de poder soportar alta disponibilidad del sistema: si alguno de los equipos deja de ofrecer servicio por cualquier razón, el sistema debe continuar dando servicio por medio de los servidores restantes.

Es decir, si un servidor deja de funcionar, las solicitudes de un usuario tienen que ser redireccionadas a los servidores restantes sin tener ningún tipo de interrupción en el servicio ni requerir ningún tipo de acción.

Disponibilidad

Grado en que una aplicación o servicio está disponible cuándo y cómo los usuarios esperan. Se mide por la percepción del usuario final.

Uptime

Cantidad de tiempo que un sistema, servidor o dispositivo trabaja sin interrupciones.

Downtime

Concepto opuesto: el tiempo durante el cual el sistema no está operativo o está inaccesible.

Arquitectura de Software

Lo que hay que examinar

Inactividad NO planificada

- Fallos del servidor: hardware, sistema operativo, aplicación.
- Fallos de datos: fallas en el almacenamiento, errores humanos, fallos del sitio.
- Corte de energía o de climatización.
- Pérdida del enlace de comunicaciones.
- Incidentes de seguridad.

Inactividad PLANIFICADA

- Cambios en el sistema: actualizaciones, parches, nuevas versiones.
- Cambios de datos: migraciones, reorganización, cargas masivas.
- Mantenimiento preventivo de hardware.
- Planificar la inactividad puede ser muy complejo, especialmente en empresas que soportan usuarios en múltiples zonas horarias.

Cuatro características de una solución de alta disponibilidad:

Fiabilidad

Hardware y software fiables, incluida la base de datos.

Recuperación

Capacidad de restaurar en el tiempo que exige el SLA.

Operación continua

Mantener el acceso a los datos incluso durante el mantenimiento.

Detección de errores

Descubrir el problema rápido: si demora 90 minutos, el SLA ya se incumplió.

Arquitectura de Software

Los «nueves» de la disponibilidad

Disponibilidad	Caída al año	Caída al mes	Caída al día	Qué arquitectura lo sostiene
90%	36,5 días	73 hrs	2,4 hrs	Un servidor, sin redundancia
95%	18,3 días	36,5 hrs	1,2 hrs	Un servidor con respaldo manual
98%	7,3 días	14,6 hrs	28,8 min	Respaldo diario y soporte en horario hábil
99%	3,7 días	7,3 hrs	14,4 min	Redundancia parcial
99,5%	1,8 días	3,66 hrs	7,22 min	Redundancia en la capa de aplicación
99,9%	8,8 hrs	43,8 min	1,46 min	Redundancia completa + failover automático
99,95%	4,4 hrs	21,9 min	43,8 s	Multi-zona, monitoreo 24/7
99,99%	52,6 min	4,4 min	8,6 s	Multi-zona activo-activo, equipo de guardia
99,999%	5,26 min	26,3 s	0,86 s	Multi-región, automatización total
99,9999%	31,5 s	2,62 s	0,08 s	Muy pocas organizaciones lo alcanzan

El 99,9% mensual equivale a 43,8 minutos de caída: si su plan de recuperación tarda una hora, ese compromiso ya es incumplible.

Arquitectura de Software

Cómo se calcula la disponibilidad

$$\text{Disponibilidad} = ((A - B) / A) \times 100 \%$$

Donde:

Variable	Definición	Ejemplo
A	Horas comprometidas de disponibilidad	24 × 365 = 8.760 horas / año
B	Número de horas fuera de línea durante el tiempo de disponibilidad comprometido	15 horas por falla en un disco + 9 horas por mantenimiento preventivo no planeado = 24 horas
Resultado	Disponibilidad efectiva del período	$((8.760 - 24) / 8.760) \times 100 = 99,73\%$



Note que el mantenimiento no planificado también descuenta. Por eso los contratos definen ventanas de mantenimiento acordadas, que quedan excluidas del cálculo: eso se negocia y se escribe en el SLA.

Arquitectura de Software

Detección de errores y observabilidad

Si un componente de la arquitectura falla, la rápida detección es esencial. Aunque sea posible recuperarse rápidamente de un corte, si toma 90 minutos descubrir el problema, no se puede satisfacer el SLA.

Métricas

Números en el tiempo: uso de CPU, latencia, tasa de error, cantidad de peticiones. Responden «¿cómo está el sistema ahora?».

Registros (logs)

El detalle de lo que ocurrió, con contexto. Responden «¿qué pasó exactamente en ese momento?».

Trazas distribuidas

El recorrido completo de una petición a través de varios servicios. Responden «¿dónde se demoró?».

Alertas

Reglas que avisan antes de que el usuario se dé cuenta. Una alerta que nadie atiende no sirve: hay que definir el turno.

Tablero operativo

Vista única del estado del servicio, disponible para el mandante. Es una entrega concreta que se puede ofrecer.

Prueba de recuperación

Restaurar el respaldo en un ambiente de prueba, de forma periódica. Un respaldo que nunca se restauró no es un respaldo.

Arquitectura de Software

Rendimiento: cómo se mide de verdad

Latencia

Tiempo que tarda una operación en responder. Se compromete por tipo de operación, no en general.

Throughput

Cantidad de operaciones atendidas por unidad de tiempo. Es la medida de capacidad.

Concurrencia

Cuántos usuarios u operaciones simultáneas soporta antes de degradarse.

Utilización

Qué porcentaje de CPU, memoria y disco se ocupa en régimen y en el máximo.

Por qué el promedio no sirve:

Medida	Qué dice	Por qué importa
Promedio	El tiempo típico de respuesta	Un 5% muy lento no mueve el promedio
Percentil 95 (p95)	El 95% de las peticiones responde en este tiempo o menos	Es el compromiso razonable en un SLA: representa la experiencia de casi todos
Percentil 99 (p99)	Sólo 1 de cada 100 peticiones es más lenta que esto	Revela problemas intermitentes que el promedio esconde
Máximo	La peor respuesta observada	Detecta tiempos de espera agotados y bloqueos

Escriba «el p95 de la consulta de saldo es menor a 2 segundos con 500 usuarios concurrentes», no «el sistema será rápido».

Arquitectura de Software

SLA, SLO y SLI · lo que se firma

SLI · indicador

La métrica que se mide.

Ejemplo: porcentaje de peticiones atendidas correctamente en menos de 2 segundos.

SLO · objetivo

El valor que se quiere cumplir internamente.

Ejemplo: 99,9% de las peticiones cumplen el indicador, medido mensualmente.

SLA · acuerdo

El compromiso contractual con el mandante, con consecuencias si no se cumple.

Ejemplo: 99,5% mensual, con descuento del 5% de la cuota si no se alcanza.

Lo que un SLA debe declarar para ser defendible:

- Qué servicio cubre y qué queda excluido.
- Cómo y dónde se mide, y quién es el árbitro de la medición.
- El horario de cobertura: 24/7 no cuesta lo mismo que horario hábil.
- Las ventanas de mantenimiento acordadas, excluidas del cálculo.
- Los tiempos de respuesta y de solución por severidad del incidente.
- Las penalidades y su tope máximo.



Regla de oro: ofrezca el nivel de servicio que su arquitectura efectivamente sostiene. Un SLA que no puede cumplir es una multa diferida, y debe ir en la matriz de riesgos.

Sección 5 · Atributos de calidad

1

Cálculo de disponibilidad

Calcule la disponibilidad compuesta de su arquitectura: componentes en serie multiplican sus disponibilidades y bajan el resultado.

2

Pruebas de carga

Investigue una herramienta de pruebas de carga y diseñe el escenario que validaría el p95 que va a comprometer.

3

SLA reales

Descargue los acuerdos de nivel de servicio publicados por dos proveedores y compare exclusiones, medición y compensaciones.

4

Observabilidad

Revise el estándar OpenTelemetry y defina las cinco métricas y las tres alertas mínimas de su solución.

5

Ingeniería de confiabilidad

Lea sobre SRE: presupuesto de error, indicadores y objetivos de servicio. Es el marco detrás de todo lo visto en esta sección.

06

SECCIÓN 6

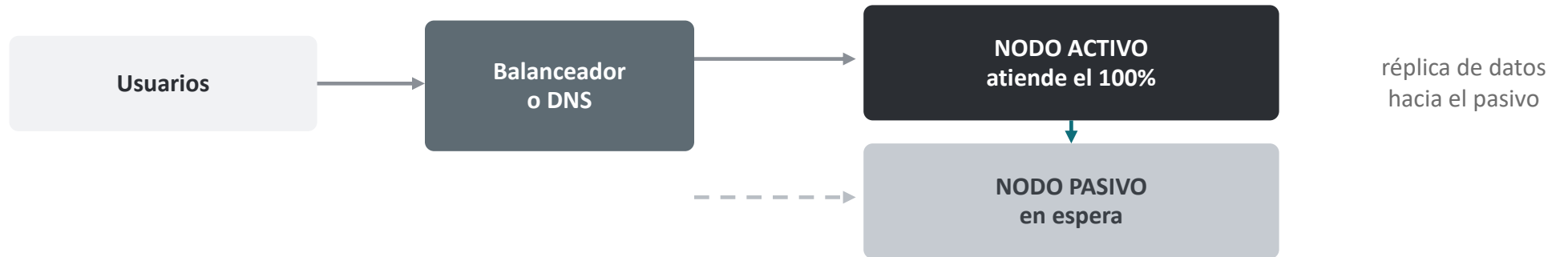
Continuidad

- Activo-Pasivo y Activo-Activo: qué significa cada uno y qué cuesta
- RTO y RPO: los dos números que definen toda la estrategia
- Sincronizar datos entre sitios: replicación, topologías y conflictos
- Escenarios híbridos: nube + on-premise y nube + nube
- Respalos: tipos, política, inmutabilidad y la prueba de restauración

Arquitectura de Software

Activo – Pasivo · qué significa

Hay dos nodos —o dos sitios— capaces de dar el servicio, pero sólo uno atiende tráfico. El otro espera, listo para tomar el relevo.



La conmutación (failover) puede ser automática o manual. Ese detalle es el que define el RTO real.

A favor

- Simple de diseñar y de operar.
- No exige que la aplicación tolere ejecución simultánea.
- Sin conflictos de datos: sólo un nodo escribe.

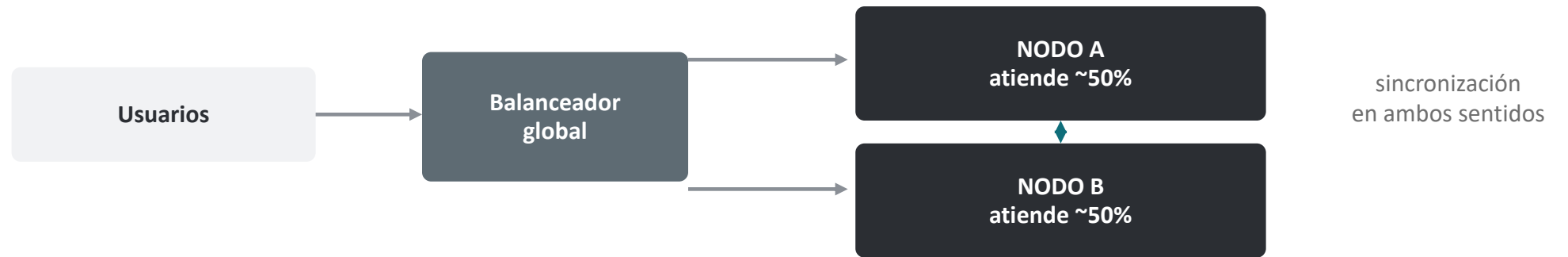
En contra

- Se paga capacidad que no produce.
- El nodo pasivo suele descubrirse roto justo cuando se necesita.
- El RTO nunca es cero: siempre hay un tiempo de conmutación.

Arquitectura de Software

Activo – Activo · qué significa

Los dos nodos —o los dos sitios— atienden tráfico al mismo tiempo. Si uno cae, el otro absorbe su carga.



A favor

- Toda la capacidad comprada está produciendo.
- La conmutación es casi instantánea: el otro nodo ya está atendiendo.
- Permite mantenimiento sin ventana: se saca un nodo de rotación.

En contra

- Exige aplicación sin estado y sesión compartida.
- Los datos deben sincronizarse en ambos sentidos: aparecen los conflictos.
- Hay que dimensionar cada nodo para absorber la carga del otro (regla del 50%).

Arquitectura de Software

Pasivo frío, tibio y caliente

El nodo en espera puede estar apagado, encendido pero sin datos al día, o listo para recibir tráfico en segundos. Cada nivel tiene su precio.

Modalidad	En qué estado está el sitio alternativo	RTO típico	RPO típico	Costo relativo
Frío	Hardware o suscripción disponible, pero sin sistema instalado. Hay que aprovisionar y restaurar desde respaldo.	Horas a días	Hasta el último respaldo	Bajo
Tibio	Sistema instalado y actualizado, con datos replicados periódicamente. No atiende tráfico.	Minutos a horas	Minutos a horas	Medio
Caliente	Réplica completa y sincronizada, lista para tomar tráfico con sólo cambiar el enrutamiento.	Segundos a minutos	Segundos o cero	Alto
Activo-Activo	Ya está atendiendo tráfico. No hay «conmutación»: hay redistribución de carga.	Casi cero	Cero o casi cero	Muy alto



El mandante no elige una modalidad: elige un RTO y un RPO. La modalidad es la consecuencia técnica de esa elección, y el costo es la consecuencia económica.

Arquitectura de Software

RTO y RPO · los dos números

Puede haber muchas opciones para recuperarse de un fallo. Lo primero es determinar qué fallos pueden ocurrir y de qué forma recuperarse en el tiempo que satisface las necesidades del negocio.

RTO · Recovery Time Objective

- ¿Cuánto tiempo puede estar caído el sistema?
- Se mide desde la falla hasta el servicio restablecido.
- RTO de 4 horas: basta restaurar desde respaldo.
- RTO de 15 minutos: exige un sitio alternativo tibio o caliente.
- RTO cercano a cero: exige activo-activo en dos ubicaciones.

RPO · Recovery Point Objective

- ¿Cuántos datos puede perder el negocio?
- Se mide en tiempo: cuánto trabajo se pierde.
- RPO de 24 horas: respaldo diario es suficiente.
- RPO de 1 hora: respaldo incremental frecuente o replicación asincrónica.
- RPO cero: replicación sincrónica, y se paga en latencia.

Cómo se determinan: no los define el equipo técnico, los define el impacto en el negocio del mandante. Pregunta guía: «si el sistema se cae un martes a las 11 de la mañana, ¿qué deja de poder hacer la organización y cuánto cuesta cada hora?».

Arquitectura de Software

Activo-Activo vs. Activo-Pasivo · decidir

Criterio	Activo – Pasivo	Activo – Activo
Uso de la capacidad instalada	50%: la mitad espera	100%: todo produce
RTO alcanzable	Segundos a horas, según frío/tibio/caliente	Casi cero
Complejidad de la aplicación	Baja: puede guardar estado local	Alta: debe ser sin estado, con sesión compartida
Complejidad de los datos	Baja: un solo nodo escribe	Alta: hay que resolver escritura concurrente y conflictos
Licenciamiento del motor	A veces se licencia sólo el activo	Se licencian ambos nodos
Costo de red entre sitios	Bajo: sólo replicación	Alto: tráfico permanente en ambos sentidos
Riesgo de partición (split-brain)	Bajo	Alto: exige árbitro o quórum
Mantenimiento sin ventana	Parcial: se conmuta y se mantiene el otro	Sí: se saca un nodo de rotación
Cuándo elegirlo	La mayoría de los casos del curso: cumple 99,5% – 99,9% a costo razonable	Servicios críticos con RTO cercano a cero y presupuesto que lo respalde

Arquitectura de Software

El problema de sincronizar datos

Replicar la aplicación es fácil: se copia y se levanta otra instancia. Replicar los datos es el problema difícil, porque los datos cambian.

La latencia es física

Confirmar una escritura en un sitio remoto toma el tiempo de ida y vuelta del enlace. Ese tiempo se suma a cada transacción y no se puede optimizar por software.

Hay que elegir

Ante una caída del enlace entre sitios sólo se puede sostener dos de tres: consistencia, disponibilidad y tolerancia a la partición. La partición ocurre igual, así que la elección real es entre consistencia y disponibilidad.

Partición de cerebro

Si el enlace se corta y ambos sitios se creen activos, ambos aceptan escrituras y los datos divergen. Reconciliar después es caro y a veces imposible.

No todo dato es igual

El saldo de una cuenta exige consistencia inmediata. El contador de visitas no. Clasificar los datos por criticidad permite no pagar consistencia estricta en todo.

Primera decisión de la sección: qué datos se replican, en qué sentido y con qué garantía. No es una decisión de infraestructura: es una decisión de negocio.

Arquitectura de Software

Replicación sincrónica vs. asincrónica

SINCRÓNICA

- La transacción se confirma sólo cuando ambos sitios escribieron.
- $RPO = 0$: no se pierde ningún dato confirmado.
- Cada escritura paga la latencia del enlace: si son 40 ms de ida y vuelta, cada transacción cuesta 40 ms más.
- Si el sitio remoto no responde, o se detiene el servicio o se degrada a asincrónica.
- Viable sólo con enlaces de baja latencia: mismo edificio, misma ciudad o zonas de la misma región.

ASINCRÓNICA

- La transacción se confirma localmente y después se envía al otro sitio.
- $RPO > 0$: se pierde lo que estaba en tránsito al momento de la falla.
- No penaliza el tiempo de respuesta de la aplicación.
- El sitio remoto puede caerse sin afectar la operación: se acumula el retraso.
- Es la única opción razonable entre ciudades, entre países o entre proveedores de nube.



Regla práctica: sincrónica dentro de la región (entre zonas de disponibilidad), asincrónica entre regiones o hacia on-premise. Declare el retraso esperado de la réplica: ése es su RPO real.

Arquitectura de Software

Topologías de replicación

Primario – réplica

Un solo nodo acepta escrituras; los demás son copias de sólo lectura.

- + Simple, sin conflictos posibles.
- La réplica no puede escribir: en un failover hay que promoverla.

Uso: el caso más frecuente y el recomendado por defecto.

Multi-primario

Varios nodos aceptan escrituras y se replican entre sí.

- + Habilita activo-activo real con escritura en ambos sitios.
- Aparecen conflictos cuando dos sitios modifican el mismo dato.

Uso: sólo si el negocio exige escribir en ambos sitios.

Quórum

La escritura se confirma cuando la acepta la mayoría de los nodos.

- + Tolera la caída de una minoría sin perder consistencia.
- Requiere número impar de nodos y un tercer sitio árbitro.

Uso: bases distribuidas y clústeres de tres o más nodos.

Envío de registro / CDC

Se replica el registro de transacciones, o se capturan los cambios y se publican como eventos.

- + Sirve para alimentar otros sistemas, reportería o un lago de datos.
- Retraso variable; no es un mecanismo de alta disponibilidad.

Uso: integración y analítica.

Arquitectura de Software

Escenario A · nube + on-premise

El mandante conserva su data center y la solución se extiende a la nube. La pregunta es qué dato vive dónde y en qué sentido viaja.



Qué hay que definir	Decisión típica
Conectividad	Enlace dedicado si hay replicación continua; VPN sólo para volúmenes bajos. Declare su costo.
Sentido del dato	El primario suele quedar donde está el sistema heredado; la nube recibe réplica de lectura.
Identidad	Una sola fuente de identidad —el directorio del mandante—, federada a la nube. Nunca dos listas.
Latencia	Medir el ida y vuelta real antes de comprometer tiempos: el enlace se suma a cada consulta.

Arquitectura de Software

- Fundamentos
- Lógica
- Física
- Nube
- Calidad
- Continuidad
- Seguridad
- Identidad
- Entrega
- Tendencias
- Propuesta

Escenario B · nube + nube

Dos proveedores distintos, o dos regiones del mismo proveedor. Reduce el riesgo de proveedor único, pero introduce costos que suelen olvidarse.

Aspecto	Dos regiones · mismo proveedor	Dos proveedores distintos
Complejidad	Moderada: mismas herramientas y misma consola	Alta: dos ecosistemas, dos equipos formados
Replicación de datos	El proveedor ofrece replicación entre regiones como servicio	Hay que construirla: replicación del motor o flujo de eventos propio
Costo de transferencia	Tráfico entre regiones, facturado por GB	Salida a Internet en ambos sentidos: el ítem más caro y el más olvidado
Enrutamiento del tráfico	DNS con verificación de estado del proveedor	DNS de un tercero independiente, para no depender de ninguno de los dos
Identidad y secretos	Un solo sistema de identidad	Federación entre dos sistemas distintos, o un proveedor de identidad externo
Qué protege	Caída de una región completa	Caída total de un proveedor y dependencia comercial
Cuándo se justifica	Exigencia de continuidad ante desastre regional	Exigencia contractual explícita del mandante, o riesgo de proveedor evaluado y documentado

Advertencia para la oferta: la multinube «por si acaso» es innovación que resta. Si la propone, cuantifique el costo adicional de red, licencias y horas de operación.

Arquitectura de Software

Conflictos: cuando dos sitios escriben lo mismo

En una topología multi-primario, tarde o temprano dos sitios modifican el mismo registro casi al mismo tiempo. Hay que decidir de antemano qué pasa.

Gana la última escritura

Se conserva la versión con la marca de tiempo más reciente.

Simple, pero pierde datos en silencio y depende de relojes bien sincronizados.

Control de versión

Cada registro lleva una versión; si no coincide, la escritura se rechaza y la aplicación decide.

No pierde datos, pero traslada el problema a la aplicación.

Fusión automática

Estructuras diseñadas para combinarse sin conflicto (contadores, conjuntos, listas).

Elegante, pero sólo sirve para ciertos tipos de dato.

Evitarlo por diseño

Repartir los datos de modo que cada sitio sea dueño exclusivo de una porción: por sucursal, por región, por rango de clientes.

Es la estrategia más robusta: el conflicto simplemente no ocurre.

Y para el corte del enlace: defina un árbitro o un tercer sitio de quórum que decida quién sigue siendo el activo. Sin árbitro, ambos sitios se crearán activos y los datos divergirán.

Arquitectura de Software

Failover y failback · el procedimiento

1	Detección	Monitoreo que confirma la falla y descarta un falso positivo. Umbral y tiempo de confirmación declarados.
2	Decisión	Automática por regla, o manual con responsable identificado y canal de autorización definido.
3	Promoción	La réplica pasa a primaria: se habilita la escritura y se verifica la integridad de los datos.
4	Redirección	Cambio de enrutamiento. Con DNS, el tiempo de vida del registro define cuánto tardan los usuarios en llegar al nuevo sitio.
5	Verificación	Pruebas funcionales mínimas antes de declarar el servicio restablecido. Comunicación al mandante.
6	Failback	Vuelta al sitio original: resincronizar los datos generados durante la contingencia, y recién entonces conmutar de vuelta.

Un plan de recuperación que nunca se ensayó no es un plan: comprometa una prueba de conmutación al menos anual y declárela en la oferta.

Arquitectura de Software

Respaldo no es lo mismo que replicación

Es el malentendido más caro de esta unidad: creer que por tener réplica no se necesita respaldo.

REPLICACIÓN

- Mantiene una copia al día en otro sitio.
- Protege contra: caída de un servidor, de una zona o de un sitio completo.
- NO protege contra: borrado accidental, corrupción de datos, error de la aplicación o cifrado por ransomware.
- Porque el error se replica al otro sitio en segundos.
- Sirve para la disponibilidad.

RESPALDO

- Guarda copias de distintos momentos en el tiempo.
- Protege contra: borrado, corrupción, error humano, ransomware y fallas lógicas.
- NO reemplaza a la replicación: restaurar toma tiempo, a veces horas.
- Permite volver a un punto anterior al error.
- Sirve para la recuperación.



Toda arquitectura seria tiene las dos cosas. En el informe deben aparecer como dos secciones distintas, con dos costos distintos.

Arquitectura de Software

Tipos de respaldo

Tipo	Qué copia	Espacio que ocupa	Tiempo de copia	Tiempo de restauración
Completo	Todo, cada vez.	Máximo	Largo	Corto: un solo archivo
Incremental	Sólo lo cambiado desde la copia anterior, sea cual sea.	Mínimo	Muy corto	Largo: hay que encadenar todas las copias
Diferencial	Todo lo cambiado desde el último respaldo completo.	Medio	Medio	Medio: completo + último diferencial
Instantánea (snapshot)	Estado del volumen en un instante, a nivel de almacenamiento.	Bajo al inicio, crece con los cambios	Segundos	Muy corto
Continuo (PITR)	Registro permanente de transacciones que permite volver a cualquier segundo.	Alto	Continuo	Variable: exige reproducir el registro

Advertencia sobre las instantáneas: viven en el mismo almacenamiento que el dato original. Son rapidísimas para deshacer un error, pero no son un respaldo: si se pierde la cabina o la cuenta, se pierden con ella.

Arquitectura de Software

La política de respaldo

Regla 3-2-1

Tres copias de los datos, en dos medios distintos, con una fuera del sitio. Es el estándar citable y el mínimo defendible.

Extensión 3-2-1-1-0

Una de las copias además inmutable o desconectada, y cero errores en la verificación. Es la respuesta al ransomware.

Frecuencia

Se deriva del RPO: si el RPO es 1 hora, no puede haber respaldo diario. Declare frecuencia por tipo de dato.

Retención GFS

Esquema abuelo-padre-hijo: diarias por 30 días, semanales por 3 meses, mensuales por 1 año, anuales por 5. Ajústelo a la normativa del caso.

Inmutabilidad

Copias que no se pueden borrar ni alterar durante un período, ni siquiera por un administrador. Es la defensa contra el borrado malicioso.

Cifrado

El respaldo contiene los mismos datos personales que la producción: va cifrado, con las llaves gestionadas aparte.

Aislamiento de credenciales

Quien administra la producción no debe poder borrar los respaldos. Cuentas y permisos separados.

Registro y alertas

Cada ejecución deja registro, y el fallo genera alerta. Un respaldo que falla en silencio durante tres meses es lo habitual.

Arquitectura de Software

Restaurar · lo único que importa

Nadie necesita respaldos. Lo que se necesita son restauraciones. Y sólo se sabe que un respaldo sirve cuando se restaura.

Pruebe la restauración

Restaurar en un ambiente aislado, de forma periódica, y medir cuánto tardó. Ese número es su RTO real, no el que estimó en la propuesta.

Pruebe el escenario completo

No sólo un archivo: la base completa, con la aplicación levantada y funcionando. Una restauración parcial no demuestra nada.

Documente el procedimiento

Paso a paso, ejecutable por alguien que no lo escribió, a las 3 de la mañana y sin acceso a quien lo diseñó.

Considere el escenario peor

Ransomware: la producción y las réplicas están cifradas. ¿Desde dónde restaura? Ésa es la copia inmutable y desconectada.

Compromiso concreto y barato de ofrecer: «Prueba de restauración trimestral en ambiente aislado, con informe de resultados y tiempo medido, entregado al mandante». Diferencia una oferta seria de una que sólo dice que respalda.

Arquitectura de Software

Continuidad en la oferta · qué declarar

Elemento	Qué declarar	Ejemplo
Modalidad	Activo-activo, activo-pasivo caliente, tibio o frío, y por qué.	Activo-pasivo caliente entre dos zonas de disponibilidad.
RTO comprometido	Tiempo máximo de interrupción y cómo se alcanza.	2 horas, con promoción automática de la réplica.
RPO comprometido	Pérdida máxima de datos y el mecanismo que la sostiene.	15 minutos, por replicación asincrónica continua.
Replicación	Tipo, sentido, latencia esperada y qué datos incluye.	Asincrónica del primario a la réplica; retraso medio menor a 10 s.
Respaldo	Frecuencia, tipo, destino, cifrado y retención.	Completo semanal + incremental diario; retención GFS a 5 años.
Inmutabilidad	Si existe copia inmutable y por cuánto tiempo.	Copia inmutable de 35 días en almacenamiento de objetos.
Pruebas	Frecuencia de la prueba de restauración y de la de conmutación.	Restauración trimestral; conmutación anual, con informe.
Costo asociado	Cuánto de la inversión y del gasto anual corresponde a continuidad.	Se identifica como línea separada en el flujo de caja.

Sección 6 · Continuidad

1

Análisis de impacto al negocio

Investigue cómo se hace un BIA y úselo para justificar el RTO y el RPO de su caso con cifras, no con intuición.

2

Replicación de bases de datos

Estudie la replicación del motor que eligió: modos síncrono y asíncrono, promoción de réplica y tiempo de conmutación.

3

Teorema CAP y quórum

Profundice en CAP y en los esquemas de quórum. Entienda por qué no se puede tener consistencia y disponibilidad ante una partición.

4

Respaldo inmutable

Revise cómo se configura una copia inmutable y qué protección real ofrece frente a ransomware.

5

Plan de recuperación

Busque una plantilla de DRP y escriba el procedimiento de conmutación de su solución, paso a paso y con responsables.

07

SECCIÓN 7

Seguridad

- Defensa en profundidad, capa por capa del modelo OSI
- La capa de seguridad de una solución expuesta a Internet
- Identidad, acceso y protección de datos
- Caso integrador: red interna, home office, sistemas internos y portales públicos
- Monitoreo, respuesta a incidentes y cumplimiento normativo

Arquitectura de Software

Defensa en profundidad, capa por capa

«La solución será segura» no es una medida. Seguridad es un control concreto en cada capa, y cada control es un componente que se cotiza.

Capa OSI	Amenaza típica	Control que corresponde
7 · Aplicación	Inyección SQL, XSS, abuso de la API, credenciales robadas	WAF con reglas OWASP, validación de entradas, límite de llamadas, autenticación fuerte
6 · Presentación	Tráfico en claro, certificados vencidos, cifrado obsoleto	TLS vigente de extremo a extremo, gestión y renovación automática de certificados
5 · Sesión	Robo o fijación de sesión, sesiones que nunca expiran	Cookies seguras, expiración e inactividad, revocación central de sesiones
4 · Transporte	Puertos abiertos de más, agotamiento de conexiones	Grupos de seguridad con puertos mínimos, límites de conexión, protección contra inundación
3 · Red	Movimiento lateral dentro de la red, acceso desde direcciones no autorizadas	Segmentación en subredes, firewall entre zonas, listas de origen permitido, VPN
2 · Enlace	Equipos no autorizados en la red interna	VLAN por tipo de usuario, control de acceso por puerto
1 · Física	Acceso físico al equipamiento, corte de enlace	Control de acceso a la sala, enlaces redundantes con proveedores distintos

En el informe, esta tabla —con la columna de la derecha completada para su caso— responde de una sola vez a la pregunta por la seguridad de la solución.

Arquitectura de Software

La superficie de exposición

Antes de proteger, hay que saber qué está expuesto. Todo lo que se puede alcanzar desde Internet es superficie de ataque, y todo lo que no se publica no hay que defenderlo.

Portal público

El sitio del cliente final. Expuesto por definición: es el que más protección necesita.

API pública

Consumida por aplicaciones móviles o por terceros. Necesita autenticación, versión y límite de uso.

Endpoints de webhook

URLs que reciben notificaciones de terceros. Son públicas y hay que validar el origen de cada llamada.

Consolas de administración

Nunca deben publicarse a Internet. Se acceden por VPN o por acceso condicional con doble factor.

Bases de datos

Jamás expuestas. Si el diagrama muestra la base de datos con dirección pública, la propuesta ya perdió puntaje.

Ambientes no productivos

Desarrollo y QA olvidados con datos reales y sin protección son la puerta de entrada más común.



Principio de mínima exposición: se publica el puerto 443 del portal y de la API, y nada más. Todo el resto se alcanza desde dentro o por acceso controlado.

Arquitectura de Software

La capa de seguridad de una solución expuesta



Cada escalón que se omite hay que justificarlo. Un portal público sin WAF es un hallazgo que el evaluador va a marcar.

Arquitectura de Software

Identidad y acceso

Hoy la mayoría de los incidentes no rompe el muro: entra con una credencial válida. Por eso la identidad es el nuevo perímetro.

Identidad centralizada

Una sola fuente de usuarios para todos los sistemas. Cuando alguien deja la organización, se desactiva en un lugar y pierde el acceso a todo.

Inicio de sesión único

El usuario se autentica una vez y accede a los sistemas autorizados. Menos contraseñas, menos reutilización, mejor trazabilidad.

Doble factor

Obligatorio para administradores y para todo acceso desde fuera de la red corporativa. Es la medida individual con mejor relación costo-beneficio.

Permisos por rol

Se otorgan a roles, no a personas. Mínimo privilegio: cada rol tiene sólo lo que necesita para su función.

Cuentas de servicio

Las que usan los sistemas entre sí: nominadas, con permisos acotados, credenciales rotadas y nunca compartidas con personas.

Revisión periódica

Recertificación de accesos al menos anual. Los permisos se acumulan: nadie devuelve un acceso que ya no usa.

«Confianza cero»: ninguna red es confiable por sí sola, ni siquiera la interna. Cada acceso se autentica y se autoriza según quién es, desde qué dispositivo y a qué recurso, venga de donde venga.

Arquitectura de Software

Protección de los datos

Cifrado en tránsito

TLS vigente en toda comunicación, incluida la interna entre servicios. Sin excepciones «porque es la red privada».

Cifrado en reposo

Discos, base de datos, respaldos y almacenamiento de objetos cifrados. En la nube suele venir activado; declárelo igual.

Gestión de llaves

Las llaves viven en un servicio de gestión de llaves o en un almacén de secretos, con rotación y con acceso auditado.

Minimización

No recolectar ni almacenar lo que no se necesita. El dato que no existe no se puede filtrar ni hay que protegerlo.

Enmascaramiento

Los ambientes de desarrollo y QA usan datos anonimizados o sintéticos, nunca la copia de producción tal cual.

Registro de auditoría

Quién accedió a qué dato personal y cuándo, conservado por el período que exige la normativa e inalterable.

Seguridad en el ciclo de desarrollo:

OWASP Top 10

Referencia estándar de las vulnerabilidades más frecuentes en aplicaciones web. Citable en la oferta.

Análisis automático

Revisión de código y de dependencias en cada cambio, dentro de la canalización.

Gestión de parches

Bibliotecas y sistemas actualizados. La mayoría de los ataques usa fallas ya conocidas.

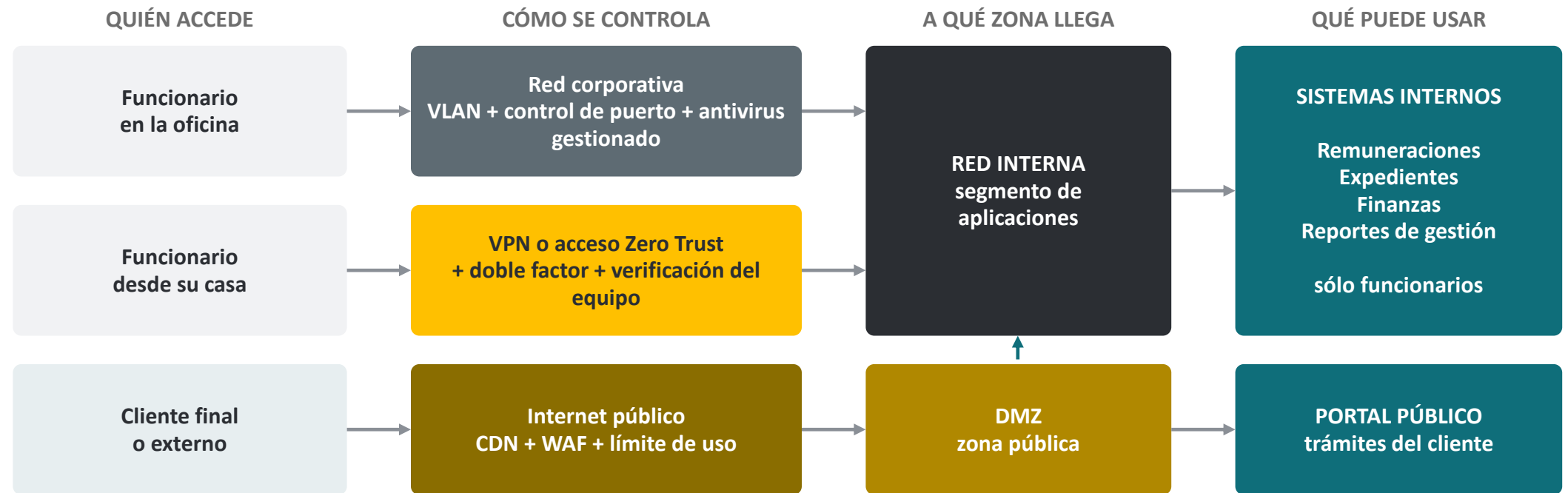
Pruebas de intrusión

Antes de salir a producción, y periódicas después. Es un ítem cotizable de la oferta.

Arquitectura de Software

Caso · empresa con trabajo mixto

Tres orígenes de acceso, tres caminos de control, dos tipos de sistema.



el portal accede a la API interna por un único puerto

Regla de oro: el funcionario en su casa NO recibe más permisos por estar en la VPN. Recibe los mismos que en la oficina, y para los trámites entra al portal público como cualquier persona.

Arquitectura de Software

Dos zonas, dos reglas

	Sistemas internos · administrativos	Portal público · cliente final
Quién entra	Sólo funcionarios con cuenta corporativa	Cualquier persona en Internet; con o sin registro previo
Desde dónde	Red corporativa, o desde la casa por VPN / acceso Zero Trust	Desde cualquier red y cualquier dispositivo
Autenticación	Identidad corporativa con inicio de sesión único y doble factor	Registro propio o identidad digital externa; doble factor para operaciones sensibles
Exposición a Internet	Ninguna: no tiene dirección pública	Total: es su razón de ser
Dónde se despliega	Subred privada de aplicaciones	DMZ, detrás de CDN y WAF
Datos que maneja	Información sensible de la organización y de terceros	Datos del propio usuario, acotados a su trámite
Carga esperada	Predecible: cantidad conocida de funcionarios en horario hábil	Variable e impredecible: se dimensiona con autoescalado
Disponibilidad	Horario hábil suele bastar; se puede acordar ventana de mantenimiento amplia	24/7 con ventana mínima: el ciudadano entra a cualquier hora
Riesgo principal	Abuso de privilegios y fuga interna de información	Ataques automatizados, denegación de servicio y suplantación

Arquitectura de Software

Cómo se conecta el funcionario desde la casa

VPN tradicional

El equipo del funcionario se conecta a la red corporativa y queda «dentro».

- + Conocida, soportada por todo, fácil de justificar.
- Da acceso a toda la red: si el equipo doméstico está comprometido, el problema entra completo.
- Concentra tráfico y puede saturarse.

Acceso Zero Trust

El funcionario no entra a la red: se le publica cada aplicación de forma individual, verificando identidad, doble factor y estado del equipo en cada acceso.

- + Sin acceso lateral a la red; permisos por aplicación.
- + Mejor experiencia y mejor trazabilidad.
- Requiere un servicio adicional y su costo.

Publicación con acceso condicional

La aplicación interna se publica en Internet detrás de un proxy con doble factor y reglas por país, dispositivo y horario.

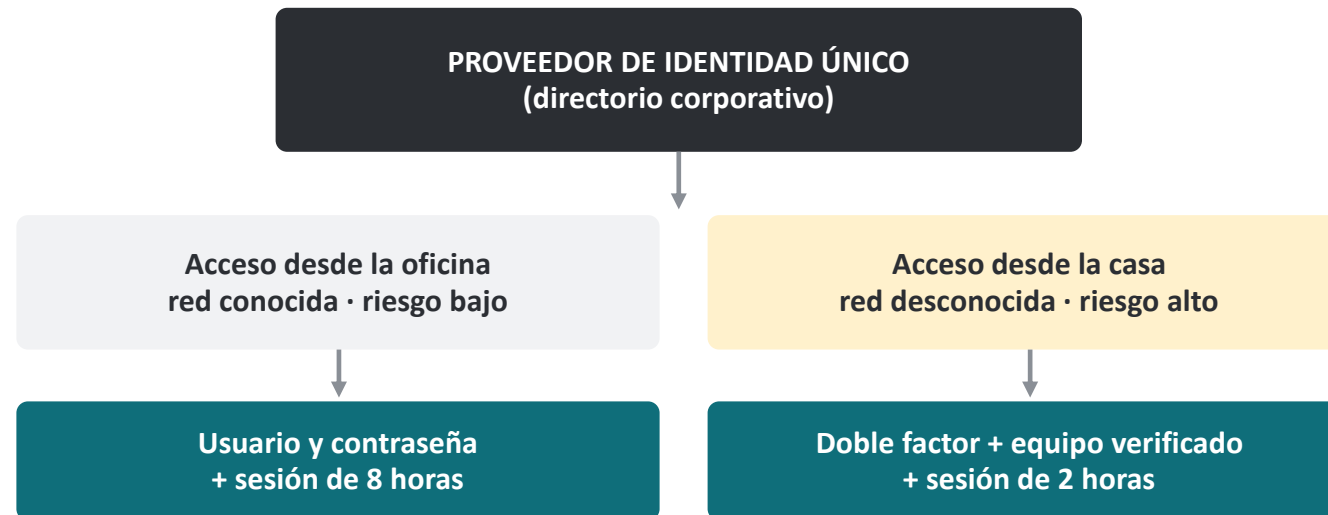
- + No requiere cliente instalado.
- Aumenta la superficie expuesta: exige WAF y monitoreo estricto.
- No apto para sistemas muy sensibles.

Común a las tres: doble factor obligatorio, equipo con antivirus gestionado y disco cifrado, sesión con expiración, y registro de cada acceso.
Sin eso, ninguna de las tres es segura.

Arquitectura de Software

Una identidad, permisos distintos

El mismo funcionario entra desde la oficina y desde la casa, y a veces también usa el portal público como ciudadano. Es una sola identidad con reglas distintas según el contexto.



Eso es acceso condicional: los permisos son los mismos, lo que cambia es la exigencia para obtenerlos. Y hay operaciones —aprobar un pago, exportar una base— que pueden exigir doble factor siempre, esté donde esté el funcionario.

Arquitectura de Software

Monitoreo, incidentes y cumplimiento

Centralización de registros

Los eventos de seguridad de todos los componentes van a un solo lugar, correlacionados y conservados por el período que exige la normativa.

Detección y alertas

Reglas que avisan ante accesos anómalos, intentos masivos de autenticación o exportaciones inusuales de datos.

Plan de respuesta

Quién hace qué, en qué orden y a quién se avisa. Escrito antes del incidente, no durante.

Notificación de brechas

La normativa vigente en Chile exige informar a la Agencia y a los titulares dentro de un plazo acotado: sin detección, ese plazo no se cumple.

Marco normativo aplicable en Chile:

Obligación	Qué exige	Consecuencia en la arquitectura
Registro de tratamiento	Documentar qué datos personales se tratan, para qué y con qué base legal.	Inventario de datos y trazabilidad por sistema.
Derechos del titular	Acceso, rectificación, cancelación y oposición.	Funciones de exportación y eliminación, no sólo de captura.

Ley N° 21.719, publicada el 13 de diciembre de 2024; entrada en plena vigencia el 1 de diciembre de 2026. Cite la norma en el informe según APA 7.ª ed.

Arquitectura de Software

Monitoreo, incidentes y cumplimiento · continuación 1

Obligación	Qué exige	Consecuencia en la arquitectura
Notificación de brechas	Informar a la Agencia y a los titulares en el plazo establecido.	Detección temprana, registro de auditoría y procedimiento definido.
Seguridad proporcional al riesgo	Medidas técnicas acordes al tipo de dato tratado.	Cifrado, control de acceso y minimización de datos.
Encargados de tratamiento	Acuerdo con los proveedores que traten datos por cuenta del responsable.	El proveedor de nube es un encargado: hay que declararlo y contratarlo.

Arquitectura de Software

Seguridad en la oferta · qué declarar

Ámbito	Qué declarar	Ejemplo de redacción
Exposición	Qué se publica a Internet y qué no.	Sólo el portal y la API pública, por el puerto 443, detrás de CDN y WAF.
Perímetro	Componentes y sus reglas.	Protección contra denegación de servicio; WAF con conjunto de reglas OWASP en modo bloqueo.
Segmentación	Zonas de red y tráfico permitido entre ellas.	Tres subredes: web, aplicación y datos; el tráfico sólo desciende un nivel.
Identidad	Fuente, factores y política de sesión.	Directorio corporativo con SSO; doble factor para acceso remoto y para administradores.
Acceso remoto	Mecanismo elegido y sus condiciones.	Acceso Zero Trust por aplicación, con verificación de equipo y sesión de 2 horas.
Datos	Cifrado, llaves, retención y datos en ambientes no productivos.	Cifrado en tránsito y reposo; ambientes no productivos con datos anonimizados.
Desarrollo	Controles dentro del ciclo de construcción.	Análisis de dependencias y de código en cada cambio; prueba de intrusión previa a producción.
Operación	Monitoreo, respuesta y evidencia.	Registros centralizados con retención de 12 meses; plan de respuesta con responsable designado.

Sección 7 · Seguridad

1

OWASP Top 10

Recorra los diez riesgos y verifique cuáles aplican a su solución. Es la referencia citable por excelencia en seguridad de aplicaciones.

2

Ley 21.719

Lea la ley de protección de datos personales y liste las obligaciones que su solución debe cumplir desde el diseño.

3

Confianza cero

Investigue el modelo Zero Trust y compárelo con la VPN tradicional para el escenario de teletrabajo de su caso.

4

Gestión de secretos

Pruebe un almacén de secretos y elimine toda credencial que hoy esté escrita en un archivo de configuración.

5

Modelado de amenazas

Aplique el método STRIDE a un módulo de su solución y documente las tres amenazas más probables con su mitigación.

08

SECCIÓN 8

Identidad y sesión de usuarios

- Autenticación, autorización y sesión: tres preguntas distintas que se resuelven distinto
- Usuarios propios, directorio corporativo (LDAP / Active Directory) y proveedor de identidad
- Protocolos: SAML 2.0, OAuth 2.0 y OpenID Connect — para qué sirve cada uno
- Keycloak, Entra ID, Cognito, Auth0 y ClaveÚnica: alternativas y cuándo elegirlas
- Sesión en el servidor o token autocontenido (JWT): cookies, expiración y revocación
- HTTPS, doble factor y qué declarar en la oferta

Arquitectura de Software

El inicio de sesión es arquitectura, no una pantalla

«El sistema tendrá login con usuario y contraseña» no es una decisión de arquitectura: es la ausencia de una. Detrás hay al menos seis decisiones con consecuencias técnicas y económicas.

¿Dónde viven los usuarios?

¿En su base de datos, en el directorio del mandante, en un proveedor externo, o en varios a la vez?

¿Cómo se prueba quién es?

Contraseña, doble factor, certificado, clave del Estado. Cada opción tiene un costo y un nivel de garantía.

¿Cómo se recuerda que ya entró?

Sesión en el servidor o token autocontenido. Esta decisión condiciona el balanceo y el escalamiento.

¿Qué puede hacer una vez dentro?

Roles, permisos, y dónde se decide: en la puerta de enlace o dentro de cada servicio.

¿Cómo se cierra el acceso?

Expiración, cierre de sesión, revocación y desvinculación de la persona. Lo que más se olvida.

¿Cuánto cuesta?

Un producto de identidad se licencia por usuario activo al mes, o se opera uno propio con su servidor y su gente.



Si no puede responder esas seis preguntas con nombres concretos, su propuesta tiene un vacío justo en el componente que usan el 100% de los usuarios.

Arquitectura de Software

Tres preguntas que no son la misma

Identificación ¿Quién dice ser?

El usuario declara una identidad: un nombre de usuario, un correo, un RUT.

No prueba nada todavía.

Analogía: decir su nombre en el mostrador.

Autenticación ¿Puede probarlo?

Presenta algo que sólo él tiene o sabe: contraseña, código temporal, llave física, certificado.

El resultado es sí o no.

Analogía: mostrar el pasaporte.

Autorización ¿Qué puede hacer?

Ya autenticado, qué recursos y operaciones tiene permitidos.

Depende del rol, del contexto y del dato.

Analogía: la tarjeta de embarque dice a qué avión sube y en qué asiento.

Y una cuarta, la que más se olvida: la **SESIÓN**. Autenticarse ocurre una vez; la sesión es lo que hace que el sistema siga sabiendo quién es usted en las siguientes doscientas peticiones, sin volver a pedirle la contraseña. Casi todos los problemas de seguridad de este bloque están en la sesión, no en el login.

Arquitectura de Software

El vocabulario mínimo

Credencial

Lo que se presenta para probar la identidad: contraseña, código, certificado, huella.

Factor

Categoría de credencial: algo que se sabe, algo que se tiene, algo que se es.

Proveedor de identidad (IdP)

El sistema que guarda los usuarios y los autentica. Keycloak, Entra ID, ClaveÚnica.

Aplicación cliente

El sistema que confía en el proveedor de identidad para saber quién entró.

Directorio

Base de datos jerárquica de personas, grupos y equipos de una organización. LDAP y Active Directory.

Federación

Acuerdo por el que una aplicación acepta identidades emitidas por otra organización.

Inicio de sesión único (SSO)

El usuario se autentica una vez y entra a varios sistemas sin repetir credenciales.

Token

Credencial temporal emitida tras autenticarse, que acompaña cada petición posterior.

Atributo o «claim»

Dato que el proveedor afirma sobre el usuario: correo, RUT, rol, unidad.

Sesión

El período durante el cual el sistema recuerda al usuario sin volver a pedirle credenciales.

Alcance («scope»)

Porción de permisos que una aplicación pide para actuar en nombre del usuario.

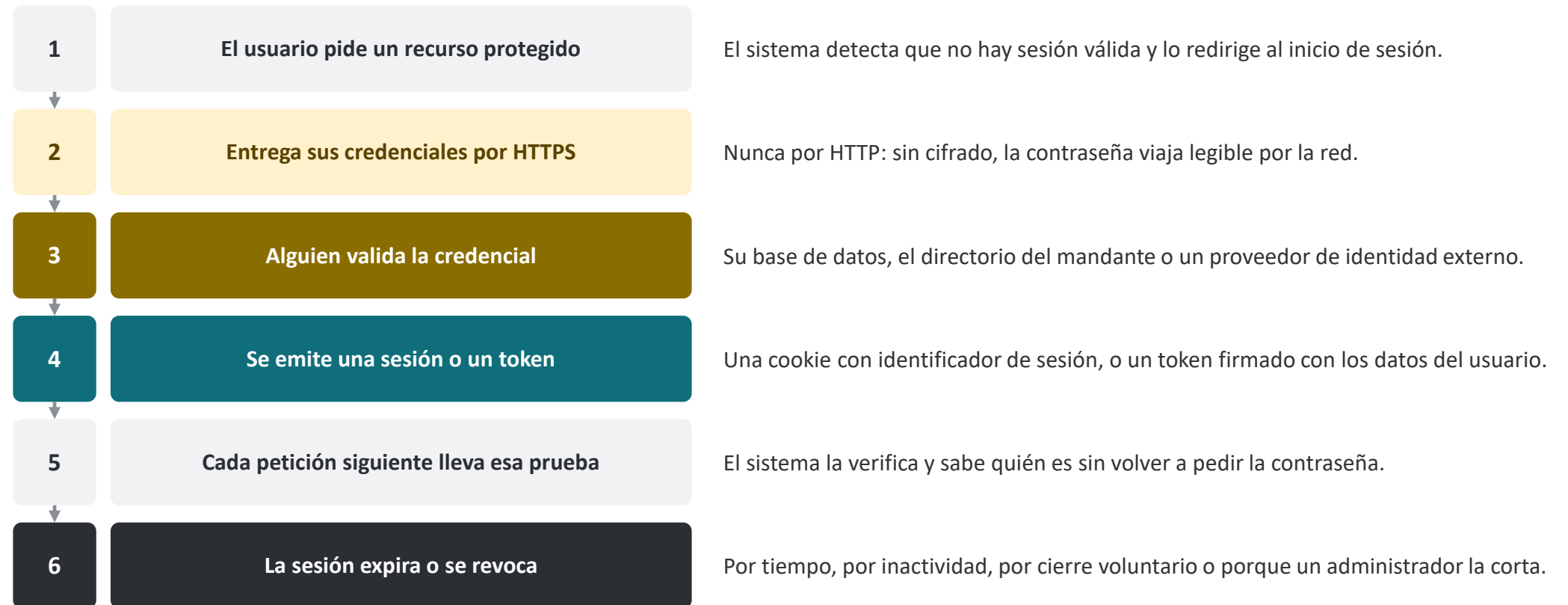
Revocación

Invaldar una credencial o una sesión antes de que expire por sí sola.

Arquitectura de Software

El flujo básico, paso a paso

Todo inicio de sesión tiene la misma forma. Lo que cambia entre alternativas es quién valida la credencial y qué se entrega después.



Arquitectura de Software

Alternativa 1 · usuarios propios en su base de datos

El sistema guarda su propia tabla de usuarios y valida la contraseña contra ella. Es la opción más simple y la que más responsabilidad transfiere a su equipo.

Cuándo tiene sentido

- Portal público de clientes finales que no pertenecen a ninguna organización.
- El mandante no tiene directorio corporativo ni proveedor de identidad.
- Sistema pequeño, autocontenido, con un solo frente de acceso.
- No hay presupuesto para un producto de identidad y el volumen de usuarios es bajo.
- Se necesita registro autónomo: el usuario se crea la cuenta solo.

Lo que asume al elegirla

- Guardar bien las contraseñas es su responsabilidad, y equivocarse es una brecha notificable.
- Hay que construir registro, recuperación, bloqueo, expiración y doble factor: no vienen gratis.
- Sin inicio de sesión único: el usuario tendrá una contraseña más que recordar.
- Cuando alguien deja la organización, hay que acordarse de desactivarlo en este sistema también.
- Auditoría y cumplimiento quedan enteramente de su lado.

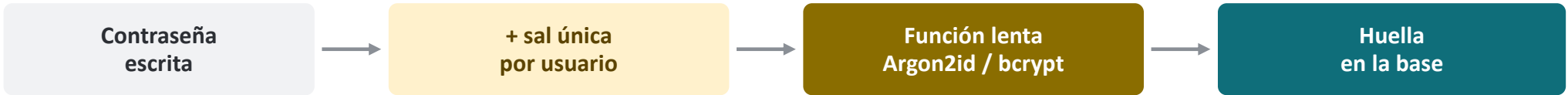


Estimación honesta: construir bien esta alternativa —registro, recuperación, bloqueo, doble factor, auditoría— cuesta varias semanas de desarrollo. Ese esfuerzo va en la carta Gantt y en el presupuesto.

Arquitectura de Software

Cómo se guarda una contraseña

Una contraseña no se guarda: se guarda el resultado de una función que no se puede deshacer, calculada con una sal distinta para cada usuario.



Regla	Por qué	Qué escribir en la oferta
Nunca en texto claro ni cifrado reversible	Si el sistema puede recuperar la contraseña, un atacante también	«Las contraseñas se almacenan con Argon2id»
Función de derivación lenta y con costo ajustable	Argon2id, scrypt o bcrypt. MD5 y SHA-1 se rompen en segundos	Algoritmo y parámetros de costo declarados
Sal única por usuario	Impide resolver todas las contraseñas iguales de una vez	Generada aleatoriamente, guardada junto a la huella
Longitud sobre complejidad	Una frase larga es más fuerte y más memorable que «Abc123!»	Mínimo 12 caracteres, sin reglas de composición
Sin caducidad periódica obligatoria	Forzar cambios mensuales empeora las contraseñas elegidas	Cambio sólo ante sospecha de compromiso

Referencia citable: NIST SP 800-63B, Digital Identity Guidelines. Es la fuente que respalda «longitud sobre complejidad» y «sin caducidad obligatoria».

Arquitectura de Software

Cómo se guarda una contraseña · continuación 1

Regla	Por qué	Qué escribir en la oferta
Contrastar con listas de contraseñas filtradas	El ataque más eficaz es probar contraseñas ya conocidas	Verificación contra listas públicas al registrar
Límite de intentos y retardo progresivo	Frena la prueba masiva sin bloquear cuentas legítimas para siempre	Política de bloqueo temporal declarada

Arquitectura de Software

Alternativa 2 · el directorio corporativo (LDAP)

LDAP —Protocolo Ligero de Acceso a Directorios— es la forma estándar de consultar un directorio de personas, grupos y equipos de una organización.

Qué es un directorio

Una base de datos jerárquica optimizada para leer: personas, grupos, unidades organizacionales y equipos, en forma de árbol.

Cómo se nombra a alguien

Con su nombre distinguido, la ruta completa en el árbol:
cn=jperez, ou=Finanzas,
dc=empresa, dc=cl

Implementaciones

Active Directory de Microsoft es la más extendida en empresas y servicios públicos. También OpenLDAP, 389 Directory Server y Samba AD.

Puertos y cifrado

389 sin cifrar y 636 para LDAPS. En una oferta sería sólo se propone LDAPS o StartTLS: nunca 389 en claro.

Grupos como roles

La pertenencia a grupos del directorio se traduce en roles de la aplicación. Así el mandante administra los accesos donde ya lo hace.

Lo que NO hace

No emite tokens, no da inicio de sesión único entre aplicaciones web y no tiene doble factor propio. Para eso hace falta una capa encima.



Ventaja decisiva para la oferta: la aplicación no guarda ni una sola contraseña. Cuando el mandante desvincula a un funcionario, éste pierde el acceso al sistema sin que nadie tenga que hacer nada.

Arquitectura de Software

Cómo autentica la aplicación contra LDAP

El truco es simple: la aplicación intenta conectarse al directorio usando las credenciales que escribió el usuario. Si el directorio la acepta, la contraseña es correcta.

- 1 El usuario escribe su nombre y contraseña en el formulario de la aplicación.
 - 2 La aplicación se conecta al directorio con una cuenta de servicio de sólo lectura.
 - 3 Busca a la persona por su nombre de usuario o correo y obtiene su nombre distinguido completo.
 - 4 Intenta una nueva conexión con ese nombre distinguido y la contraseña escrita por el usuario.
 - 5 Si el directorio acepta la conexión, la contraseña era correcta. Si la rechaza, no lo era.
 - 6 La aplicación lee los grupos de la persona y los traduce a sus propios roles.
 - 7 Recién ahora la aplicación crea su propia sesión local para el usuario.
- Todo el diálogo viaja por LDAPS. La cuenta de servicio se guarda en el almacén de secretos, nunca en el código.

Arquitectura de Software

LDAP · lo que hay que acordar con el mandante

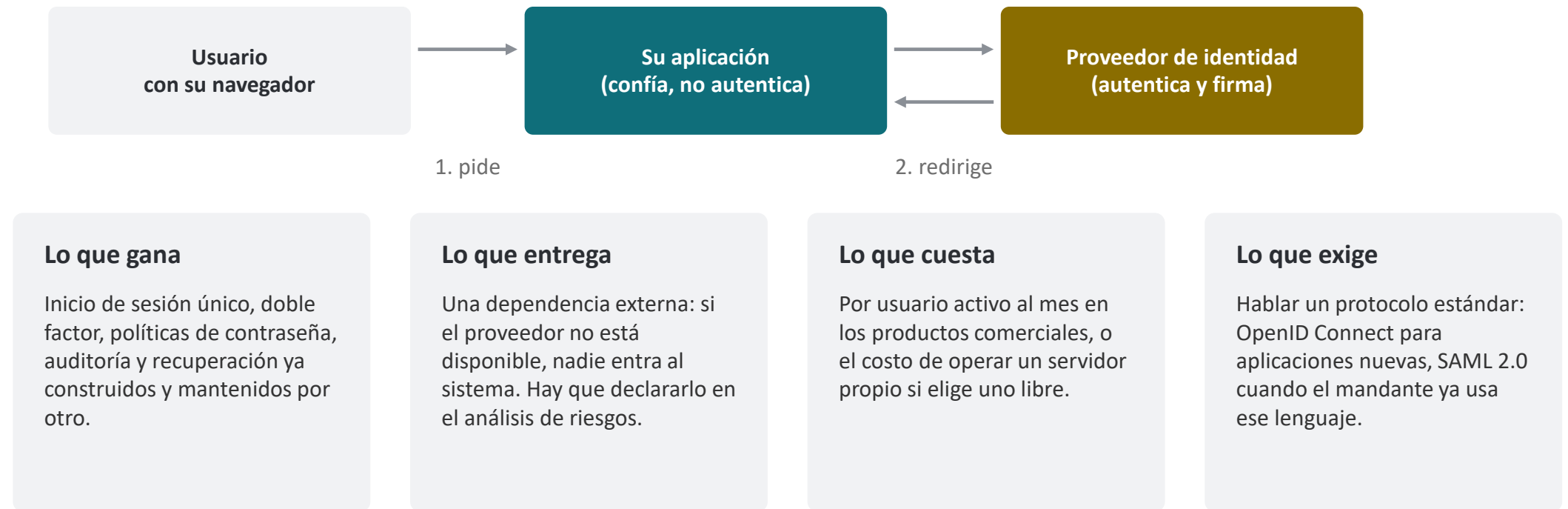
Punto a acordar	Qué preguntar	Riesgo si no se acuerda
Servidor y disponibilidad	¿Cuántos controladores de dominio hay y cuáles puede consultar el sistema?	Un solo servidor de directorio es punto único de falla del inicio de sesión
Cuenta de servicio	¿Quién la crea, con qué permisos y cómo se rota la contraseña?	Se termina usando una cuenta de administrador, que es un hallazgo grave
Atributo de búsqueda	¿Se busca por nombre de usuario, correo o RUT? ¿Es único?	Usuarios que no pueden entrar o, peor, que entran como otra persona
Grupos y roles	¿Qué grupos existen y quién los administra? ¿Se crean grupos nuevos para el sistema?	El mandante no puede otorgar accesos sin llamar al proveedor
Red y puertos	¿Hay conectividad desde donde corre el sistema hasta el directorio? ¿Puerto 636 abierto?	Si el sistema está en la nube, hay que resolver un túnel o un enlace privado
Certificado de LDAPS	¿Quién lo emite y cuándo vence? ¿Es de una autoridad interna?	El día que vence el certificado, nadie puede iniciar sesión
Usuarios externos	Los clientes finales no están en el directorio: ¿dónde viven?	Hay que sostener dos mecanismos de identidad en paralelo

Los dos últimos puntos son los que más proyectos atrasan: el certificado que vence sin aviso y el descubrimiento tardío de que hay usuarios fuera del directorio.

Arquitectura de Software

Alternativa 3 · un proveedor de identidad

La aplicación deja de preguntar contraseñas. Redirige al usuario a un proveedor de identidad, y confía en la respuesta firmada que recibe de vuelta.



Este es el modelo detrás de «Iniciar sesión con Google», de ClaveÚnica del Estado de Chile y de cualquier portal corporativo con inicio de sesión único.

Arquitectura de Software

SAML 2.0, OAuth 2.0 y OpenID Connect

Tres estándares que suenan parecido y resuelven cosas distintas. Confundirlos es el error de vocabulario más común en este tema.

	SAML 2.0	OAuth 2.0	OpenID Connect
Qué resuelve	Autenticación federada entre organizaciones	Autorización delegada: NO autentica	Autenticación sobre OAuth 2.0
Pregunta que responde	¿Quién es este usuario, según su organización?	¿Puede esta aplicación actuar en nombre del usuario?	¿Quién es este usuario, y con qué datos?
Formato	XML, con firma digital	Token opaco o JWT	Token de identidad en formato JWT
Año y madurez	2005, muy maduro y muy extendido en el sector público	2012, base de todo lo demás	2014, es el estándar para aplicaciones nuevas
Dónde se usa	Portales corporativos, universidades, servicios del Estado	Acceso de aplicaciones a APIs de terceros	Aplicaciones web y móviles modernas, APIs propias
Complejidad	Alta: XML, firmas, metadatos	Media	Media, con buenas bibliotecas para todo lenguaje
Qué elegir hoy	Sólo si el mandante ya lo exige	Como base, junto con OpenID Connect	Opción por defecto para un sistema nuevo

OAuth 2.0 responde «¿qué puede hacer?»; OpenID Connect agrega «¿y quién es?». Usarlo solo para entrar es un error conocido.

Arquitectura de Software

El flujo de OpenID Connect, en concreto

Flujo de código de autorización con PKCE: el recomendado hoy tanto para aplicaciones web como móviles.

1	La aplicación redirige el navegador al proveedor, con un código de verificación que sólo ella conoce.
2	El proveedor muestra su propia pantalla de inicio de sesión y pide doble factor si corresponde.
3	El usuario se autentica. Su contraseña nunca pasa por la aplicación.
4	El proveedor devuelve el navegador a la aplicación con un código de un solo uso, de vida muy corta.
5	La aplicación canjea ese código por tokens, desde su servidor y probando que es la misma que inició el flujo.
6	Recibe un token de identidad (quién es), uno de acceso (qué puede hacer) y uno de refresco (para renovar).
7	Verifica la firma del token contra las llaves públicas del proveedor y crea la sesión.
Flujos que ya no se deben usar: el implícito y el de contraseña directa. Si los ve en un ejemplo de Internet, el ejemplo está desactualizado.	

Arquitectura de Software

Fundamentos

Lógica

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

Keycloak

Proveedor de identidad de código abierto, respaldado por Red Hat. Habla OpenID Connect y SAML 2.0, se instala en un contenedor y no tiene costo de licencia.

Reinos («realms»)

Espacios de usuarios completamente separados. Uno para funcionarios y otro para clientes externos, en el mismo servidor.

Federación de usuarios

Se conecta al Active Directory o al LDAP del mandante y usa esos usuarios sin copiarlos.

Intermediación de identidad

Permite además entrar con Google, Microsoft o ClaveÚnica, y unifica todo en una sola identidad para la aplicación.

Doble factor incluido

Códigos temporales, correo, y llaves de seguridad o passkeys mediante WebAuthn. Configurable por rol.

Roles, grupos y políticas

Administración de permisos desde una consola web, sin tocar la aplicación ni desplegar de nuevo.

Personalización visual

Las pantallas de inicio de sesión se ajustan a la imagen del mandante mediante plantillas.

Lo que hay que cotizar igual	Detalle
Infraestructura	Al menos dos instancias tras un balanceador, más una base de datos PostgreSQL propia
Operación	Actualizaciones, respaldo del reino, monitoreo y certificados: es un sistema crítico más que operar
Puesta en marcha	Configuración de reinos, clientes, roles, federación y plantillas: semanas de trabajo, no horas
Soporte	Opcional, con Red Hat build of Keycloak, si el mandante exige respaldo formal de un fabricante

Arquitectura de Software

Alternativas de producto, comparadas

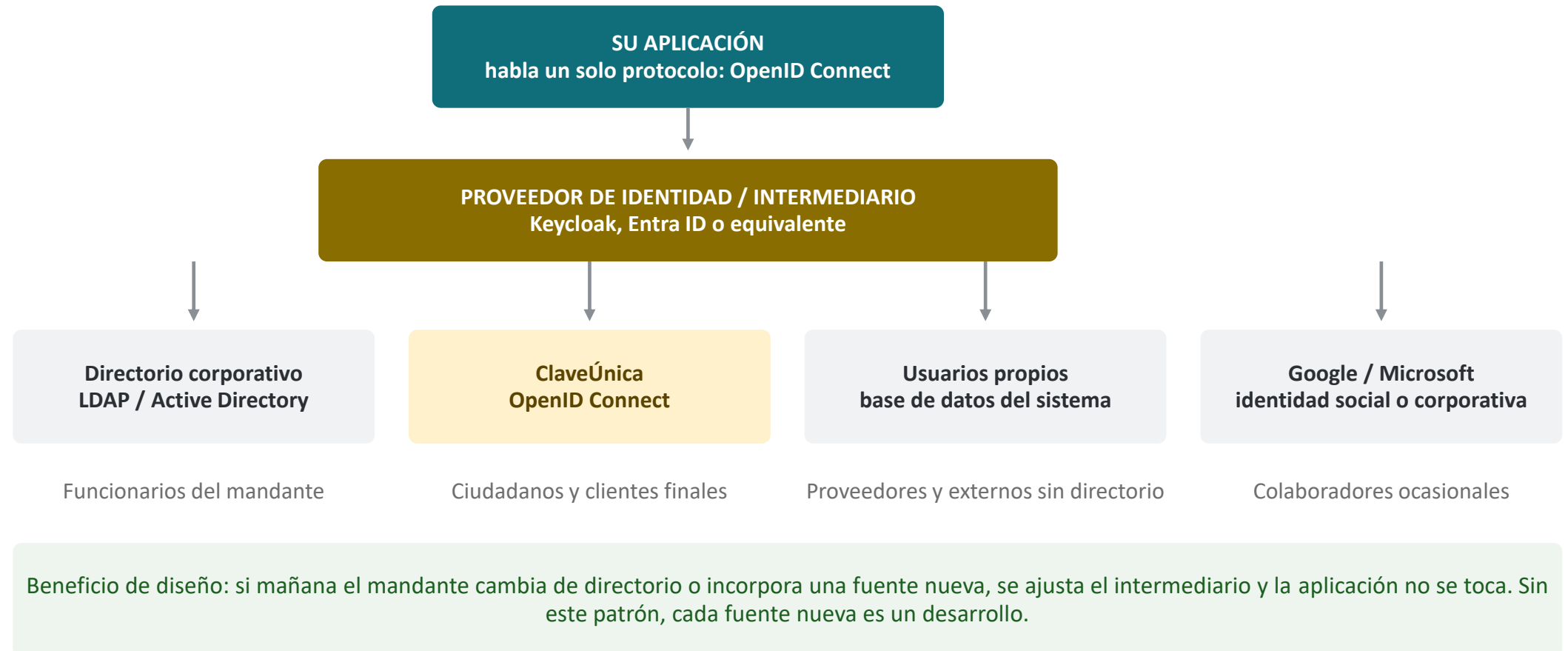
Producto	Tipo	Modelo de costo	Fortaleza	Cuándo elegirlo
Keycloak	Libre	Sin licencia; costo de operar	Completo, sin ataduras, se instala donde sea	On-premise o nube, sin presupuesto de licencias
Microsoft Entra ID	Servicio	Por usuario al mes, por niveles	Integración total con Microsoft 365 y Windows	El mandante ya vive en el ecosistema Microsoft
Auth0 / Okta	Servicio	Por usuario activo al mes	Muy rápido de implementar, excelente documentación	Startups y proyectos con plazo corto
Amazon Cognito	Servicio	Por usuario activo al mes, con capa gratuita	Integrado con el resto de AWS	La solución ya corre en AWS
Azure AD B2C / Entra External ID	Servicio	Por autenticación mensual	Pensado para clientes finales, no funcionarios	Portal público sobre Azure
Zitadel / Authentik / FusionAuth	Libre o mixto	Sin licencia o por volumen	Más livianos que Keycloak, despliegue simple	Equipos pequeños que quieren autoalojar
ClaveÚnica (Chile)	Estatal	Sin costo para el organismo	Identidad del Estado, ya la tiene el ciudadano	Trámites y servicios públicos a personas naturales

ClaveÚnica se integra mediante OpenID Connect y requiere solicitar la incorporación del servicio ante la División de Gobierno Digital: es un trámite con plazo, y ese plazo va en la carta Gantt.

Arquitectura de Software

Cómo conviven varias fuentes de identidad

Casi ningún caso tiene una sola fuente de usuarios. El patrón correcto es un intermediario: la aplicación habla con uno solo, y éste habla con todos.



Arquitectura de Software

La sesión: las dos familias

Autenticarse ocurre una vez. La sesión es lo que sostiene las siguientes doscientas peticiones. Hay dos maneras de hacerlo, y eligen cosas distintas.

Sesión en el servidor (cookie con identificador)

El servidor guarda el estado de la sesión y le entrega al navegador una cookie con un identificador opaco, sin significado.

En cada petición busca ese identificador en su almacén y recupera quién es el usuario.

El almacén suele ser Redis o la base de datos.

Token autocontenido (JWT firmado)

El servidor entrega un token que lleva dentro los datos del usuario, firmado digitalmente.

En cada petición verifica la firma: no necesita consultar nada ni recordar nada.

Es el modelo natural de las APIs y los microservicios.



Consecuencia física directa: con sesión en el servidor y varios nodos, o fija al usuario a un nodo —y pierde balanceo— o monta un almacén compartido, que es un componente más que cotizar y del que hay que asegurar la disponibilidad.

Arquitectura de Software

Sesión en servidor o token: cuál elegir

Criterio	Sesión en el servidor	Token autocontenido (JWT)
Dónde vive el estado	En el servidor o en un almacén compartido	En el propio token, en el cliente
Revocación inmediata	Trivial: se borra del almacén	Difícil: el token vale hasta que expira
Escalado horizontal	Requiere almacén compartido o sesión fija	Natural: cualquier nodo lo valida solo
Tamaño en cada petición	Pequeño: sólo un identificador	Mayor: viaja el contenido completo en cada llamada
Entre dominios y móviles	Incómodo: depende de cookies y dominios	Natural: es sólo una cabecera
Microservicios	Cada servicio debe consultar el almacén	Cada servicio verifica la firma sin consultar a nadie
Riesgo principal	Que el almacén de sesiones se caiga o se sature	Que un token robado sirva hasta que expire
Cuándo conviene	Portal web clásico, un solo frente, necesidad de corte inmediato	APIs, aplicaciones móviles, microservicios, varios frentes

La solución habitual es mixta: token de acceso de vida muy corta —cinco a quince minutos— más un token de refresco revocable. Se obtiene el escalado del token y el control de la sesión en servidor.

Arquitectura de Software

Anatomía de un token JWT

Un JWT son tres partes separadas por puntos, codificadas en base64. Codificado no es cifrado: cualquiera puede leer su contenido.

ENCABEZADO	Algoritmo de firma y qué llave se usó	{ "alg": "RS256", "kid": "a3f..." }
CONTENIDO	Los datos que el proveedor afirma del usuario	{ "sub": "12345", "roles": ["auditor"], "exp": 1735689600 }
FIRMA	Prueba criptográfica de que nadie lo modificó	Calculada por el proveedor con su llave privada

Regla de uso	Por qué
Nunca poner datos sensibles dentro	El contenido es legible por cualquiera que tenga el token
Expiración corta en el token de acceso	Es la única defensa real si el token es robado: entre 5 y 15 minutos
Firma asimétrica en sistemas distribuidos	Los servicios validan con la llave pública y no emiten tokens falsos
Verificar emisor, destinatario y expiración	Una firma válida no basta: hay que comprobar que el token era para usted
Rotación de llaves publicada	El proveedor publica sus llaves públicas y las rota sin romper a nadie

Arquitectura de Software

Token de acceso y token de refresco

Dos tokens con vidas distintas: uno corto para trabajar y uno largo y revocable para renovarlo sin molestar al usuario.

Token de acceso

- Vida muy corta: de 5 a 15 minutos.
- Acompaña cada petición a la API, en la cabecera de autorización.
- No se puede revocar: por eso dura poco.
- Contiene los permisos: el servicio lo verifica sin consultar a nadie.
- Si se filtra, el daño está acotado a esa ventana de minutos.

Token de refresco

- Vida larga: horas o días, según la política de sesión.
- Sólo se usa contra el proveedor de identidad, nunca contra la API.
- Se guarda en el servidor o en una cookie HttpOnly, nunca en el almacenamiento del navegador.
- Es revocable: cerrar sesión o desvincular a alguien lo invalida de inmediato.
- Rotación: cada uso emite uno nuevo y anula el anterior.
- Detección de reutilización: si se usa uno ya canjeado, se anula toda la cadena de sesión.



En su oferta escriba los dos números: duración del token de acceso y duración de la sesión. Son dos parámetros que el mandante puede exigir y que se revisan en una auditoría.

Arquitectura de Software

La cookie de sesión y sus marcas

La cookie que transporta la sesión debe llevar marcas explícitas. Cada una desactiva un ataque conocido.

Marca	Qué hace	Ataque que evita
Secure	La cookie sólo viaja por HTTPS	Robo de la sesión leyendo tráfico sin cifrar en una red pública
HttpOnly	El código JavaScript de la página no puede leerla	Robo del token mediante inyección de scripts en la página
SameSite=Lax o Strict	No se envía cuando la petición viene desde otro sitio	Peticiones falsificadas desde un sitio malicioso
Domain y Path acotados	Limita a qué servidores y rutas se envía	Exposición innecesaria de la sesión a subdominios de terceros
Max-Age o Expires	Fija cuándo caduca sola	Sesiones eternas en equipos compartidos
Prefijo __Host-	Obliga a Secure, sin Domain y con Path raíz	Que un subdominio comprometido escriba cookies para el dominio principal
Renovar el identificador al iniciar sesión	Se descarta el identificador previo y se emite uno nuevo	Fijación de sesión: que el atacante imponga un identificador conocido

Patrón recomendado hoy para aplicaciones de página única: el token nunca llega al navegador. Un intermediario propio —el patrón «servidor para el frontend»— lo guarda del lado del servidor y entrega sólo una cookie HttpOnly.

Arquitectura de Software

Cerrar sesión de verdad

Cerrar sesión no es borrar la cookie del navegador. Hay tres cierres distintos, y una arquitectura seria los distingue.

Cierre local

Se borra la cookie o el token del dispositivo.

El usuario deja de estar autenticado en esa aplicación y en ese equipo.

Es lo mínimo, y es lo único que hacen muchos sistemas.

Cierre en el proveedor

Se avisa al proveedor de identidad, que termina la sesión central.

Sin esto, volver a entrar es instantáneo: el proveedor lo reconoce y no vuelve a pedir nada.

Cierre único global

El proveedor avisa a todas las aplicaciones donde el usuario tenía sesión abierta, para que la cierren también.

Es lo que exige un entorno con inicio de sesión único real.

Situación	Qué debe pasar
El usuario cierra sesión	Se anula el token de refresco y la sesión del proveedor; el de acceso caduca solo en minutos
Un administrador desvincula a una persona	Se anulan todas sus sesiones activas en todos los dispositivos, de inmediato
Se detecta un robo de credenciales	Anulación masiva de sesiones y cambio forzado de contraseña
El usuario cambia su contraseña	Todas las sesiones anteriores se invalidan, salvo la actual

Arquitectura de Software

Fundamentos

HTTPS: el requisito previo de todo lo anterior

Lógica

Todo lo anterior asume que el canal está cifrado. Sin HTTPS, la contraseña, la cookie y el token viajan legibles por la red.

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

Qué es

HTTP dentro de un canal cifrado con TLS.
Hoy corresponde usar TLS 1.2 como mínimo y preferir TLS 1.3.

El certificado

Prueba que el servidor es quien dice ser. Lo emite una autoridad certificadora, tiene fecha de vencimiento y hay que renovarlo.

Dónde termina el cifrado

Normalmente en el balanceador o en el WAF. Desde ahí hacia adentro conviene volver a cifrar: es la práctica de confianza cero.

HSTS

Cabecera que le dice al navegador que jamás use HTTP con ese dominio. Evita el primer salto sin cifrar.

Redirección obligatoria

Todo lo que llegue por el puerto 80 se redirige a 443. No debe existir ninguna ruta accesible sin cifrar.

Entre servicios

TLS mutuo: además del servidor, el cliente presenta certificado. Es la forma de que un servicio pruebe su identidad a otro.

Decisión de la oferta	Alternativas	Costo típico
Emisor del certificado	Let's Encrypt automatizado · autoridad comercial · autoridad interna del mandante	Desde sin costo hasta cientos de dólares al año
Alcance	Un dominio · comodín para todos los subdominios · varios dominios	Aumenta con el alcance
Renovación	Automática mediante ACME, o manual con recordatorio en calendario	El certificado vencido es una de las caídas más frecuentes y más evitables

Arquitectura de Software

Doble factor: no todos valen lo mismo

Un segundo factor multiplica la seguridad del inicio de sesión. Pero hay una diferencia enorme entre unos y otros.

Factor	Cómo funciona	Fortaleza	Debilidad	Costo
Código por SMS	Se envía un número de seis dígitos al teléfono	Baja	Suplantación de la línea telefónica y reenvío por engaño	Por mensaje enviado: puede ser significativo
Código por correo	El mismo mecanismo, pero al correo del usuario	Baja	Si el correo está comprometido, el factor no aporta nada	Casi nulo
Aplicación de códigos temporales	Una aplicación genera un código que cambia cada 30 segundos	Media-alta	Todavía se puede pedir por engaño en un sitio falso	Sin costo
Notificación con número	Llega un aviso al teléfono y hay que confirmar un número mostrado en pantalla	Alta	Requiere una aplicación instalada y gestionada	Incluido en los productos de identidad
Llave de seguridad o passkey	Criptografía ligada al dominio del sitio: huella, rostro o llave física	Muy alta	Recuperación si el usuario pierde el dispositivo	Sin costo con passkeys; la llave física cuesta



Las llaves de seguridad y las passkeys son las únicas resistentes al engaño: están ligadas al dominio real y no funcionan en un sitio falso, aunque el usuario caiga.

Arquitectura de Software

Autorización: dónde se decide qué puede hacer

Por rol (RBAC)

Los permisos se asignan a roles y los roles a personas. Simple, entendible y suficiente para la mayoría de los casos del curso.

Por atributos (ABAC)

La decisión considera atributos: unidad del funcionario, monto de la operación, horario, canal de acceso.

Por relación

«Puede ver este expediente porque es el abogado asignado». Se decide sobre el dato concreto, no sobre el tipo de dato.

Alcances de la API

Cuando una aplicación actúa en nombre del usuario, el token limita qué porción de la API puede tocar.

Dónde se aplica cada control:

Punto de control	Qué decide ahí	Qué NO debe decidir ahí
Interfaz de usuario	Qué menús y botones se muestran	Nada de seguridad: es sólo comodidad, el cliente se puede manipular
Puerta de enlace de API	Que el token sea válido, no haya expirado y el rol tenga acceso al recurso	Reglas que dependen del dato concreto que se está pidiendo
Servicio de negocio	Si este usuario puede hacer esta operación sobre este registro en particular	Nada: aquí es donde la decisión es definitiva
Base de datos	Aislamiento entre inquilinos y cifrado de columnas sensibles	Lógica de permisos de negocio, que quedaría invisible y duplicada

Control de acceso roto es la vulnerabilidad número uno del OWASP Top 10. Casi siempre por confiar en una validación que sólo estaba en el navegador.

Arquitectura de Software

Identidad de los sistemas, no sólo de las personas

Cuando el sistema de facturación llama a la API de pagos no hay ninguna persona escribiendo una contraseña. Esa llamada también necesita identidad.

Mecanismo	Cómo funciona	Nivel	Cuándo usarlo
Llave de API	Una cadena secreta fija que acompaña cada llamada	Bajo	Integraciones simples de bajo riesgo; nunca para datos sensibles
Credenciales de cliente (OAuth 2.0)	El sistema pide un token al proveedor con su identificador y su secreto	Bueno	Estándar para llamadas entre sistemas propios y con terceros
TLS mutuo	Cada extremo presenta su certificado y ambos se verifican	Muy alto	Integraciones bancarias, salud, tráfico interno entre servicios
Identidad de carga de trabajo	La plataforma emite la identidad al contenedor; no hay secreto que guardar	Muy alto	Servicios en la nube o en Kubernetes que llaman a otros servicios
Usuario y contraseña compartidos	Una cuenta genérica usada por varios sistemas y personas	Inaceptable	Nunca: rompe la trazabilidad y nadie se atreve a rotarla

Regla transversal: ningún secreto vive en el código ni en el repositorio. Van a un almacén de secretos —Vault, Key Vault, Secrets Manager— con rotación programada y acceso auditado. Es un componente del diagrama y una línea del presupuesto.

Arquitectura de Software

Los errores que el evaluador busca

1

Contraseñas mal guardadas

Texto claro, cifrado reversible o funciones rápidas como MD5. Es la falla más grave y la más fácil de detectar.

2

Sin HTTPS en alguna ruta

Basta una página de inicio de sesión accesible por HTTP para que todo el esquema pierda sentido.

3

Token en el almacenamiento del navegador

Cualquier script inyectado en la página puede leerlo. Va en cookie con marca HttpOnly.

4

Sesiones sin expiración

Un usuario que nunca cierra sesión y un equipo compartido son una combinación previsible.

5

Permisos sólo en el frontend

El botón oculto no protege nada. La API tiene que negar la operación por sí sola.

6

Identificadores predecibles

Cambiar el número en la dirección y ver el registro de otra persona. Es el hallazgo clásico.

Arquitectura de Software

Los errores que el evaluador busca · continuación 1

7

Un solo rol «administrador»

Sin granularidad no hay mínimo privilegio, y toda la operación termina usando la cuenta más poderosa.

8

Sin registro de accesos

Si nadie puede reconstruir quién entró y qué hizo, no hay auditoría posible ni respuesta a incidentes.

9

Doble factor sólo opcional

Debe ser obligatorio para administradores y para todo acceso desde fuera de la red del mandante.

10

Sin proceso de desvinculación

Cuentas activas de personas que ya no trabajan ahí. Se detecta en la primera auditoría.

Arquitectura de Software

Cómo elegir para su caso

Si su caso es...	Fuente de identidad	Sesión	Doble factor
Sistema interno para funcionarios de una organización con directorio	LDAP / Active Directory, mediante un proveedor de identidad	Sesión en el servidor o token corto	Obligatorio fuera de la red interna
Portal público para ciudadanos, sector público chileno	ClaveÚnica mediante OpenID Connect	Token corto con refresco	Lo aporta ClaveÚnica
Portal de clientes de una empresa privada	Usuarios propios o servicio de identidad para clientes	Token corto con refresco	Opcional, obligatorio para operaciones sensibles
Solución mixta: funcionarios y clientes externos	Proveedor de identidad con dos reinos separados	Token, con políticas distintas por reino	Obligatorio para funcionarios
API para integrarse con sistemas de terceros	Credenciales de cliente OAuth 2.0 o TLS mutuo	Token de acceso, sin sesión	No aplica: es identidad de máquina
Aplicación móvil	OpenID Connect con código de autorización y PKCE	Token corto con refresco rotatorio	Biometría del dispositivo como segundo factor

Ninguna fila es obligatoria: es un punto de partida. Lo que el evaluador exige es que la elección esté justificada con el tipo de usuario y el nivel de riesgo del caso.

Arquitectura de Software

Qué declarar en la oferta

Elemento	Qué escribir	Dónde impacta
Fuente de identidad	Nombre del producto o servicio, y de dónde salen los usuarios	Diagrama lógico y físico; licencias
Protocolo	OpenID Connect, SAML 2.0 o autenticación directa contra LDAP	Esfuerzo de desarrollo e integración
Almacenamiento de contraseñas	Si aplica: algoritmo y parámetros; o bien «el sistema no almacena contraseñas»	Cumplimiento y seguridad
Mecanismo de sesión	Sesión en el servidor o token, con la duración de cada uno en minutos	Arquitectura física: almacén compartido o balanceo libre
Segundo factor	Qué factor, para qué perfiles y en qué circunstancias es obligatorio	Costo por mensaje o por licencia; experiencia de usuario
Modelo de roles	Lista de roles previstos y quién los administra después de la puesta en marcha	Alcance funcional y capacitación
Certificados	Emisor, alcance, mecanismo de renovación y responsable	Operación y riesgo de indisponibilidad
Ciclo de vida de cuentas	Alta, modificación, desvinculación y recertificación periódica	Procedimientos de operación
Registro de auditoría	Qué eventos se registran, dónde se guardan y por cuánto tiempo	Almacenamiento y cumplimiento normativo
Costo del componente	Licencias por usuario, infraestructura del proveedor y esfuerzo de integración	Directamente al flujo de caja

Estas diez filas son media página del informe y suelen valer más que tres páginas describiendo pantallas.

Sección 8 · Identidad y sesión de usuarios

1

OpenID Connect en la práctica

Levante Keycloak en un contenedor, cree un reino y conecte una aplicación de ejemplo. Observe los tokens que emite.

2

Anatomía de un token

Tome un JWT real de una sesión suya y decodifíquelo. Identifique emisor, destinatario, expiración y roles.

3

Directorios LDAP

Revise la estructura de un árbol LDAP y la diferencia entre buscar y «atarse». Pruebe con un servidor de ejemplo.

4

ClaveÚnica

Lea la documentación de integración de la División de Gobierno Digital y estime el plazo del trámite de incorporación.

5

Guía NIST SP 800-63B

Lea el capítulo de contraseñas: es la fuente que respalda «longitud sobre complejidad» y la ausencia de caducidad obligatoria.

6

OWASP

Revise las categorías de control de acceso roto y de fallas de identificación y autenticación, con sus ejemplos.

09

SECCIÓN 9

Entrega

- Por qué se necesitan varios ambientes y qué hace cada uno
- Desarrollo, QA, preproducción y producción: propósito, datos y dimensionamiento
- Qué es CI/CD y cómo se ve una canalización completa
- Cómo montar el ambiente de integración y entrega continua, paso a paso
- Estrategias de despliegue y qué comprometer en la oferta

Arquitectura de Software

Por qué varios ambientes

Un ambiente es una instalación completa de la solución —aplicación, base de datos y configuración— destinada a un propósito específico. Separarlos no es un lujo: es la única forma de cambiar sin romper.

Aislar el riesgo

Un error en desarrollo no puede afectar al usuario final. Sin separación, cada prueba es un riesgo para la operación.

Probar de verdad

Las pruebas necesitan un lugar donde se pueda romper todo, borrar datos y volver a empezar.

Validar con el mandante

La contraparte necesita ver y aprobar la funcionalidad antes de que llegue a producción.

Verificar el despliegue

El procedimiento de instalación se ensaya varias veces antes de ejecutarlo en producción.

Separar responsabilidades

Quien desarrolla no debería poder modificar producción por su cuenta. Es un control de auditoría.

Cumplir la normativa

Los datos personales de producción no pueden circular libremente por los ambientes de prueba.

Arquitectura de Software

Los cuatro ambientes

	DESARROLLO	QA / PRUEBAS	PREPRODUCCIÓN	PRODUCCIÓN
Para qué existe	Construir y probar mientras se programa	Verificar que lo construido cumple los requisitos	Ensayar el despliegue y validar con el mandante	Dar el servicio real
Quién lo usa	Equipo de desarrollo	Equipo de pruebas y de calidad	Mandante y equipo de operaciones	Usuarios finales
Datos	Sintéticos o mínimos	Sintéticos, con casos de prueba diseñados	Copia anonimizada de producción	Reales
Tamaño respecto de producción	10% a 20%	20% a 30%	Idéntico en configuración, menor en cantidad	100%
Disponibilidad	Horario hábil; se puede apagar de noche	Horario hábil	Cuando se necesita, más las ventanas de ensayo	La comprometida en el SLA
Quién despliega	Automático en cada cambio	Automático al aprobar	Automático con aprobación	Automático con aprobación formal
Frecuencia de despliegue	Varias veces al día	Diaria	Semanal o por hito	Según el plan de liberación
Monitoreo	Básico	Básico	Igual que producción, para validar el comportamiento	Completo, con alertas y turno

Arquitectura de Software

Cuánto cuestan los ambientes

Los ambientes no productivos suelen sumar entre un 30% y un 50% del gasto de infraestructura. Si no están en el presupuesto, aparecen igual —como sobrecosto.

Ambiente	Proporción del gasto de producción	Cómo se reduce
Desarrollo	10% – 20%	Instancias pequeñas; apagado automático fuera de horario hábil (ahorra hasta 65%)
QA / Pruebas	20% – 30%	Se levanta bajo demanda para la campaña de pruebas y se destruye al terminar
Preproducción	30% – 50%	Misma configuración que producción pero con menos nodos; se enciende para el ensayo del despliegue
Producción	100% (referencia)	Autoescalado y capacidad reservada para la carga base

Ambientes efímeros

Levantar el ambiente completo con infraestructura como código para una prueba y destruirlo después. Se paga sólo lo usado.

Apagado programado

Desarrollo y QA apagados de 20:00 a 8:00 y los fines de semana: son unas 128 horas de las 168 de la semana.

Etiquetado por ambiente

Cada recurso etiquetado con su ambiente permite saber cuánto cuesta cada uno y decidir dónde recortar.

Arquitectura de Software

Datos en ambientes no productivos

Copiar la base de producción a QA «para probar con datos reales» es la práctica más extendida y una de las más riesgosas.

Por qué es un problema

- Multiplica los lugares donde viven los datos personales.
- Los ambientes no productivos tienen menos controles, menos monitoreo y más gente con acceso.
- Es un tratamiento de datos sin la base legal que lo justifique.
- Ante una fuga, el incidente y la notificación son igual de obligatorios.

Qué hacer en cambio

- Datos sintéticos generados a partir del modelo, para desarrollo y QA.
- Copia anonimizada o seudonimizada para preproducción, cuando se necesita volumen realista.
- Subconjuntos: un 5% de los registros suele bastar para probar.
- Si se debe usar una copia, cifrada, con acceso restringido y con fecha de eliminación.

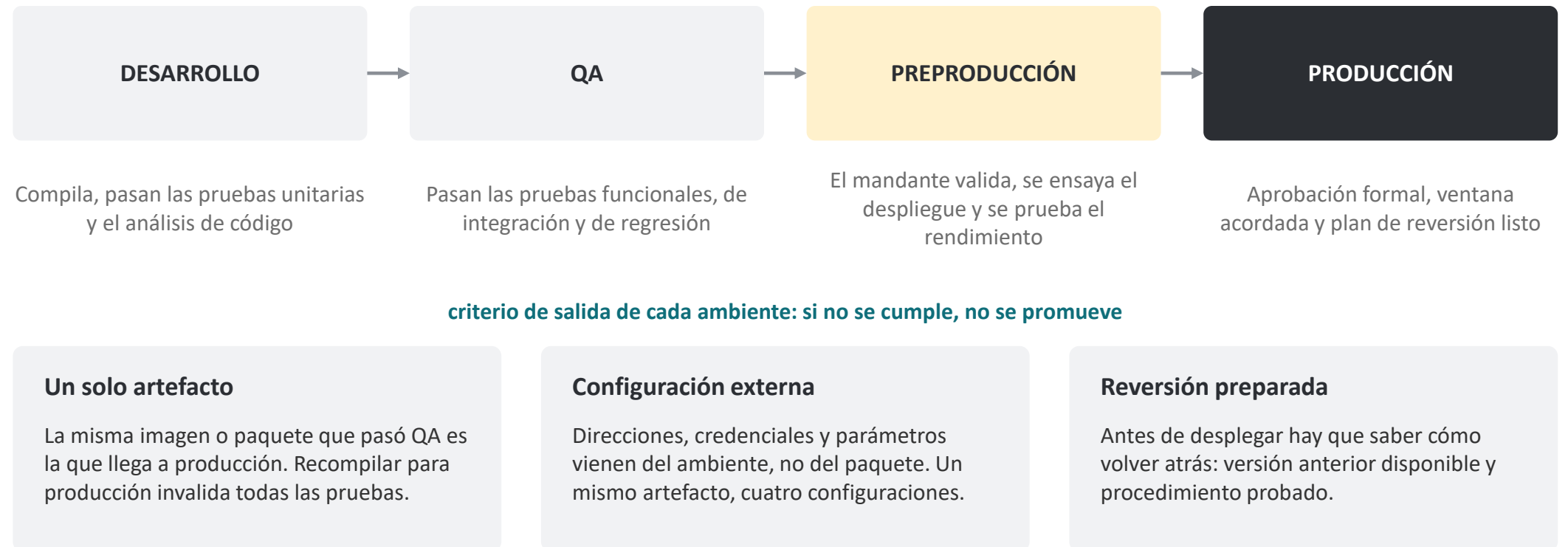


Compromiso concreto para la oferta: «los ambientes no productivos operan con datos anonimizados; ningún dato personal identificable sale de producción». Es barato, es verificable y responde directamente al criterio de restricciones legales.

Arquitectura de Software

La promoción entre ambientes

Principio fundamental: se construye una sola vez y ese mismo artefacto se promueve. Lo que cambia entre ambientes es la configuración, nunca el código.



Arquitectura de Software

Qué es CI/CD

Es la automatización del camino que va desde que alguien escribe una línea de código hasta que esa línea está funcionando para el usuario.

CI • Integración continua

Cada cambio se integra al código común y se verifica automáticamente: compila, pasan las pruebas, se analiza la calidad y la seguridad.

Objetivo: que un error se detecte en minutos y no en la etapa de pruebas.

Es el piso mínimo. Todo proyecto debería tenerlo.

CD • Entrega continua

Todo cambio que pasó las verificaciones queda listo para desplegarse, con un artefacto versionado y un procedimiento automatizado.

El paso a producción existe, pero lo autoriza una persona.

Objetivo: que publicar deje de ser un evento riesgoso.

CD • Despliegue continuo

Todo cambio que pasa las verificaciones llega a producción de forma automática, sin intervención humana.

Exige una batería de pruebas muy sólida y despliegue progresivo con reversión automática.

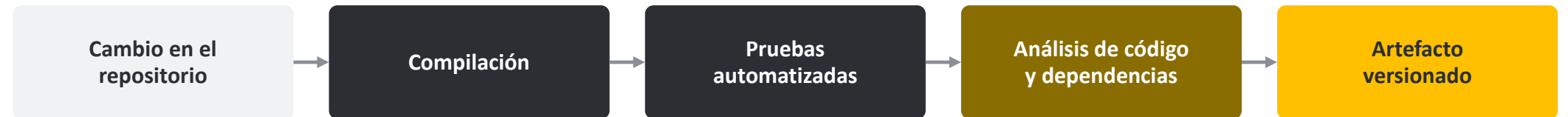
Raramente apropiado en un proyecto con contrato y ventanas de cambio.

Para su oferta, lo razonable es comprometer integración continua y entrega continua hasta preproducción, con aprobación formal para el paso a producción.

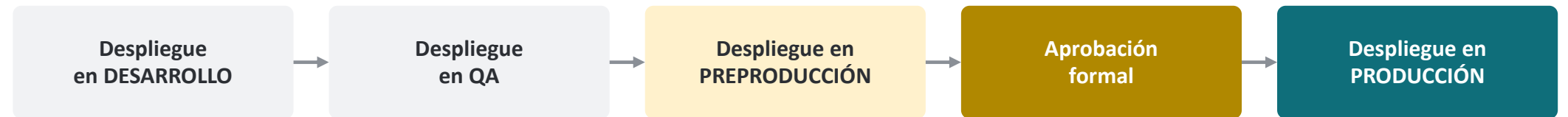
Arquitectura de Software

Anatomía de una canalización

INTEGRACIÓN CONTINUA · ocurre con cada cambio, en minutos



ENTREGA CONTINUA · el mismo artefacto recorre los ambientes



en cualquier etapa que falle, la canalización se detiene y avisa: nada avanza a medias

Todo queda registrado

Quién hizo el cambio, qué pruebas pasó, quién aprobó y a qué hora se desplegó. Es trazabilidad de auditoría, gratis.

Reversión en un paso

Si el despliegue sale mal, se vuelve a la versión anterior con el mismo mecanismo, en minutos.

El despliegue deja de dar miedo

Cuando publicar es un procedimiento automático y ensayado, se publica seguido y en cambios pequeños: menos riesgo por vez.

Arquitectura de Software

Cómo montar el ambiente de CI/CD

Ocho pasos, en orden. Los cuatro primeros se hacen en días y ya entregan la mayor parte del beneficio.

1

Repositorio y estrategia de ramas

Un repositorio por componente, con una rama principal siempre desplegable y ramas cortas por cambio. Nadie escribe directo en la principal.

2

Servidor de canalización

El servicio que ejecuta la automatización, integrado al repositorio. Puede ser el del propio proveedor del repositorio.

3

Construcción automatizada

Un comando único que compila y produce el artefacto —imagen de contenedor o paquete— de forma reproducible.

4

Pruebas en la canalización

Unitarias siempre; de integración sobre servicios simulados; un umbral mínimo de cobertura acordado con el equipo.

5

Análisis de calidad y seguridad

Revisión estática del código, análisis de dependencias con vulnerabilidades conocidas y escaneo de la imagen.

6

Almacén de artefactos

Registro privado donde se publica cada artefacto con su versión inmutable. Es la fuente de lo que se despliega.

7

Infraestructura como código

Los ambientes se describen en archivos versionados, de modo que DEV, QA, PREPROD y PROD sean iguales salvo en tamaño.

8

Despliegue automatizado con aprobación

Un mismo procedimiento para los cuatro ambientes, con compuerta de aprobación antes de producción y reversión preparada.

Arquitectura de Software

Estrategias de despliegue

Estrategia	Cómo funciona	Interrupción	Costo	Cuándo usarla
Recrear	Se detiene la versión antigua y se levanta la nueva.	Sí, hay corte	Mínimo	Sistemas internos con ventana acordada
Progresivo (rolling)	Se reemplazan las instancias de a poco, unas pocas por vez.	No	Bajo	El caso general; por defecto en el orquestador
Azul – verde	Se levanta el entorno nuevo completo en paralelo y se conmuta el tráfico.	No	Alto: doble infraestructura	Cambios grandes con reversión en segundos
Canario	Recibe una fracción del tráfico y se amplía si las métricas se mantienen.	No	Medio	Servicios de alto volumen y cambios de riesgo
Interruptores de función	El código nuevo se despliega apagado y se enciende por configuración.	No	Bajo	Para separar despliegue y puesta en marcha

En la oferta, declarar la estrategia elegida y el tiempo de reversión es lo que convierte «desplegaremos sin interrupción» en un compromiso verificable.

Arquitectura de Software

Configuración y secretos por ambiente

El mismo artefacto en cuatro ambientes significa que todo lo que cambia entre ellos vive fuera del artefacto.

Configuración por ambiente

Direcciones, tamaños, umbrales y parámetros de negocio, versionados en el repositorio de configuración —nunca dentro del código.

Secretos aparte

Contraseñas, llaves y certificados en un almacén de secretos, inyectados en el arranque, con acceso auditado y rotación definida.

Credenciales distintas

Cada ambiente tiene sus propias credenciales. Si QA usa las de producción, un error en QA es un error en producción.

Separación de red

El ambiente de QA no debe poder alcanzar la base de datos de producción. Es una regla de firewall, no una promesa.

Nada de secretos en el repositorio

Análisis automático que rechaza el cambio si detecta una credencial. Una llave publicada por error se considera comprometida.

Terceros en modo prueba

Los ambientes no productivos apuntan a los sandbox de los proveedores externos, jamás a sus interfaces productivas.

Incidente clásico: el ambiente de pruebas apuntando a la base de datos de producción. Ocurre por configuración compartida y termina con datos reales modificados por una prueba automatizada.

Arquitectura de Software

Qué comprometer en la oferta

Compromiso	Redacción tipo	Qué demuestra
Ambientes	Cuatro ambientes: desarrollo, QA, preproducción y producción, aprovisionados como código.	Que el proyecto tiene disciplina de entrega y que los ambientes están costeados.
Datos no productivos	Los ambientes no productivos operan con datos anonimizados o sintéticos.	Cumplimiento normativo y control del riesgo de fuga.
Integración continua	Cada cambio se compila, se prueba y se analiza automáticamente antes de integrarse.	Calidad verificable y detección temprana de defectos.
Cobertura de pruebas	Umbral mínimo de cobertura acordado, verificado en la canalización.	Un compromiso de calidad medible, no una declaración de intenciones.
Despliegue	Procedimiento automatizado e idéntico para los cuatro ambientes, con aprobación formal para producción.	Que el paso a producción es repetible y auditable.
Reversión	Estrategia progresiva con reversión a la versión anterior en menos de N minutos.	Que un despliegue fallido no se transforma en una caída prolongada.
Trazabilidad	Registro de qué se desplegó, quién lo aprobó y cuándo, conservado durante todo el contrato.	Evidencia de auditoría y control de cambios.
Métricas de entrega	Informe periódico de frecuencia de despliegue, tiempo de entrega, tasa de fallo y tiempo de recuperación.	Gestión basada en datos y transparencia con el mandante.

Sección 9 · Entrega

1

Monte una canalización

Cree un repositorio y configure integración continua: compilar, probar y publicar un artefacto en cada cambio. Se hace en una tarde.

2

Infraestructura como código

Pruebe una herramienta de aprovisionamiento declarativo y levante y destruya un ambiente completo con un comando.

3

Datos de prueba

Investigue técnicas de anonimización y generación de datos sintéticos para no copiar producción a los ambientes de prueba.

4

Estrategias de despliegue

Profundice en azul-verde, canario e interruptores de función. Elija una y defina su tiempo de reversión.

5

Métricas DORA

Estudie las cuatro métricas de entrega —frecuencia, tiempo de entrega, tasa de fallo y tiempo de recuperación— y decida cuáles comprometer.

10

SECCIÓN 10

Tendencias

- Contenedores, orquestación e ingeniería de plataforma
- Infraestructura como código, GitOps y entrega continua
- Observabilidad, malla de servicios y computación en el borde
- Arquitecturas de datos e inteligencia artificial en la solución
- FinOps, sostenibilidad y cómo tratar la innovación sin caer en la moda

Arquitectura de Software

Mapa de tendencias

No hay que usarlas todas. Hay que conocerlas para poder elegir con criterio —y para poder responder si el evaluador pregunta.

Contenedores y orquestación

Empaquetado portátil y coordinación automática de decenas de servicios.

Ingeniería de plataforma

Una plataforma interna que estandariza cómo se despliega y se opera.

Infraestructura como código

La infraestructura se define en archivos versionados, no a mano.

Observabilidad

Métricas, registros y trazas unificados para entender sistemas distribuidos.

Computación en el borde

Procesar cerca de donde se generan los datos, no en un solo centro.

Datos en tiempo real

Flujos continuos de eventos en vez de procesos por lotes nocturnos.

Inteligencia artificial

Modelos de lenguaje, recuperación aumentada y agentes dentro del producto.

FinOps y sostenibilidad

Gobernar el costo y el consumo energético como atributos de diseño.

Arquitectura de Software

Contenedores y orquestación

Los contenedores dejaron de ser una novedad: son la unidad de despliegue estándar. La discusión actual ya no es si usarlos, sino cuánta complejidad de orquestación se justifica.

Lo que se consolidó

- La imagen del contenedor como artefacto único que viaja entre ambientes.
- Los servicios gestionados de orquestación, que evitan administrar el clúster.
- El despliegue progresivo con reversión automática como práctica normal.
- El registro de imágenes con análisis de vulnerabilidades integrado.

La discusión abierta

- Para muchas soluciones, un orquestador completo es excesivo: bastan contenedores gestionados.
- La complejidad operativa se subestima de forma sistemática.
- Alternativas más simples ganaron terreno para cargas pequeñas y medianas.
- La pregunta correcta: ¿cuántos servicios y cuántos despliegues por semana tendré realmente?

Para su oferta: si propone orquestación, declare si el clúster es gestionado por el proveedor o por su equipo. Cambia el presupuesto de horas.

Arquitectura de Software

Ingeniería de plataforma

En vez de que cada equipo resuelva por su cuenta el despliegue, el monitoreo y la seguridad, una plataforma interna ofrece un camino estándar, documentado y automatizado.

Camino pavimentado

Una forma recomendada y automatizada de construir y desplegar, que resuelve por defecto seguridad, registro y monitoreo.

Autoservicio

El equipo de desarrollo crea sus ambientes sin abrir un ticket ni esperar a operaciones.

Plantillas

Servicios nuevos que nacen ya con pruebas, canalización de despliegue y observabilidad configuradas.

Gobierno incorporado

Las políticas de seguridad y de costo se aplican en la plataforma, no en la buena voluntad de cada equipo.

Traducción para su propuesta: aunque no construyan una plataforma, sí pueden comprometer un procedimiento único y automatizado de despliegue. Eso reduce riesgo y es un diferenciador creíble frente a competidores que despliegan a mano.

Arquitectura de Software

Infraestructura como código y GitOps

Infraestructura como código

- Servidores, redes y reglas de seguridad se describen en archivos de texto versionados.
- El ambiente se crea y se destruye con un comando, siempre igual.
- Desaparece la diferencia entre desarrollo, pruebas y producción.
- El cambio de infraestructura pasa por revisión, igual que el código.
- La documentación deja de desactualizarse: el código es la documentación.

GitOps

- El repositorio es la única fuente de verdad del estado deseado.
- Un agente compara continuamente lo desplegado con lo declarado y corrige la diferencia.
- Cada cambio queda registrado, con autor y motivo: auditoría automática.
- Volver atrás es revertir un cambio en el repositorio.
- Nadie modifica producción a mano.

Argumento para la oferta: «El ambiente completo está descrito como código; ante un desastre se reconstruye en menos de X horas y ese procedimiento se prueba trimestralmente». Es un compromiso de continuidad verificable, no una promesa.

Arquitectura de Software

DevOps, DevSecOps y entrega continua

Una arquitectura moderna sin automatización de despliegue no es moderna: es frágil.

Integración continua

Cada cambio se integra y se prueba automáticamente. Los errores aparecen en minutos, no en la etapa de pruebas.

Entrega continua

Cada versión aprobada puede desplegarse con un botón. Publicar deja de ser un evento riesgoso.

Seguridad en la canalización

Análisis de dependencias, revisión de código y escaneo de imágenes automatizados en cada cambio.

Pruebas automatizadas

Unitarias, de integración y de carga, ejecutadas sin intervención antes de cada publicación.

Despliegue progresivo

La versión nueva se libera a un porcentaje del tráfico y se revierte automáticamente si empeoran las métricas.

Cultura de postmortem

Los incidentes se analizan sin buscar culpables y producen mejoras concretas en el sistema.

En el presupuesto de la oferta esto aparece como horas de ingeniería y herramientas. Omitirlo no lo hace gratis: lo transforma en sobre costo durante la operación.

Arquitectura de Software

Observabilidad, malla de servicios y eBPF

Observabilidad unificada

Métricas, registros y trazas correlacionados bajo un estándar abierto de instrumentación, de modo que la aplicación no quede amarrada a una herramienta.

Permite responder preguntas que no se anticiparon al construir el sistema.

Malla de servicios

Una capa de red que resuelve fuera del código el cifrado entre servicios, los reintentos, los tiempos límite, el enrutamiento y la telemetría.

Muy potente y muy pesada: se justifica cuando hay muchos servicios, no cuando hay cinco.

eBPF

Tecnología que permite observar y controlar el tráfico y el comportamiento del sistema desde el núcleo del sistema operativo, sin modificar la aplicación.

Está detrás de las herramientas de red, seguridad y observabilidad más recientes.

Criterio de proporción: la observabilidad siempre se justifica —sin ella no hay SLA. La malla de servicios y eBPF, en cambio, sólo aparecen en soluciones con muchos servicios y equipos dedicados a operarlas.

Arquitectura de Software

Computación en el borde

En vez de enviar todos los datos a un centro de procesamiento, parte del cómputo ocurre cerca de donde se generan.

Latencia

Procesar localmente evita el viaje de ida y vuelta. Crítico cuando la respuesta debe ser inmediata.

Ancho de banda

Se envía sólo el resultado y no el flujo completo de datos. Baja el costo de transmisión.

Continuidad

La operación sigue funcionando aunque se caiga el enlace con el centro de datos.

Privacidad

El dato sensible puede procesarse en el sitio y salir sólo agregado o anonimizado.

Casos donde aparece con naturalidad:

- Sucursales, faenas mineras o instalaciones con conectividad intermitente.
- Sensores industriales y control de procesos que no toleran latencia.
- Video y visión por computador, donde transmitir todo el flujo es inviable.
- Puntos de venta y logística en terreno.

Arquitectura de Software

Datos en tiempo real

El modelo tradicional · por lotes

- Los datos se acumulan y se procesan de noche.
- El reporte de hoy refleja lo que pasó ayer.
- Simple de construir y de operar.
- Suficiente para contabilidad, cierres y reportes periódicos.

El modelo de flujo · streaming

- Cada hecho de negocio se publica como evento en el momento en que ocurre.
- Los tableros y las alertas reflejan el estado actual.
- Habilita detección de fraude, alertas operativas y personalización inmediata.
- Más complejo: exige orden, reintentos y manejo de eventos duplicados.

Innovación acotada y defendible: no convierta todo el sistema a tiempo real. Elija un caso de uso donde la inmediatez genere valor medible para el mandante —una alerta operativa, un tablero de gestión— y proponga sólo eso. Es más creíble y mucho más barato.

Arquitectura de Software

Arquitecturas de datos

Almacén de datos

Datos estructurados, limpios y modelados para reportes y análisis de negocio.

Fortaleza: consultas rápidas y consistentes.

Límite: rígido y costoso de cambiar.

Lago de datos

Repositorio que guarda datos en bruto, de cualquier formato, a bajo costo.

Fortaleza: flexibilidad y volumen.

Límite: sin gobierno se convierte en un pantano de datos.

Lakehouse

Combina el almacenamiento barato del lago con las garantías de consistencia del almacén.

Fortaleza: una sola plataforma para reportes y analítica avanzada.

Malla de datos

Enfoque organizacional: cada dominio de negocio publica sus datos como un producto, con dueño y calidad comprometida.

Más un modelo de gobierno que una tecnología.

Para la mayoría de los casos del curso, una base de datos transaccional bien modelada más una vista de lectura para reportes es la respuesta correcta y honesta.

Arquitectura de Software

Inteligencia artificial en la arquitectura

Incorporar IA cambia la arquitectura: aparecen componentes, costos y riesgos que antes no existían.

Modelo como servicio

Se consume un modelo de lenguaje por API. No hay que entrenar nada, pero se paga por uso y los datos salen de la organización.

Recuperación aumentada (RAG)

El modelo responde apoyándose en los documentos de la organización: se buscan los fragmentos relevantes y se le entregan como contexto.

Base de datos vectorial

Almacén especializado que permite buscar por significado y no por palabra exacta. Es el componente nuevo en el diagrama.

Agentes

Modelos que encadenan pasos y ejecutan acciones sobre otros sistemas. Requieren límites, permisos y registro de todo lo que hacen.

Modelos propios

Entrenar o ajustar un modelo con datos de la organización. Alto costo, exige datos de calidad y perfiles especializados.

IA en el desarrollo

Asistentes que generan código y pruebas. Cambia la productividad estimada del equipo, pero exige revisión humana.

Si su propuesta incluye IA, el diagrama debe mostrarla: dónde está el modelo, de dónde salen los datos que lo alimentan y qué pasa cuando el modelo no está disponible.

Arquitectura de Software

IA · lo que cambia en costos, riesgos y ética

Costos nuevos

- Cobro por volumen procesado: el costo crece con el uso, no con el número de servidores.
- Latencia mayor que una consulta tradicional: hay que diseñar la experiencia con eso en mente.
- Cómputo especializado si se ejecuta el modelo internamente: es caro y escaso.
- Curaduría y actualización permanente de la base de conocimiento.

Riesgos y responsabilidades

- Respuestas incorrectas presentadas con seguridad: exige verificación y trazabilidad.
- Datos personales o confidenciales enviados a un tercero: hay que declararlo y contratarlo.
- Sesgos en las respuestas y decisiones que afectan a personas.
- Dependencia de un proveedor y de un modelo que puede cambiar sin aviso.
- Quién responde ante un error del sistema: es una pregunta contractual, no técnica.



Si incorpora IA, declare en la oferta qué decisiones toma el modelo, cuáles quedan siempre en manos de una persona y cómo se audita lo que el sistema respondió.

Arquitectura de Software

MLOps y el ciclo de vida de los modelos

Un modelo no es un entregable que se instala una vez: se degrada con el tiempo porque la realidad cambia. Operarlo es parte de la arquitectura.



Si su propuesta incluye un modelo propio, el reentrenamiento y el monitoreo son ítems de operación anual, no una tarea del proyecto inicial.

Arquitectura de Software

FinOps · el costo como atributo de diseño

En la nube, cada decisión de arquitectura tiene un precio por hora. FinOps es la práctica de gobernar ese gasto con la misma disciplina con que se gobierna el rendimiento.

Visibilidad

Etiquetar cada recurso por proyecto, ambiente y responsable. Sin etiquetas no hay control posible.

Asignación

Saber cuánto cuesta cada módulo o cada cliente. Permite decidir qué se optimiza primero.

Optimización

Ajustar el tamaño de las instancias, apagar ambientes fuera de horario, usar capacidad reservada donde la carga es estable.

Previsión

Estimar el gasto del próximo período y alertar antes de superarlo, no después de la factura.

Diferenciador concreto para la oferta: comprometer un informe mensual de consumo y un tope de gasto acordado con el mandante. Cuesta poco ofrecerlo y demuestra que entienden que la nube es un costo operacional que hay que administrar.

Arquitectura de Software

Sostenibilidad y eficiencia energética

El impacto ambiental de una solución informática es una restricción del contexto que el diseño puede reducir. Y en varias bases de licitación ya es un criterio evaluado.

Eficiencia del cómputo

Menos recursos ociosos, autoescalado y apagado de ambientes fuera de horario reducen consumo y costo al mismo tiempo.

Elección de región

Los centros de datos difieren en su matriz energética y en su eficiencia. Es un criterio declarable en la decisión.

Ciclo de vida del hardware

En on-premise: vida útil, reciclaje y disposición final del equipamiento reemplazado.

Datos que se guardan

Retener todo para siempre consume energía y dinero. Una política de retención es también una medida ambiental.

Eficiencia del software

Código y consultas eficientes reducen el hardware necesario. Optimizar es la medida más barata.

Cómo se declara

Estime el consumo asociado a la solución y señale las medidas adoptadas. No hace falta ser exacto: hace falta ser explícito.

Arquitectura de Software

Otras corrientes que conviene conocer

Bajo código / sin código

Construir aplicaciones con configuración visual en vez de programación. Rápido para procesos internos; limitado y con dependencia fuerte del proveedor.

API como producto

Las interfaces se diseñan, documentan, versionan y se les mide adopción como si fueran un producto con clientes.

Arquitectura componible

Armar la solución integrando servicios especializados de terceros —pagos, identidad, notificaciones— en vez de construir todo.

WebAssembly

Formato que permite ejecutar código de forma segura y muy liviana, tanto en el navegador como en el servidor y en el borde.

Arquitectura sin base de datos propia

Servicios gestionados que eliminan la administración del motor: menos operación, más costo variable y más dependencia.

Ingeniería del caos

Provocar fallas controladas en producción para verificar que los mecanismos de resiliencia realmente funcionan.

Arquitectura de Software

Cómo tratar la innovación en su propuesta

La innovación es un ítem evaluado de su propuesta. Pero una tecnología sin justificación resta puntaje en vez de sumarlo.

Innovación que suma

- Resuelve un problema declarado en las bases.
- Está acotada: se aplica a un módulo, no a todo el sistema.
- Tiene costo estimado en el flujo de caja.
- Tiene un plan alternativo si no funciona.
- Está respaldada por una referencia verificable, citada en norma APA.

Innovación que resta

- Aparece porque «está de moda» o porque suena bien en la presentación.
- Abarca todo el sistema y multiplica el riesgo del proyecto.
- No tiene costo asociado en la oferta económica.
- Nadie del equipo puede explicar cómo se opera.
- No resiste la primera pregunta del evaluador.



Prueba de fuego: si le preguntan «¿qué pasa si sacamos esta tecnología de su propuesta?» y la respuesta es «nada importante», entonces no es innovación: es adorno.

Sección 10 - Tendencias

1

Elija sólo una

Tome una tendencia de esta sección, investiguella a fondo y prepare una defensa de dos minutos sobre por qué su caso la necesita.

2

Ecosistema nativo de nube

Recorra el mapa de proyectos de la Cloud Native Computing Foundation e identifique los que resolverían un problema real de su caso.

3

IA en el producto

Estudie el patrón de recuperación aumentada (RAG) y estime su costo por consulta antes de proponerlo en la oferta.

4

FinOps

Investigue las prácticas de etiquetado y control de gasto en la nube, y proponga un tope mensual acordado con el mandante.

5

Sostenibilidad

Busque cómo se estima la huella de una solución informática y qué medidas concretas la reducen. Es un diferenciador poco usado.

11

SECCIÓN 11

De la teoría a su propuesta

- Qué se evalúa exactamente en la arquitectura de su oferta
- Una ruta de siete pasos, del requisito al diagrama y del diagrama al costo
- Matriz de trazabilidad y ficha de parámetros a declarar
- Comparación de alternativas con criterios explícitos y ponderados
- Errores frecuentes y cómo presentar la arquitectura ante cada audiencia

Arquitectura de Software

Qué se evalúa exactamente

Resultado de Aprendizaje 1: construye la arquitectura lógica y física de la solución informática requerida en el caso asignado, considerando estándares tecnológicos vigentes y las restricciones organizacionales, económicas y ambientales del contexto.

CE 1.1 • Trazabilidad

Especifica los módulos, interfaces e integraciones de la arquitectura lógica, estableciendo la trazabilidad de cada componente con los requisitos declarados en las bases técnicas.

CE 1.2 • Dimensionamiento

Dimensiona los componentes de la arquitectura física y de la base de datos —servidores, redes, equipos de seguridad, tamaño, uptime y tiempos de respuesta— con parámetros verificables y consistentes con el volumen de operación previsto.

CE 1.3 • Estándares

Aplica estándares y buenas prácticas vigentes de la industria en las decisiones de arquitectura, señalando la referencia que respalda cada opción adoptada.

CE 1.4 • Restricciones

Identifica las restricciones organizacionales, económicas y ambientales que condicionan la solución, e indica de qué modo son incorporadas en el diseño.

Arquitectura de Software

Fundamentos

Lógica

Física

Nube

Calidad

Continuidad

Seguridad

Identidad

Entrega

Tendencias

Propuesta

La ruta, en siete pasos

1	Leer las bases	Extraer todos los requisitos, explícitos e implícitos.
2	Definir el alcance	Qué entra, qué no entra, qué supuestos se declaran.
3	Arquitectura lógica	Módulos, interfaces e integraciones, trazables a los requisitos.
4	Arquitectura física	Nodos, redes, seguridad y dimensionamiento con parámetros.
5	Comparar alternativas	Criterios explícitos, ponderados y aplicados de forma uniforme.
6	Justificar	Estándares, buenas prácticas y referencias en norma APA.
7	Costear	Traducir cada componente en inversión y costo de operación.

Arquitectura de Software

Pasos 1 y 2 · de las bases al alcance

Paso 1 · Leer las bases técnicas

- Numere cada requisito: RQ-01, RQ-02... Ese número es la clave de toda la trazabilidad posterior.
- Separe requisitos funcionales de atributos de calidad.
- Anote los volúmenes: usuarios, transacciones, documentos, sucursales, horarios de operación.
- Identifique los sistemas del mandante con los que hay que integrarse.
- Marque lo que no está dicho: si no se declara, hay que preguntarlo o declararlo como supuesto.

Paso 2 · Definir el alcance

- Escriba explícitamente qué queda dentro del contrato.
- Escriba explícitamente qué queda fuera: es la protección del proyecto y de su margen.
- Declare los supuestos con su fundamento: «se asume 10% de concurrencia en hora punta».
- Declare las dependencias del mandante: accesos, ambientes, datos, contrapartes.
- Todo lo que quede ambiguo aquí se transformará en conflicto durante la ejecución.



Un alcance bien delimitado es una decisión de arquitectura: define las fronteras del sistema, y por lo tanto qué se construye, qué se integra y qué se deja afuera.

Arquitectura de Software

Paso 3 · La arquitectura lógica

Objetivo: que cualquier persona entienda de qué partes se compone la solución y cómo se relacionan, sin necesidad de conocer la tecnología.

Diagrama de contexto

El sistema como una caja, rodeado de sus usuarios y de los sistemas externos con los que conversa. Una sola lámina, entendible por cualquiera.

Diagrama de módulos

El interior del sistema: los grandes bloques funcionales, con sus responsabilidades declaradas en una frase cada uno.

Interfaces

Qué expone cada módulo y qué consume. Con qué protocolo y con qué frecuencia. Aquí aparecen las APIs.

Integraciones

Cada sistema externo: qué dato se intercambia, en qué sentido, con qué periodicidad y qué pasa si no responde.

Modelo de datos conceptual

Las entidades principales y sus relaciones. No el modelo físico completo: las entidades del negocio.

Decisiones

El estilo elegido —monolito modular, capas, servicios— con las razones y las alternativas descartadas.

Prueba de calidad: si el diagrama menciona marcas de productos, todavía es arquitectura física disfrazada. La lógica se describe con funciones, no con proveedores.

Arquitectura de Software

La matriz de trazabilidad

Cada componente de su arquitectura debe poder responder: ¿qué requisito de las bases justifica que existas?

Requisito (bases técnicas)	Componente lógico	Nodo físico	Atributo de calidad comprometido
RQ-01 · Registro de solicitudes en línea	Módulo de Solicitudes + Portal Web	Servidores web y de aplicación	p95 < 2 s con 500 concurrentes
RQ-02 · Integración con el ERP del mandante	Servicio de Integración ERP	Servidor de integración	Reintento automático; cola de respaldo
RQ-03 · Firma electrónica de documentos	Módulo de Firma + proveedor externo	Servicio externo (SaaS)	Disponibilidad dependiente de tercero: declarada
RQ-04 · Reportería de gestión mensual	Módulo de Reportes + vista de lectura	Réplica de sólo lectura	No impacta la operación transaccional
RQ-05 · Trazabilidad y auditoría por 5 años	Servicio de Auditoría	Almacenamiento de objetos	Retención 60 meses; escritura inmutable
RQ-06 · Operación 24/7 con 99,5% mensual	Transversal	Dos zonas de disponibilidad	99,5% mensual; RTO 2 h; RPO 15 min

Doble lectura: hacia abajo verifica que no falte ningún requisito; hacia arriba verifica que no haya componentes que nadie pidió (y que nadie va a pagar).

Arquitectura de Software

Paso 4 · La ficha de parámetros

Estos son los parámetros verificables que exige el criterio de evaluación. Complételos todos, con su supuesto de origen.

Parámetro	Unidad	De dónde sale	Ejemplo
Usuarios totales	N.º	Bases técnicas del caso	12.000
Usuarios concurrentes en hora punta	N.º	Supuesto declarado sobre el total	1.200 (10%)
Transacciones por segundo	TPS	Concurrentes × operaciones por minuto ÷ 60	80
Tiempo de respuesta comprometido	s (p95)	Requisito de las bases o compromiso propio	< 2 s
Disponibilidad comprometida	%	Requisito de las bases; define la redundancia	99,5% mensual
RTO / RPO	h / min	Impacto de la interrupción para el negocio	2 h / 15 min
Volumen inicial de datos	GB	Registros × tamaño de fila + índices	180 GB
Crecimiento de datos	GB / mes	Transacciones mensuales × tamaño medio	12 GB/mes
Retención de información	meses	Normativa aplicable y política del mandante	60 meses

Arquitectura de Software

Paso 4 · La ficha de parámetros · continuación 1

Parámetro	Unidad	De dónde sale	Ejemplo
Ancho de banda requerido	Mbps	Tamaño medio de respuesta × TPS	45 Mbps
Ventana de mantenimiento	h / mes	Acuerdo con el mandante; se excluye del SLA	4 h mensuales, domingo de madrugada
Horizonte de evaluación	años	Definido para la evaluación económica	5 años

Arquitectura de Software

Paso 5 · Comparar alternativas

Construya un cuadro comparativo con criterios explícitos, pondérelos según lo que el caso hace importante y aplíquelos de manera uniforme a todas las alternativas.

Criterio	Peso	A · On-premise	B · Nube pública	C · Híbrido
Inversión inicial	25%	2	5	3
Costo total a 5 años	20%	3	4	4
Cumplimiento de residencia de datos	20%	5	3	5
Elasticidad ante estacionalidad	15%	1	5	4
Tiempo de puesta en marcha	10%	2	5	3
Capacidad de operación del mandante	10%	3	4	3
TOTAL PONDERADO	100%	2,75	4,25	3,85

Escala declarada: 1 = muy desfavorable, 5 = muy favorable. Los pesos se justifican con el caso, no con la preferencia del equipo. Y la decisión final puede no ser el puntaje más alto: si eligen otra, expliquen por qué —eso también se evalúa.

Arquitectura de Software

Paso 6 · Justificar con estándares y referencias

«Lo elegimos porque nos pareció mejor» no es un fundamento. «Lo elegimos siguiendo X, que establece Y» sí lo es.

Tipo de fuente	Para qué sirve	Ejemplos de uso en la oferta
Normas y estándares	Respaldar decisiones de calidad, seguridad y gestión.	Modelo de calidad de producto de software; normas de seguridad de la información; marcos de gestión de servicios de TI.
Normativa aplicable	Fundar restricciones del contexto y obligaciones del diseño.	Ley de protección de datos personales; normativa sectorial del mandante; bases de licitación.
Documentación de proveedores	Sustentar dimensionamiento, precios y niveles de servicio.	Calculadoras de costo, acuerdos de nivel de servicio publicados, guías de arquitectura de referencia.
Marcos de buenas prácticas	Ordenar decisiones de arquitectura y de operación.	Marcos de arquitectura bien diseñada, guías de arquitectura nativa de nube, patrones de integración.
Literatura técnica	Fundamentar patrones y sus consecuencias.	Libros y artículos sobre patrones de arquitectura, sistemas distribuidos y microservicios.

Toda referencia debe ser verificable y estar citada en norma APA 7.ª ed., tanto en el texto como en el listado final.

Arquitectura de Software

Paso 7 · Las restricciones del contexto

No basta con identificarlas: hay que indicar de qué modo son incorporadas en el diseño.

Tipo de restricción	Ejemplos en un caso TIC	Cómo se incorpora en el diseño
Organizacional	Capacidades del área de TI del mandante; políticas internas; sistemas heredados; disponibilidad de contrapartes.	Se elige un servicio gestionado para no exigir perfiles que el mandante no tiene.
Económica	Presupuesto de inversión; presupuesto anual de operación; forma de pago; horizonte de evaluación.	Se prefiere un perfil de gasto operacional para evitar el desembolso inicial.
Legal y normativa	Protección de datos personales; residencia de los datos; requisitos de auditoría y retención.	La región de despliegue y la política de retención se definen por la normativa.
Ambiental	Consumo energético; ciclo de vida del hardware; disposición del equipamiento reemplazado.	Autoescalado y apagado de ambientes no productivos; retención acotada de datos.
Técnica	Estándares del mandante; protocolos de integración obligatorios; conectividad disponible.	Las interfaces se diseñan según el estándar exigido, no según la preferencia del equipo.
Temporal	Plazo de implementación; hitos contractuales; ventanas de cambio del mandante.	Se prefiere una arquitectura simple y una entrega por fases antes que una solución elegante e imposible de terminar.

Arquitectura de Software

De la arquitectura al flujo de caja

Regla de consistencia: todo componente del diagrama tiene una línea en el presupuesto, y toda línea del presupuesto tiene un componente en el diagrama.

Componente de la arquitectura	Inversión (año 0)	Costo de operación (anual)
Servidores / instancias de cómputo	Compra de hardware o nada, si es nube	Arriendo mensual o energía, soporte y renovación
Motor de base de datos	Licencia inicial	Mantenimiento de licencia o servicio gestionado
Almacenamiento y respaldo	Cabina o volumen inicial	Crecimiento mensual + retención + pruebas de restauración
Red y seguridad perimetral	Firewall, WAF, certificados	Renovaciones, enlaces, actualizaciones de reglas
Observabilidad	Configuración inicial	Volumen de registros y métricas ingeridas
Ambientes no productivos	Habilitación	Consumo de desarrollo, pruebas y capacitación
Operación y soporte	Traspaso y documentación	Horas de operación, turnos y plan de soporte
Licencias de software base	Sistemas operativos, herramientas	Suscripciones y actualizaciones

Este cuadro es el insumo directo de la estructura de costos y del flujo de caja que construirán en la unidad de evaluación económica.

Arquitectura de Software

Errores frecuentes que hunden una propuesta

Un solo diagrama para todo

Se entrega una lámina que mezcla módulos con servidores. No es ni arquitectura lógica ni física.

Cifras sin origen

«Se requieren 4 servidores» sin explicar de dónde salió el 4. El evaluador no puede verificarlo.

Tecnología por moda

Microservicios, contenedores e IA en un proyecto de cuatro meses con cinco personas.

SLA imposible

Se compromete 99,99% con un servidor y respaldo diario. Es una multa diferida.

Sin alternativas comparadas

Se presenta una solución sin mostrar qué se evaluó ni por qué se descartó lo demás.

Componentes que nadie pidió

Módulos que no responden a ningún requisito: aumentan el costo y bajan la competitividad.

Integraciones sin detalle

Una flecha que dice «se integra con el ERP» sin protocolo, frecuencia ni plan ante falla.

Arquitectura sin costo

El capítulo técnico y el económico no se corresponden. Es la inconsistencia que más se castiga.

Arquitectura de Software

Cómo presentar la arquitectura

Audiencia técnica

- Muestre el diagrama de despliegue y las cifras de dimensionamiento.
- Explique el punto único de falla que eliminó y el que decidió aceptar.
- Use los términos correctos: p95, RTO, RPO, zona de disponibilidad.
- Tenga a mano el detalle de las integraciones y los protocolos.
- Prepare la respuesta a «¿y si se cae X?» para cada componente.

Audiencia comercial y directiva

- Muestre el diagrama de contexto, no el de despliegue.
- Traduzca a consecuencias: «el servicio sigue operando aunque falle un centro de datos».
- Hable de costo total, de riesgo y de plazo, no de tecnología.
- Una sola cifra memorable por atributo: disponibilidad, tiempo de respuesta, costo mensual.
- Cierre con qué recibe el mandante y qué se compromete por contrato.



Tiene 15 minutos para toda la propuesta. La arquitectura no debería tomar más de 4 o 5: dos diagramas, tres cifras y una decisión bien justificada valen más que diez láminas.

Arquitectura de Software

Qué hacer esta semana

Cinco productos concretos para llegar con la arquitectura resuelta a la próxima instancia de validación.

1

Requisitos numerados

Listado RQ-01, RQ-02... extraído de las bases técnicas de su caso, separando funcionales de atributos de calidad.

2

Dos diagramas

Uno de contexto y uno de módulos para la arquitectura lógica; uno de despliegue para la física. Distintos y consistentes entre sí.

3

Ficha de parámetros

La tabla completa, con cada cifra y su supuesto de origen declarado.

4

Matriz de decisión

Al menos una comparación ponderada: on-premise vs nube, o el estilo arquitectónico elegido.

5

Tres ADR

Las tres decisiones más importantes, con contexto, alternativas, fundamento, consecuencias y su referencia.

Traiga estos cinco productos a la próxima sesión. Sobre ellos vamos a trabajar la estimación, la planificación y la estructura de costos: sin arquitectura definida, no hay nada que estimar.

Arquitectura de Software

Estimar el costo de la solución · el método

La estimación no se adivina: se construye a partir del diagrama físico, componente por componente, con una unidad de medida declarada para cada uno.

1

1 · Listar los componentes

Tome el diagrama físico y escriba cada elemento en una fila: cómputo, base de datos, red, almacenamiento, seguridad, monitoreo, respaldo.

2

2 · Definir la unidad

Para cada componente, en qué se mide: horas de instancia, vCPU-hora, GB al mes, millón de solicitudes, GB transferidos.

3

3 · Cuantificar

Cuántas unidades al mes, derivadas del volumen de operación del caso y de los supuestos declarados.

4

4 · Multiplicar por ambientes

Producción, certificación, testing y desarrollo. Cada uno consume, y todos van en el presupuesto.

5

5 · Agregar lo transversal

Plan de soporte del proveedor, transferencia de datos, monitoreo y respaldo. Es lo que siempre se olvida.

6

6 · Proyectar

Del costo mensual al horizonte de evaluación: doce meses, cinco años, crecimiento de la demanda y contingencia.

Arquitectura de Software

Las unidades de medida

Estas son las unidades en que los proveedores cobran. Cada línea de su presupuesto tiene que estar expresada en una de ellas.

Unidad	Qué mide	Dónde aparece
GB · TB	Gigabyte y Terabyte: volumen de datos almacenados o transferidos.	Almacenamiento, respaldo, transferencia
IDT · Inbound Data Transfer	Datos que entran a la nube. Habitualmente no se cobra.	Red virtual, almacenamiento
IRDT · Intra Region Data Transfer	Tráfico entre recursos dentro de la misma región, por ejemplo entre dos zonas de disponibilidad.	Red virtual; es el costo oculto de la alta disponibilidad
ODT · Outbound Data Transfer	Datos que salen hacia Internet. Es la transferencia que sí se cobra, y suele sorprender.	Red virtual, contenido descargado por usuarios
vCPU-hora	Un procesador virtual durante una hora. Unidad de cobro de los contenedores sin servidor.	Fargate, Container Apps
GB-hora	Un gigabyte de memoria durante una hora. Acompaña siempre a la vCPU-hora.	Fargate, Container Apps
Hora de instancia	Una máquina virtual encendida durante una hora, del tamaño contratado.	EC2, Azure VM, base de datos gestionada

Arquitectura de Software

Las unidades de medida · continuación 1

Unidad	Qué mide	Dónde aparece
Millón de solicitudes	Unidad de cobro de las pasarelas de API y de las funciones.	API Gateway, Lambda
ACL · Access Control List	Lista de control de acceso: conjunto de reglas del firewall de aplicación. Se cobra por lista y por regla.	WAF
Métrica · panel · alarma	Unidades de cobro del servicio de monitoreo: cada métrica personalizada, cada tablero y cada alarma suman.	CloudWatch, Azure Monitor
Evento registrado	Cada acción sobre la plataforma que queda auditada. Se cobra por millón de eventos.	CloudTrail
Pod o tarea	Una unidad de ejecución de contenedor, con su CPU y memoria asignadas.	Fargate, Kubernetes

Arquitectura de Software

El caso de ejemplo

Una solución en contenedores con cuatro ambientes, base de datos gestionada, pasarela de API y los servicios transversales de seguridad y monitoreo.

Componente de la solución	Dimensionamiento declarado
Base de datos productiva	PostgreSQL gestionado, 4 vCPU / 16 GB RAM, 30 GB de almacenamiento, multizona, 100% del mes encendida
Base de datos de certificación y testing	PostgreSQL gestionado, 2 vCPU / 8 GB RAM, 30 GB, dos nodos, una sola zona
Ambiente productivo	7 tareas de 0,25 vCPU / 0,5 GB y 5 tareas de 0,5 vCPU / 1 GB, 20 GB cada una, 30 días al mes
Ambiente de testing	6 tareas de 0,25 vCPU / 0,5 GB y 5 tareas de 0,5 vCPU / 1 GB
Ambiente de certificación	6 tareas de 0,25 vCPU / 0,5 GB y 5 tareas de 0,5 vCPU / 1 GB
Gestor de ambientes	2 tareas de 0,5 vCPU / 1 GB
Servicios de comunicaciones	2 tareas de 1 vCPU / 2 GB y 1 tarea de 4 vCPU / 8 GB
Pasarela de API	1 millón de solicitudes REST al mes, más canales de tiempo real
Firewall de aplicación	2 listas de control de acceso, con sus reglas y grupos de reglas
Registro de imágenes	50 GB de almacenamiento mensual y 1 GB de descarga
Monitoreo y auditoría	50 métricas, 1 tablero, 1 alarma; 3 millones de eventos de escritura y de lectura
Almacenamiento de objetos	Tres depósitos: 50 GB, 100 GB y 30 GB mensuales, con su transferencia asociada

Arquitectura de Software

Escenario 1 · contenedores sin servidor, una zona

Solución sobre contenedores sin servidor, desplegada en una sola región y una sola zona de disponibilidad.

Servicio	Qué incluye	USD / mes
Plan de soporte del proveedor	Soporte con tiempos de respuesta comprometidos; se cobra como porcentaje del gasto	171,47
Base de datos productiva	PostgreSQL gestionado 4 vCPU / 16 GB, multizona, bajo demanda	583,60
Bases de certificación y testing	PostgreSQL gestionado 2 vCPU / 8 GB, dos nodos, zona única	316,42
Contenedores · ambiente productivo	7 tareas pequeñas + 5 tareas medianas	151,08
Contenedores · testing	6 tareas pequeñas + 5 tareas medianas	142,19
Contenedores · certificación	6 tareas pequeñas + 5 tareas medianas	142,19
Contenedores · gestor de ambientes	2 tareas medianas	17,78
Contenedores · comunicaciones	2 tareas de 1 vCPU + 1 tarea de 4 vCPU	213,28
Transferencia de datos de la red virtual	Cuatro tramos de red, con su tráfico de entrada, interno y de salida	64,35
Pasarela de API	1 millón de solicitudes REST y canales de tiempo real	46,02

Arquitectura de Software

Escenario 1 · contenedores sin servidor, una zona · continuación 1

Servicio	Qué incluye	USD / mes
Firewall de aplicación	2 listas de control con sus reglas	16,80
Registro de imágenes	50 GB almacenados, 1 GB descargado	5,02
Monitoreo	50 métricas, 1 tablero, 1 alarma	16,00
Auditoría de la plataforma	3 millones de eventos de escritura y de lectura	21,50
Almacenamiento de objetos	Tres depósitos de 50, 100 y 30 GB con su transferencia	20,69
TOTAL MENSUAL	Todos los ambientes incluidos	1.928,39

Arquitectura de Software

Cómo se calcula una línea · las tareas de contenedor

Toda línea del presupuesto es una multiplicación. Ésta es la de las tareas de contenedor, que es la más representativa.

Paso	Dato	Resultado
Tamaño de la tarea	0,25 vCPU y 0,5 GB de memoria	Es la unidad más pequeña que se puede contratar
Tiempo encendida	1 tarea durante 30 días, las 24 horas	720 horas al mes
Costo unitario mensual	vCPU-hora × 720 + GB-hora × 720	≈ 8,89 USD por tarea al mes
Cantidad de tareas	7 tareas de este tamaño en el ambiente productivo	$8,89 \times 7 = 62,23$ USD
Tarea mediana	0,5 vCPU y 1 GB, es decir el doble de recursos	≈ 17,77 USD por tarea al mes
Cantidad de tareas medianas	5 tareas en el ambiente productivo	$17,77 \times 5 = 88,85$ USD
Total del ambiente productivo	62,23 + 88,85	151,08 USD al mes



Observe la proporción: doble de CPU y memoria, doble de precio. Esa linealidad es lo que permite estimar sin tener la solución construida —y lo que permite al evaluador verificar su cifra.

Los valores unitarios cambian con el tiempo y con la región: lo que no cambia es el procedimiento.

Arquitectura de Software

Escenario 2 · máquinas virtuales, una zona

La misma solución, pero ejecutada sobre nodos propios que hay que dimensionar, administrar y parchar.

Servicio	Qué incluye	USD / mes
Plan de soporte del proveedor	Sube respecto del escenario anterior porque el gasto total es mayor	361,64
Base de datos productiva	PostgreSQL gestionado 4 vCPU / 16 GB, multizona	583,60
Bases de certificación y testing	PostgreSQL gestionado 2 vCPU / 8 GB, dos nodos	316,42
Nodo productivo 1	8 vCPU / 32 GB / 300 GB de disco	279,81
Nodo productivo 2	Mismo equipo, pero contratado en otra región	1.623,81
Nodo de testing y certificación	8 vCPU / 32 GB / 300 GB	279,81
Coordinadores del clúster	Tres nodos de 4 vCPU / 8 GB / 100 GB	446,76
Transferencia de datos de la red virtual	Comunicaciones, portales y gestión de ambientes	64,35
Firewall de aplicación y registro de imágenes	Mismas reglas y mismo registro que el escenario anterior	21,82
TOTAL MENSUAL	Todos los ambientes incluidos	3.978,02

Lección de la quinta línea: el mismo nodo, con la misma especificación, cuesta casi seis veces más por estar en otra región. Revise siempre la columna de región antes de firmar una estimación.

Arquitectura de Software

Los cuatro escenarios, comparados

La misma solución, estimada de cuatro formas. Ésta es la tabla que va en la comparación de alternativas del informe.

Escenario	Cómputo	Zonas	USD / mes	USD / año	Diferencia
1	Contenedores sin servidor	Una zona	1.928	23.141	Referencia
2	Contenedores sin servidor	Dos zonas	2.590	31.085	+34%
3	Nodos propios (máquinas virtuales)	Una zona	3.978	47.736	+106%
4	Nodos propios (máquinas virtuales)	Dos zonas	5.062	60.745	+163%

Los contenedores sin servidor salieron a la mitad

En este caso, no administrar nodos cuesta la mitad que administrarlos —y además elimina las horas de operación del sistema operativo, que no están en esta tabla.

La segunda zona cuesta un tercio más

Ése es el precio concreto de subir de una zona a dos. Es exactamente la cifra que hay que poner al lado del compromiso de disponibilidad.

El soporte crece con el gasto

El plan de soporte pasó de 171 a 460 dólares entre el escenario más barato y el más caro: se cobra como porcentaje del consumo.

Arquitectura de Software

El escenario de peor caso

Una estimación seria no entrega un solo número: entrega el caso base y el techo. Así se modela el techo.

Columna de la planilla	Qué representa	Ejemplo
Costo mensual	El gasto del escenario base que entrega la calculadora.	62,23 USD por 7 tareas
Cantidad	Cuántas unidades componen la línea: tareas, nodos, instancias.	7 tareas
Costo mensual unitario	El costo dividido por la cantidad. Es lo que cuesta agregar una unidad más.	$62,23 \div 7 = 8,89$ USD
Valor de levantamiento	Lo que cuesta poner en marcha una unidad adicional cuando sube la demanda.	0,29 USD por tarea
Factor	Cuántas veces podría multiplicarse esa cantidad en el peor escenario previsto.	50 veces
Peor caso	$\text{Cantidad} \times \text{costo unitario} + \text{valor de levantamiento} \times \text{factor} \times \text{cantidad}.$	El techo del gasto de esa línea

Por qué importa en una licitación: si el contrato es a precio fijo, el peor caso es su riesgo. Si es a precio variable, es el riesgo del mandante y él querrá un tope. En ambos casos hay que tener la cifra calculada antes de firmar.

Arquitectura de Software

De dólares al mes al flujo de caja

La calculadora del proveedor entrega dólares al mes. El flujo de caja necesita seis cosas más.

Moneda y tipo de cambio

Declare la moneda del contrato y el tipo de cambio o la unidad de reajuste que usará. Si cobra en pesos y paga en dólares, el riesgo cambiario es suyo.

Horizonte de evaluación

Proyecte los mismos meses que el resto de la evaluación. Doce meses no bastan si el horizonte son cinco años.

Crecimiento de la demanda

El gasto en la nube crece con el uso. Modele el crecimiento anual de usuarios y de datos: el almacenamiento nunca baja.

Puesta en marcha

Migración de datos, configuración inicial, certificaciones y horas de ingeniería. Es inversión del año cero, no gasto mensual.

Operación humana

Las horas de quien administra la plataforma no están en la calculadora. En el escenario de nodos propios son bastantes más.

Contingencia

Un porcentaje declarado sobre el total, justificado por la incertidumbre de la estimación. Entre 10% y 20% es defendible.



Consistencia: el total mensual de esta planilla debe ser exactamente la línea «servicios de nube» de su flujo de caja. Si los números no coinciden, el evaluador lo va a notar.

Arquitectura de Software

Errores frecuentes al estimar

Cotizar sólo producción

Los ambientes de desarrollo, testing y certificación suman entre un tercio y la mitad del gasto. En este ejemplo, más de la mitad de las tareas son de ambientes no productivos.

Olvidar la transferencia de salida

Los datos que salen hacia Internet se cobran. En soluciones con documentos o video, esta línea puede ser la más grande.

Ignorar el plan de soporte

Es un porcentaje del gasto total. Aparece en la factura desde el primer mes y nadie lo presupuesta.

No revisar la región

El mismo equipo puede costar varias veces más según dónde se contrate. Verifique la columna de región en cada línea.

Dejar fuera el monitoreo

Métricas, tableros, alarmas, registros y auditoría se cobran por volumen. Sin ellos no hay SLA, así que no son opcionales.

Estimar un solo escenario

Presente al menos dos alternativas de cómputo y el peor caso. Es lo que pide el criterio de comparación de alternativas.

Confundir mensual con anual

Multiplicar por doce parece obvio, pero es el error aritmético más común en los informes.

No guardar la evidencia

Las calculadoras generan un enlace permanente a la estimación. Guárdelo y cítelo: es evidencia verificable.

Arquitectura de Software

Qué debe entregar en el informe

Cinco entregables que cierran el capítulo de costos de infraestructura de la oferta.

Planilla de estimación

Una fila por componente, con la unidad de medida, la cantidad, el supuesto que la origina y el costo mensual.

Al menos dos escenarios

Dos alternativas de cómputo comparadas sobre la misma solución y el mismo volumen, con la diferencia porcentual.

Escenario de peor caso

El techo de gasto con el factor de crecimiento declarado, y qué pasa con el contrato si se alcanza.

Proyección al horizonte

El costo mensual convertido a la moneda y al horizonte de la evaluación, con crecimiento y contingencia.

Enlace a la estimación

La referencia verificable de la calculadora del proveedor, citada en el informe según norma APA.

Regla de cierre: cada caja de su diagrama físico debe tener una fila en la planilla, y cada fila de la planilla debe tener una caja en el diagrama. Si algo aparece en uno y no en el otro, hay un error en alguno de los dos.

Arquitectura de Software

Glosario 1 / 8

Término	Definición
Activo – Activo	Modalidad en que dos nodos o sitios atienden tráfico simultáneamente; si uno cae, el otro absorbe su carga.
Activo – Pasivo	Modalidad en que sólo un nodo atiende tráfico y el otro espera en reserva para tomar el relevo.
Alta disponibilidad	Capacidad del sistema de seguir prestando servicio aunque falle uno de sus componentes, sin interrupción ni intervención manual.
Ambiente	Instalación completa de la solución destinada a un propósito: desarrollo, QA, preproducción o producción.
Anonimización	Transformación de datos personales para que dejen de identificar a una persona. Obligatoria en ambientes no productivos.
API	Interfaz de programación de aplicaciones. Definiciones y protocolos que permiten a dos componentes de software comunicarse.
API Gateway	Punto único de entrada que recibe las llamadas a las APIs, las autentica, las enruta y agrega los resultados.
Arquitectura física	Vista que muestra la ubicación del software en el hardware: nodos, redes, capacidad y ubicación geográfica.

Arquitectura de Software

Glosario 2 / 8

Término	Definición
Arquitectura lógica	Vista que muestra módulos, interfaces e integraciones del software, con independencia de la tecnología que los ejecuta.
Artefacto	Paquete versionado producido por la construcción —imagen de contenedor o binario— que se promueve entre ambientes.
Autoescalado	Ajuste automático de la cantidad de instancias según una métrica de carga o un horario.
Azul – verde	Estrategia de despliegue que levanta el entorno nuevo completo en paralelo y conmuta el tráfico de una sola vez.
Balanceador de carga	Componente que reparte peticiones entre varios nodos y retira de rotación al que no responde.
Base de datos	El conjunto de datos y su estructura —tablas, índices, relaciones— que el motor administra.
Canario	Estrategia de despliegue que envía un pequeño porcentaje del tráfico a la versión nueva y la amplía si las métricas se mantienen.
CAPEX	Gasto de capital: inversión inicial en activos como hardware y licencias perpetuas. Se deprecia.

Arquitectura de Software

Glosario 3 / 8

Término	Definición
CDN	Red de distribución de contenido. Nodos distribuidos que entregan contenido estático desde el punto más cercano al usuario.
CI / CD	Integración continua y entrega o despliegue continuo: automatización del camino desde el cambio de código hasta producción.
Consistencia eventual	Propiedad por la cual los datos replicados quedan consistentes después de un tiempo, no de inmediato.
Contenedor	Paquete estándar que agrupa una aplicación con sus bibliotecas y dependencias, para ejecutarse igual en cualquier entorno.
DMZ	Zona desmilitarizada: segmento de red intermedio entre Internet y la red interna donde se publican los servicios expuestos.
Doble factor (MFA)	Autenticación que exige un segundo elemento además de la contraseña. Obligatoria para acceso remoto y administradores.
Elasticidad	Capacidad de asignar y retirar recursos de forma automática, respondiendo de forma flexible a la demanda.
Endpoint	URL específica donde una API recibe solicitudes.

Arquitectura de Software

Glosario 4 / 8

Término	Definición
Escalamiento horizontal	Agregar más nodos a la solución para repartir la carga entre ellos.
Escalamiento vertical	Agregar más recursos —procesador, memoria— al mismo equipo.
Evento	Registro de un hecho de negocio que ocurrió. Se publica para que otros componentes reaccionen a él.
Failover / Failback	Conmutación del servicio al sitio alternativo ante una falla, y posterior regreso al sitio original.
FinOps	Práctica de gobernar el gasto en la nube: visibilidad, asignación por responsable, optimización y previsión.
GFS	Esquema de retención de respaldos abuelo-padre-hijo: diarias, semanales, mensuales y anuales con distinta permanencia.
IaaS	Infraestructura como servicio: cómputo, almacenamiento y red entregados como servicio, con pago por uso.
IaC	Infraestructura como código: definir la infraestructura en archivos versionados en vez de configurarla manualmente.

Arquitectura de Software

Glosario 5 / 8

Término	Definición
Idempotencia	Propiedad por la cual repetir una operación produce el mismo resultado que ejecutarla una sola vez.
Inmutabilidad (respaldo)	Copia que no puede borrarse ni alterarse durante un período definido, ni siquiera por un administrador.
Latencia	Tiempo que tarda una operación en responder.
Microservicio	Servicio pequeño y autónomo que implementa una capacidad de negocio y se despliega de forma independiente.
Modelo OSI	Modelo de referencia que describe la comunicación en red en siete capas, de la física a la de aplicación.
Monolito	Aplicación cuyo código y funcionalidades están acoplados en un único paquete desplegable.
Motor de base de datos	El software que administra los datos: recibe consultas, controla concurrencia y garantiza integridad. Tiene licencia.
Multi-primario	Topología de replicación en que varios nodos aceptan escrituras. Habilita activo-activo, pero introduce conflictos.

Arquitectura de Software

Glosario 6 / 8

Término	Definición
Observabilidad	Capacidad de entender el estado interno del sistema desde fuera, combinando métricas, registros y trazas.
OPEX	Gasto operacional: costo recurrente de operar la solución. No se deprecia; se imputa al ejercicio.
PaaS	Plataforma como servicio: entorno gestionado donde se despliegan aplicaciones sin administrar el servidor.
Percentil 95 (p95)	Valor bajo el cual responde el 95% de las peticiones. Medida realista del tiempo de respuesta comprometido.
Punto único de falla	Componente cuya caída deja fuera de servicio a todo el sistema.
Quórum	Esquema en que una escritura se confirma cuando la acepta la mayoría de los nodos. Evita la partición de cerebro.
Región / Zona de disponibilidad	Región: ubicación geográfica de centros de datos. Zona: centro de datos aislado dentro de una región.
Regla 3-2-1	Política de respaldo: tres copias de los datos, en dos medios distintos, con una fuera del sitio.

Arquitectura de Software

Glosario 7 / 8

Término	Definición
Replicación asincrónica	La transacción se confirma localmente y se envía después al otro sitio. No penaliza la latencia; el RPO es mayor que cero.
Replicación sincrónica	La transacción se confirma sólo cuando ambos sitios escribieron. RPO cero, a costa de la latencia del enlace.
REST	Estilo de API sobre HTTP que usa métodos estándar —GET, POST, PUT, DELETE— para operar sobre recursos.
RTO / RPO	Tiempo máximo aceptable de interrupción / cantidad máxima aceptable de datos perdidos, medida en tiempo.
SaaS	Software como servicio: aplicación entregada como servicio bajo demanda, normalmente vía navegador.
Serverless	Modelo en que se ejecutan funciones sin administrar servidores y se paga sólo por el tiempo de ejecución.
SLA / SLO / SLI	Acuerdo contractual de nivel de servicio / objetivo interno / indicador que se mide.
Split-brain	Situación en que se corta el enlace entre sitios y ambos se creen activos, aceptando escrituras y divergiendo.

Arquitectura de Software

Glosario 8 / 8

Término	Definición
Storage de bloque	Disco crudo que el sistema operativo formatea y monta. Es el almacenamiento propio de las bases de datos.
Storage de objetos	Almacenamiento accesible por API HTTP, con capacidad casi ilimitada y bajo costo. Para documentos, imágenes y respaldos.
Throughput	Cantidad de operaciones que el sistema atiende por unidad de tiempo.
Uptime / Downtime	Tiempo en que el sistema opera sin interrupciones / tiempo en que no está operativo o es inaccesible.
VPC	Red virtual privada: red aislada dentro de la nube donde se despliegan los recursos, con control de subredes y rutas.
VPN	Túnel cifrado que conecta un equipo remoto a la red corporativa. Da acceso a la red completa: por eso se prefiere Zero Trust.
WAF	Firewall de aplicación web: filtra tráfico malicioso de capa 7 antes de que llegue a los servidores.
Zero Trust	Enfoque en que ninguna red es confiable por sí sola: cada acceso se autentica y autoriza según identidad, dispositivo y recurso.

Arquitectura de Software

Referencias

Fuentes de apoyo de esta unidad. En el informe, cite en norma APA 7.^a edición.

- ISO/IEC 25010:2023. Systems and software engineering — SQuaRE — Product quality model. Organización Internacional de Normalización. <https://iso25000.com/index.php/normas-iso-25000/iso-25010>
- Ley N° 21.719, que regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. Diario Oficial de la República de Chile, 13 de diciembre de 2024. Entrada en plena vigencia: 1 de diciembre de 2026.
- ISO/IEC 7498-1. Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model (modelo OSI de siete capas).
- OWASP Top 10. Riesgos de seguridad más críticos en aplicaciones web. <https://owasp.org/Top10>
- Kruchten, P. Architectural Blueprints — The 4+1 View Model of Software Architecture.
- Modelo C4 para la documentación de arquitecturas de software. <https://c4model.com>
- Estilos arquitectónicos: microservicios. <https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/microservicios>
- Diagramas de arquitectura de microservicios. <https://www.edrawsoft.com/es/article/microservices-architecture-diagram.html>
- Guías de arquitectura de referencia y marcos de buenas prácticas publicados por los proveedores de nube (arquitecturas bien diseñadas, patrones de aplicaciones distribuidas).
- Cloud Native Computing Foundation: ecosistema de herramientas y mejores prácticas para arquitecturas nativas de la nube. <https://www.cncf.io>

Arquitectura de Software

Dibuje la suya. Póngale números. Después defiéndala.

Antonio Moya Villegas · antonio.moya@pucv.cl · ICI-5444 · 2026